REST API: Проектирование

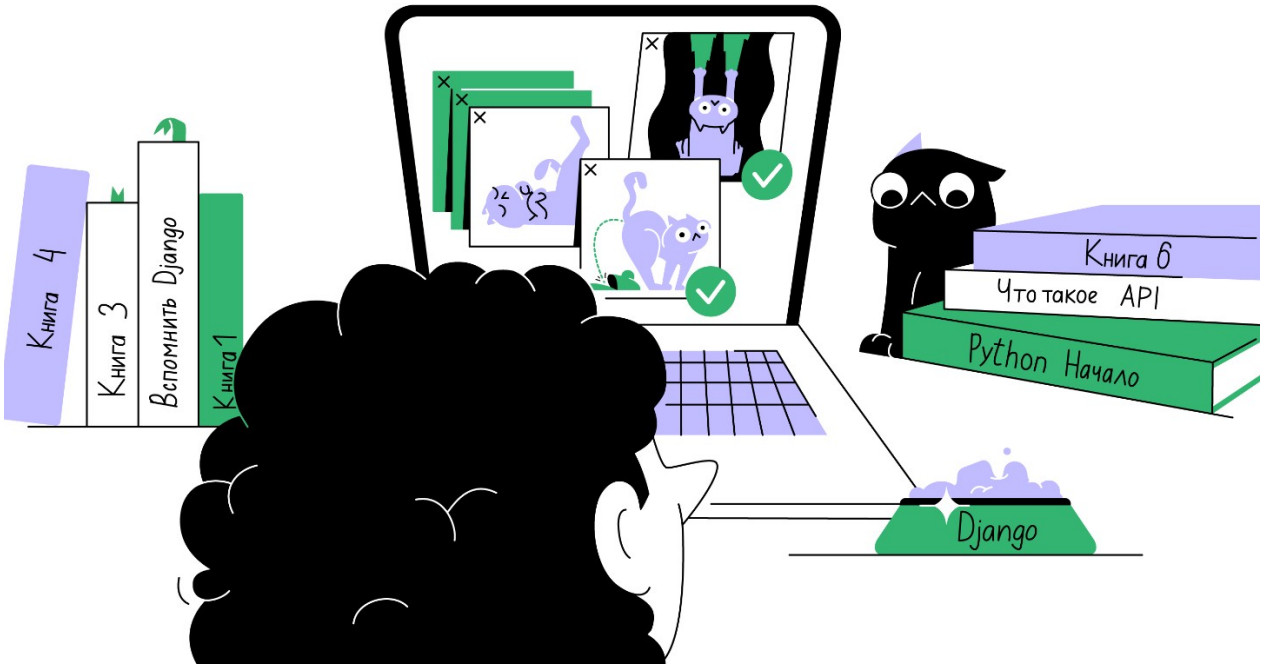
Вы уже поработали с внешними API и научились настраивать взаимодействие между ними в рамках одного проекта. Пришло время посмотреть на программные интерфейсы с другой стороны: написать API для собственного проекта.

На сегодняшний день самый актуальный и востребованный подход к созданию таких интерфейсов — это архитектура **REST**, **RE**presentational **S**tate **T**ransfer.

REST API можно написать на чистом Python, но для Django-проекта можно подключить библиотеку **Django REST Framework**, которая ускорит и упростит разработку REST API.

Django REST Framework (**DRF**) предоставляет разработчику весь необходимый набор инструментов для создания REST-сервисов на основе Django. По сути, **DRF** — это коллекция предустановленных классов. **DRF** даёт инструменты для решения штатных задач, возникающих при создании REST API.

На основе **DRF** можно минимальным количеством кода преобразовать любое Django-приложение в REST API.



А в чём польза?

Вы получите навыки, необходимые для создания классического REST API для Django-проекта. Подобная задача часто встречается на практике, и такой опыт точно вам пригодится.
Проектирование API — важная часть разработки и хорошая инвестиция в успешный результат. Соблюдение принципов проектирования поможет сделать API современным и удобным в работе. Начнём с принципов **консистентности** и **расширяемости**.

Консистентность

Консистентность — это согласованность данных друг с другом, их целостность и внутренняя непротиворечивость. Например, данные о каком-то объекте, полученные с одного эндпоинта, не должны отличаться от данных о том же объекте, но полученных с другого эндпоинта.

Другой случай соблюдения принципа консистентности: одинаковые типы данных должны быть описаны одинаково, где бы они ни использовались.

Рассмотрим работу API на примере модели Post: такая модель в проекте блога может описывать пост определённого пользователя.

Скопировать код
PYTHON

```python
# Тестовая модель
class Post(models.Model):
    text = models.TextField()
    group = models.ForeignKey(Group, on_delete=models.SET_NULL, blank=True, null=True)
    pub_date = models.DateTimeField('Дата публикации', auto_now_add=True)
    author = models.ForeignKey(User, on_delete=models.CASCADE)
```

В этой модели есть поле `DateTimeField`, данные из него будут возвращаться в ответах. Но JSON позволяет передавать дату в любом виде, и разработчик сам решает, какой формат даты выбрать.

«Пусть это будет unix-timestamp», — решил разработчик:

Скопировать код

```
# GET запрос поста с id=10
```

```
GET /api/v1/posts/10/


# Ответ API
[[10, "Это текст из моего поста.", 1618567801, 1]]
```

Когда формат даты выбран — в дело вступает принцип консистентности, который предполагает, что и все другие запросы должны возвращать дату именно в этом формате.

К API отправлен GET-запрос для получения первого комментария к посту с id=10

Скопировать код

```
GET /api/v1/posts/10/comments/1/
```

И если в ответе API дата возвращается в формате строки — то консистентность нарушена:

Скопировать код
JSON

```
# Так нельзя: договаривались, что дату возвращаем в виде unix-timestamp!
[[1, "Это мой первый комментарий.", "2020-03-23T18:02:33.123543Z", 10]]
```

При запросе комментария дата возвращается строкой. Это непорядок, надо переделать.

Если же разработчик не забыл о консистентности — ответ API будет таким:

Скопировать код
JSON

```
# Ответ API с датой в формате unix-timestamp
[[1, "Это мой первый комментарий.", 1618565516, 10]]
```

Вот теперь разработчик молодец.

Где консистентность?

Разработчикам заказали API для геолокационного сервиса логистической компании. Тимлид определил формат, в котором должны передаваться географические координаты:

- координаты передаются в словаре;
- координаты передаются под ключами `lat` (*latitude*, широта) и `lng` (*longitude*, долгота);
- значения координат передаются в формате *float*.

API обрабатывает GET-запросы к нескольким эндпойнтам:

- /towns/<id_города>/ — возвращает координаты запрошенного города и численность населения;
- /towns/<id_города>/what_around/ — возвращает координаты трёх крупнейших городов в радиусе 100 км вокруг запрошенного города;
- /towns/<id_города>/where_capital/ — возвращает координаты столицы той страны, где находится запрошенный город.
- /towns/<id_города>/warehouse/ — возвращает координаты и площадь склада, ближайшего к запрошенному городу.

API готов, тестировщик отправляет GET-запросы и проверяет ответы. Для тестов выбран город с ID=10, это небольшой уэльсский городок *Лланвайрпуллгуингиллгогерихуирндробуллллантисилиогогогох*.

Выберите варианты ответа API, в которых соблюдается принцип консистентности.

☑ Правильный ответ

Запрос: /towns/10/

Ответ:

```
{"lat": 53.223333,"lng": -4.199444, "population": 3107}
```

Здесь разработчик сделал всё правильно: передал координаты в словаре с нужными ключами и со значениями в формате float. Ошибки нет, координаты Лланвайрпуллгуингиллгогерихуирндробуллллантисилиогогогоха переданы правильно и консистентно.

☑ Тоже правильный ответ

Запрос: /towns/10/what_around/

Ответ:

Скопировать код

```
[
    {
        "lat": 53.223243,
        "lng": -4.135245
    },
    {
        "lat": 53.262778,
        "lng": -4.093889
    },
    {
```

```
        "lat": 53.141111,
        "lng": -4.27
    }
]
```

API вернул список словарей, и словари соответствуют требованиям: ключи правильные, значения в нужном формате. Разработчик сделал всё хорошо, координаты Бангора, Бомариса и Кэрнарфона пришли в консистентном виде.

- [x] Неправильный ответ

Запрос: /towns/10/where_capital/

Ответ:

```
{"lat": "51.507222","lng": "-0.1275"}
```

Это совершенно неконсистентный ответ API! Выглядит прилично, но значения координат Лондона вернулись строкой вместо float. Это нарушение задания и небрежность разработчика.

- [ ]

Запрос: /towns/10/warehouse/

Ответ:

```
{"shir": "53°15’46","dlg": "-4°05’38", "m2": 3702}
```

Согласованность

Понятие консистентности включает в себя и идею **согласованности**: добавление в API новой функциональности не должно сломать API.

Например, при проектировании блога было решено возвращать через API не словарь, а упорядоченный **список** значений. Это допустимое решение, JSON поддерживает и такую структуру данных.

Скопировать код

```
# GET запрос поста с id=10
GET /api/v1/posts/10/


# Ответ API
[[10, "Это текст из моего поста.", 1618567801, 1]]
```

Клиенты успешно принимают ответ и ожидают, что элемент списка с индексом `0` — это id объекта, а под индексом `2` хранится время в формате unix-timestamp.

Но в какой-то момент потребовалось расширить ответ и дополнительно возвращать в ответе id группы, в которой был опубликован пост. Разработчик изменяет ответ, добавив в него новый элемент:

```
Скопировать код
# Новый ответ API
[[10, "Это текст из моего поста.", 7, 1618567801, 1]]
```

На первый взгляд ничего страшного не произошло, но теперь в элементе с индексом 2 хранится не время, а id группы; в результате все приложения, которые подключены к API, перестанут работать.

Можно предположить, что именно в этот момент в службу поддержки начнут писать и звонить недовольные клиенты: «Ваш API сломался!». А он не сломался — он просто начал работать по-другому. Но клиентам от этого не легче.

Нужно искать выход из сложившейся ситуации.

Можно создать новую версию API и применить изменения в ней, оставив существующую версию в прежнем виде. Такая практика встречается повсеместно (а иногда это вообще единственный доступный вариант).

Однако будет лучше спроектировать API так, чтобы можно было **расширять** его, не создавая новые версии и не лишая клиентов возможности получать данные в привычном формате.

Расширяемость

При проектировании API именно разработчик определяет, какие данные и в каком формате будут возвращаться в ответ на тот или иной запрос.

Сейчас API возвращает данные в виде упорядоченного списка: `[[1, "Это мой первый комментарий.", 1618565516, 10]]`. Так иногда делают для повышения производительности, но в нашей ситуации этот вариант оказался непрактичен: он усложняет расширяемость. Здесь лучше отдавать данные в более распространённом для JSON формате, аналогичном словарю, со структурой `{"ключ": "значение"}`.

При такой структуре добавление новых элементов не повлечёт за собой проблем с парсингом данных. Клиенты будут получать необходимую

информацию из ответа по прежним ключам, а новые ключи могут использовать, а могут игнорировать. И ничего не сломается.

Скопировать код

```
# Ответ API в формате "ключ": "значение"
[{
    "id": 10,
    "text": "Это текст из моего поста.",
    "pub_date": 1618567801,
    "group_id": 7,
    "author_id": 1
}]
```

Уже неплохо. Но лучше бы предусмотреть и следующий уровень расширяемости: что, если потребуется добавить к информации о посте какие-то данные, которых нет в модели Post?

Например, при запросе `GET /api/v1/posts/10` возвращаемую информацию о посте с id=10 можно расширить дополнительными данными: это может быть, допустим, набор ссылок на посты, похожие на запрошенный.

Этот случай можно и нужно учесть на этапе проектирования. Лучше сразу предусмотреть возможность добавления в ответ дополнительных (но не являющихся частью модели) данных. Для этого нужно немного изменить структуру ответа.

В нашем примере для добавления списка ссылок к ответу нужно:

- выделить возвращаемую информацию о посте в JSON-объект `post`;
- добавить в возвращаемый JSON объект `links`, в котором будут перечислены нужные ссылки.

Такой подход позволит добавлять в ответ и другие объекты без опасения сломать парсеры у клиентов.

Итоговый вариант может быть примерно таким:

Скопировать код
JSON

```
// Ответ API в формате "ключ": "значение" с ссылками на похожие посты
[{
    "post": {
        "id": 10,
```

```
        "text": "Это текст из моего поста.",
        "pub_date": 1618567801,
        "group_id": 7,
        "author_id": 1
    },
    "links": {
        "link1": "/posts/12",
        "link2": "/posts/23",
            ...
    }
}]
```

Такая архитектура даст возможность в будущем включать в ответ любую дополнительную информацию, пусть даже сейчас неизвестно, что это будут за данные и откуда они возьмутся.

Вот это и называется **расширяемостью**. Тщательное проектирование API даст возможность расширять ответы без опасности поломать что-то уже существующее.

Самая лучшая структура JSON для API

В природе не существует «самой лучшей», эталонной структуры ответа API. Нельзя корректно ответить на вопрос «что лучше — велосипед или чайник?», всё зависит от ситуации; точно так же и для каждого проекта существует своя лучшая архитектура. И одна из задач разработчика — найти и реализовать её.

Для одного проекта лучшей структурой может быть тот плоский список, с которого мы начинали: ответ получается лаконичен, это экономит ресурсы и трафик; для другого проекта даже структура нашего финального JSON не сможет учесть все необходимые варианты и требования.

Как вам урок?

К следующему уроку

Инструментарий для тестирования API

В процессе работы вы уже отправляли запросы к серверам — из браузера, если в ответ ожидалась HTML-страница, или из кода посредством библиотеки *requests* (когда ответ сервера надо было обработать в коде).

Для отправки запросов есть и специальные программы, заточенные под работу с API.

Такие программы можно условно разделить на консольные (такие как **curl** и **httpie**), встраиваемые (плагины для браузера или для редактора кода, например, **REST Client** для VS Code) и графические (**Postman**).

- В **консольных** инструментах управление запросами происходит через командную строку со всеми особенностями этого подхода.
- **Встраиваемые** инструменты — это дополнительные модули к программам. Они предоставляют графический интерфейс для работы с запросами прямо в текущем рабочем пространстве программы, в результате разработчику не приходится переключаться между приложениями.
- Инструменты **с собственным графическим интерфейсом** — это самостоятельные программы, настольные приложения.

При разработке и тестировании вашего API вам обязательно понадобится один из этих инструментов. Не спешите устанавливать все сразу, выберите приложение, которое вам больше понравится. В дальнейшем курсе примеры запросов и ответов будут показаны в интерфейсе Postman, но решение, в какой из программ работать, остаётся за вами.

Не пожалейте времени и внимательно рассмотрите ответы тех API, к которым будут отправляться тестовые запросы в этом уроке. Там можно увидеть много интересного: структуру разных JSON, принципы вложенности и именования объектов и полей — всё то, о чём был разговор в прошлом уроке.

Тестирование API через httpie

httpie — консольный API-клиент. Для знакомства с ним нет нужды его устанавливать. Попробовать его в работе можно в веб-эмуляторе. Откройте его и для активации нажмите Enter или Return.

В «веб-консоли» появится заранее написанный PUT-запрос к эндпоинту pie.dev/put/; в запросе передаётся заголовок API-Key и параметр `hello` со значением `world`.



Нажатием на Enter отправьте запрос к тестовому API — и вы увидите ответ сервера:

Скопировать код
JSON

```
PUT /put HTTP/1.1
API-Key: foo
Accept: application/json, */*;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Content-Length: 18
Content-Type: application/json
Host: pie.dev
User-Agent: HTTPie/2.3.0

{
    "hello": "world"
}

HTTP/1.1 200 OK
Access-Control-Allow-Credentials: true
Access-Control-Allow-Origin: *
```

```
CF-Cache-Status: DYNAMIC
CF-RAY: 6430db7dcc26924c-EWR
Connection: keep-alive
Content-Encoding: gzip
Content-Type: application/json
Date: Tue, 20 Apr 2021 19:42:42 GMT
NEL: {"max_age":604800,"report_to":"cf-nel"}
Report-To: {"group":"cf-nel","endpoints":[{"url":"https:\/\/a.nel.cloudflare.com\/
report?
s=549OJOMAtbaQ9GN45r0Ct2rFBGJcMTFRCQvUGJZ8KCIm8noJzceAWKylg6S57AlrnpJhERD6GqiKby91%2F
bU1ZX%2F0HwwYXxDM"}],"max_age":604800}
Server: cloudflare
Set-Cookie: __cfduid=d5a88aa57823d025fbe244fa2bdef122b1618947762; expires=Thu, 20-
May-21 19:42:42 GMT; path=/; domain=.pie.dev; HttpOnly; SameSite=Lax
Transfer-Encoding: chunked
alt-svc: h3-27=":443"; ma=86400, h3-28=":443"; ma=86400, h3-29=":443"; ma=86400
cf-request-id: 099267829e0000924c7d091000000001

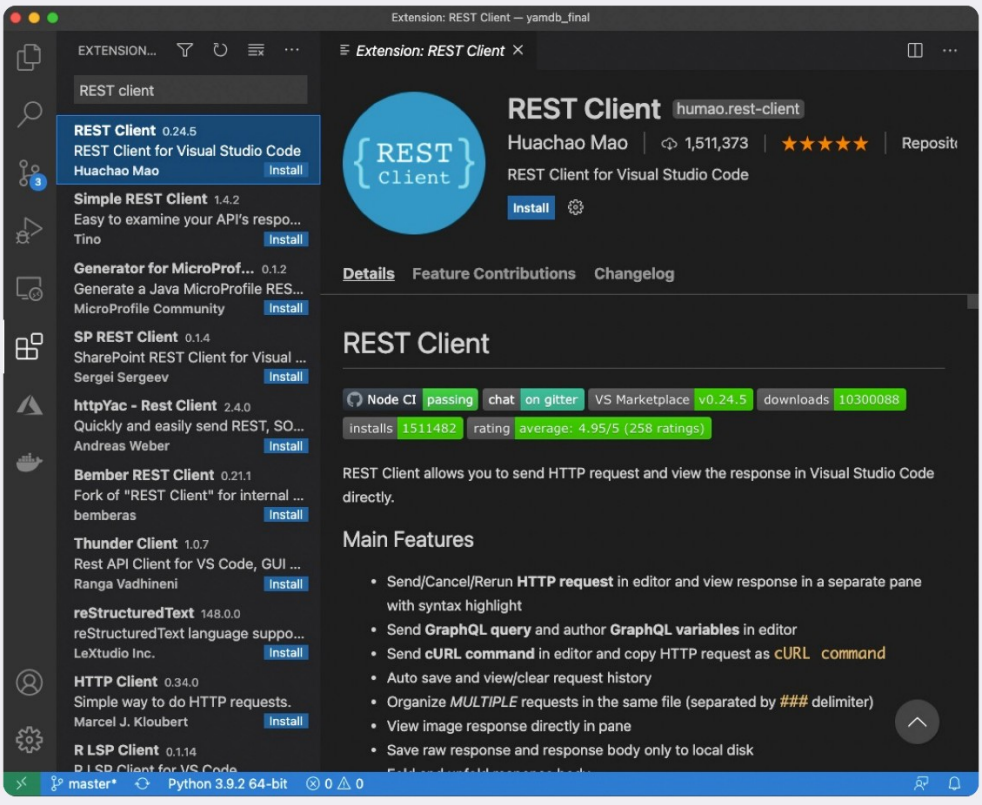
{
    "args": {},
    "data": "{\"hello\": \"world\"}",
    "files": {},
    "form": {},
    "headers": {
        "Accept": "application/json, */*;q=0.5",
        "Accept-Encoding": "gzip",
        "Api-Key": "foo",
        "Cdn-Loop": "cloudflare",
        "Cf-Connecting-Ip": "192.241.133.165",
        "Cf-Ipcountry": "US",
        "Cf-Ray": "6430db7dcc26924c-FRA",
        "Cf-Request-Id": "099267829e0000924c7d091000000001",
        "Cf-Visitor": "{\"scheme\":\"http\"}",
        "Connection": "Keep-Alive",
        "Content-Length": "18",
        "Content-Type": "application/json",
        "Host": "pie.dev",
        "User-Agent": "HTTPie/2.3.0"
    },
    "json": {
```

```
        "hello": "world"
    },
    "origin": "192.241.133.165",
    "url": "http://pie.dev/put"
}
```

Если этот консольный API-клиент придётся вам по душе — инструкцию по установке вы найдёте там же, на сайте разработчиков.

Тестирование API через расширение для VS Code

Расширение REST Client позволяет прямо из VS Code отправлять HTTP-запросы и в этом же интерфейсе просматривать ответы на них. Установка расширения стандартна: в строке поиска панели “Extensions” нужно ввести название плагина “REST Client” и нажать кнопку “Install” («Установить»).



Можно потренироваться, отправляя запросы к бесплатному API проекта {JSON} Placeholder. Этот проект имитирует социальную сеть: в нём хранятся посты и комментарии к ним. На главной странице проекта есть

примеры запросов, описание всех доступных ресурсов, эндпоинтов и методов.

Запросы в REST Client нужно сохранять в файле с расширением .http. Если вы решите поработать с этим плагином — после установки создайте в VS Code новый файл (например requests.http) и создайте в нём запрос:

Скопировать код

```
# пример GET запроса

GET https://jsonplaceholder.typicode.com/posts/1
```

В файле можно описать несколько разных запросов и отдельно выполнить любой из них. Запросы в файле отделяются друг от друга строками, содержащими три символа `#`.

Добавьте в файл ещё один запрос, пусть это будет POST.

POST-запросы в REST Client пишутся так:

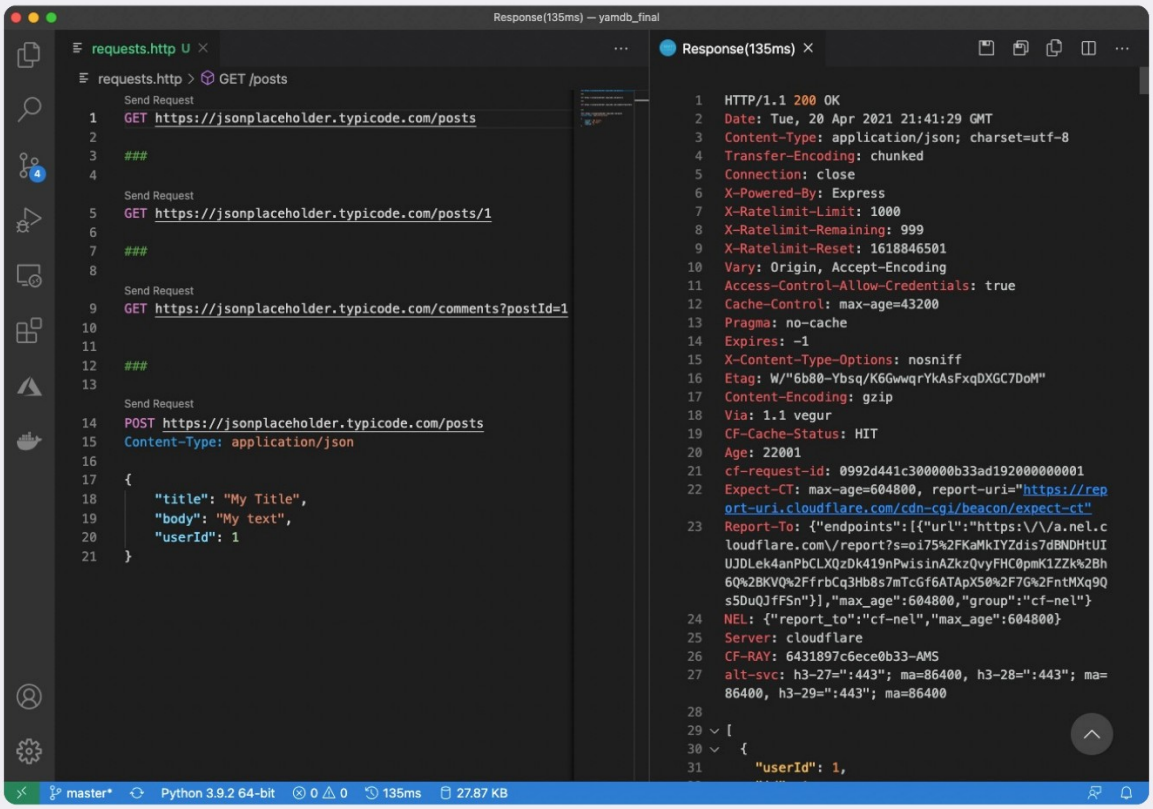
Скопировать код

```
###


POST https://jsonplaceholder.typicode.com/posts
Content-Type: application/json


{
    "title": "My Title",
    "body": "My text",
    "userId": 1
}
```

Для отправки любого запроса нажмите над ним ссылку “SendRequest”. В правой части экрана вы увидите ответ сервера:



Файл с запросами можно сохранить в репозитории, и любой разработчик сможет отправить сохранённые запросы и посмотреть результат.

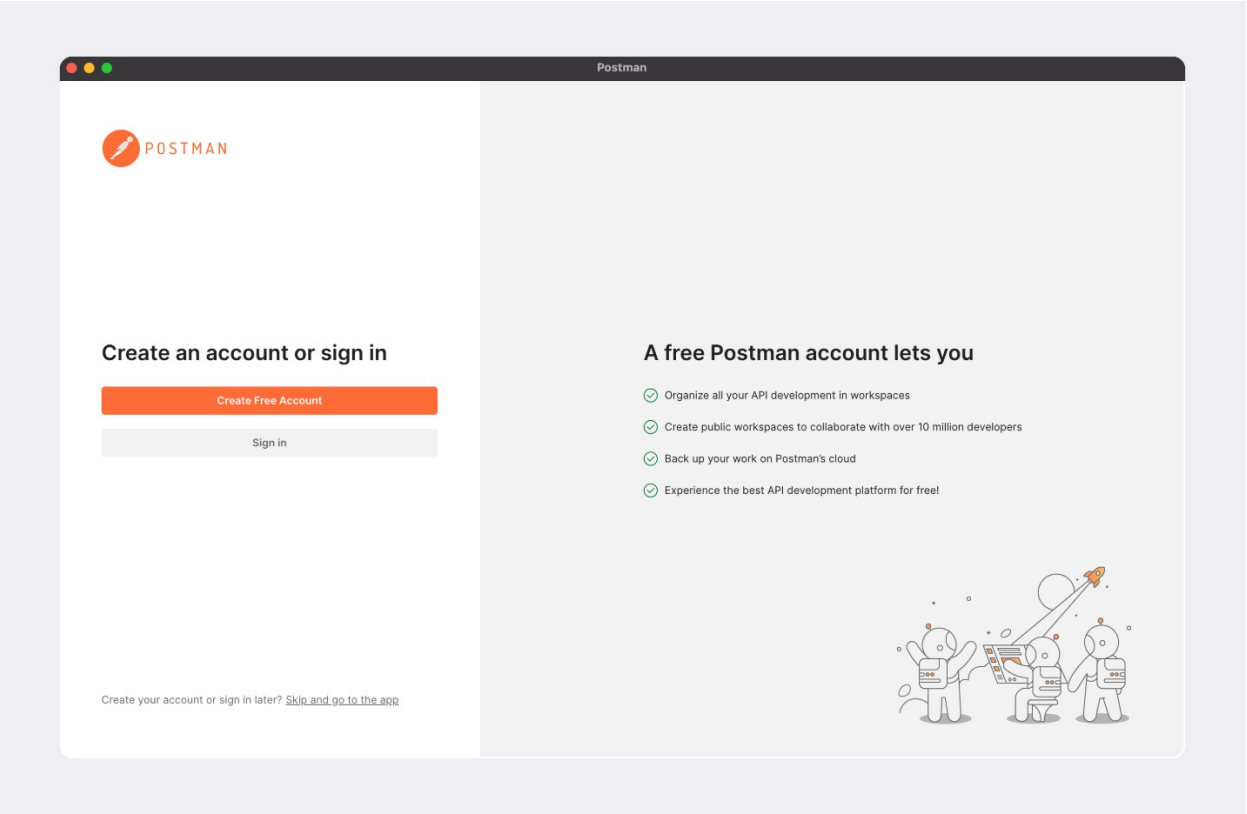
Тестирование API через Postman

Postman — популярный и удобный API-клиент, он умеет отправлять запросы и показывать ответы, сохраняет историю запросов и данные об аутентификации, позволяет проектировать и тестировать API.

Управлять запросами в Postman можно и через браузерное приложение, но для работы с локальным проектом потребуется скачать и установить программу на компьютер: работать через браузер с локальным IP 127.0.0.1 не получится, он не будет доступен.

Скачайте Postman со страницы загрузки проекта и установите его на свою рабочую машину.

Установка Postman



Авторизуйтесь в приложении, это позволит работать в интерфейсе **Workspace** (системе для создания и сохранения запросов для каждого проекта по отдельности) и даст некоторые другие бонусы, например — хранение настроек и истории запросов в облаке.

Postman Workspace

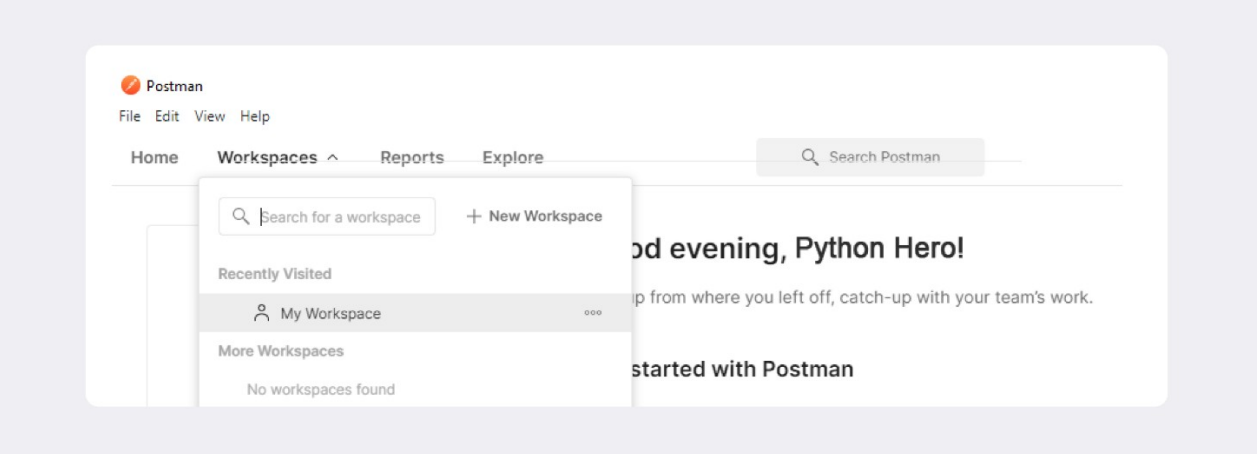
Postman может сохранять запросы, ответы, токены и другую рабочую информацию; вы в любой момент можете одним кликом отправить заранее настроенный и сохранённый запрос. При работе с несколькими проектами удобно разделять эти данные, хранить данные каждого проекта отдельно.

Workspace — это своего рода окружение проекта, способ разделить настройки разных сервисов.

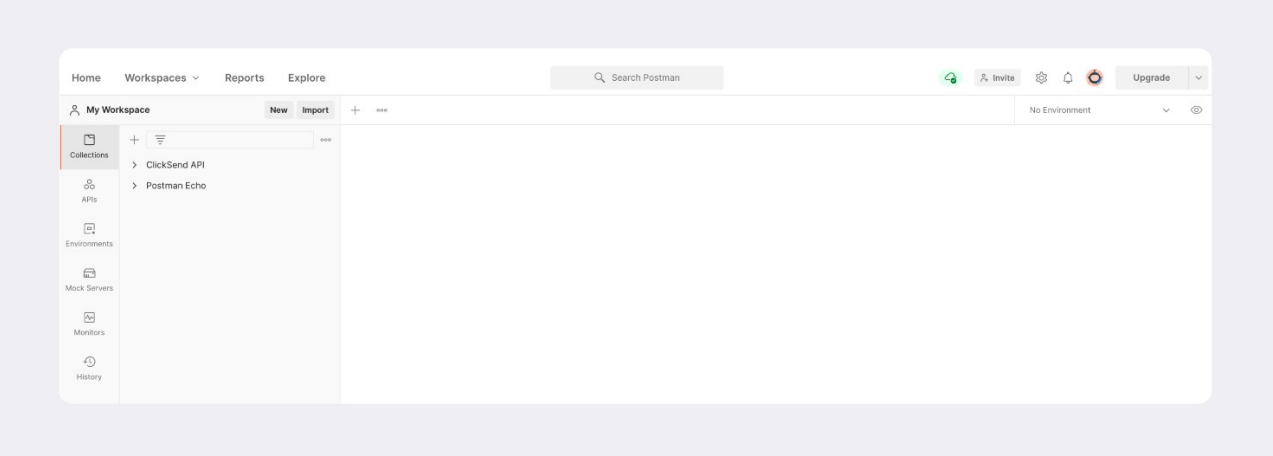
Например, вы тестировали сервис с котиками, установили для него токен аутентификации, сохранили примеры запросов, всё настроили — а потом вам пришлось перейти к тестированию другого проекта, с пёсиками например.

В такой ситуации Workspace в Postman позволит просто переключиться на другое окружение и создать там другие запросы и токены, относящиеся к новому проекту.

На стартовой странице приложения выберите закладку “Workspace” и нажмите на “My Workspace” — это первое, предварительно созданное рабочее окружение. Его можно переименовать и настроить в нём запросы для своего проекта.



После нажатия на кнопку “My Workspace” откроется основной интерфейс Postman.

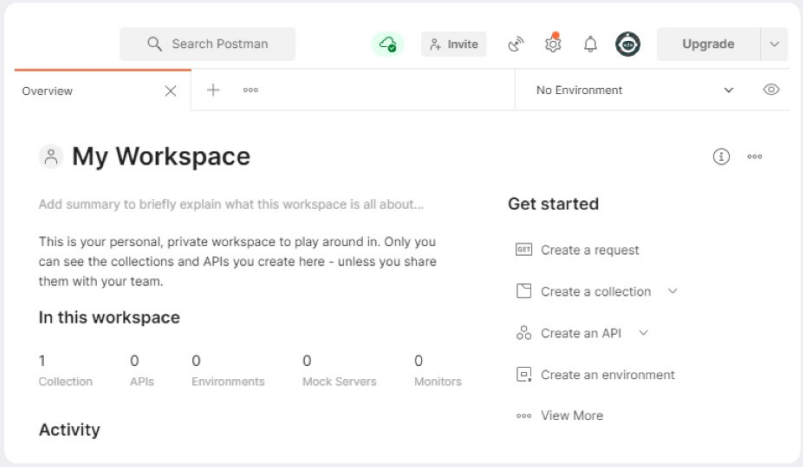


Главный экран делится на две части:

- слева — список сохранённых запросов и история изменений, у вас их пока нет;
- справа — рабочая область: тут можно создавать запросы и настраивать их.

Интерфейс может немного отличаться от скриншотов — это зависит от версии программы, но функциональность остаётся та же.

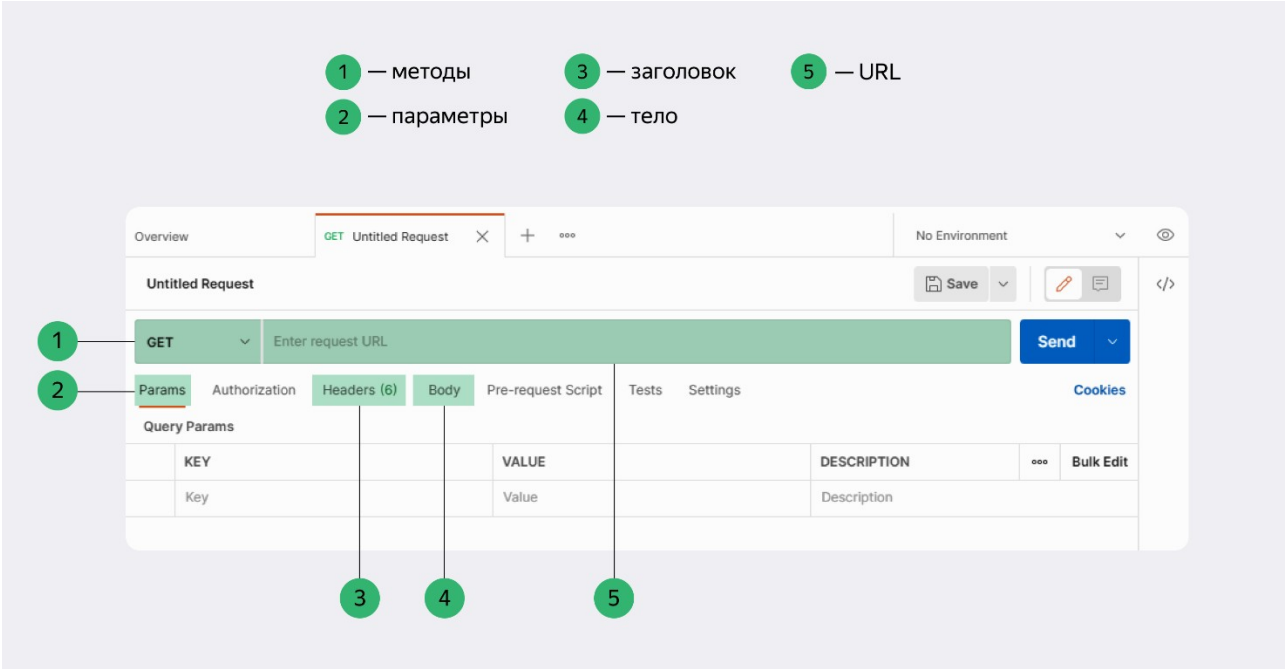
Чтобы создать запрос, кликните по кнопке «+» над рабочей областью.



Окно настройки запроса

Здесь можно указать и отредактировать любые настройки:

- метод запроса;
- URL, по которому нужно отправить запрос;
- параметры запроса;
- заголовки запроса;
- тело запроса.



Теперь можно поупражняться.

У GitHub есть отлично документированное API, вот туда и отправьте тестовый GET запрос на получение данных аккаунта *yandex-praktikum*:

Скопировать код

```
https://api.github.com/users/yandex-praktikum
```

Если всё сделано как надо — ответ будет таким:

Скопировать код
JSON


```
{
    "login": "yandex-praktikum",
    "id": 58176914,
    "node_id": "MDQ6VXNlcjU4MTc2OTE0",
    "avatar_url": "https://avatars.githubusercontent.com/u/58176914?v=4",
    "gravatar_id": "",
    "url": "https://api.github.com/users/yandex-praktikum",
    "html_url": "https://github.com/yandex-praktikum",
    "followers_url": "https://api.github.com/users/yandex-praktikum/followers",
    "following_url":
"https://api.github.com/users/yandex-praktikum/following{/other_user}",
    "gists_url": "https://api.github.com/users/yandex-praktikum/gists{/gist_id}",
    "starred_url":
"https://api.github.com/users/yandex-praktikum/starred{/owner}{/repo}",
    "subscriptions_url":
"https://api.github.com/users/yandex-praktikum/subscriptions",
    "organizations_url": "https://api.github.com/users/yandex-praktikum/orgs",
    "repos_url": "https://api.github.com/users/yandex-praktikum/repos",
    "events_url": "https://api.github.com/users/yandex-praktikum/events{/privacy}",
    "received_events_url":
"https://api.github.com/users/yandex-praktikum/received_events",
    "type": "User",
    "site_admin": false,
    "name": null,
    "company": null,
    "blog": "",
    "location": null,
    "email": null,
    "hireable": null,
    "bio": null,
    "twitter_username": null,
```

```
    "public_repos": 6,
    "public_gists": 0,
    "followers": 90,
    "following": 0,
    "created_at": "2019-11-25T13:38:59Z",
    "updated_at": "2021-03-24T16:15:56Z"
}
```

Данные аккаунтов на Github открыты, получить их может кто угодно. Запросите данные своего аккаунта:

Скопировать код

```
https://api.github.com/users/<ваш-логин-на-гитхабе>
```

Можно получить информацию и о конкретном репозитории в аккаунте: надо лишь покопаться в документации и найти нужный метод.

Пора создать свой API и подёргать его запросами за усы.

Как вам урок?

К следующему уроку

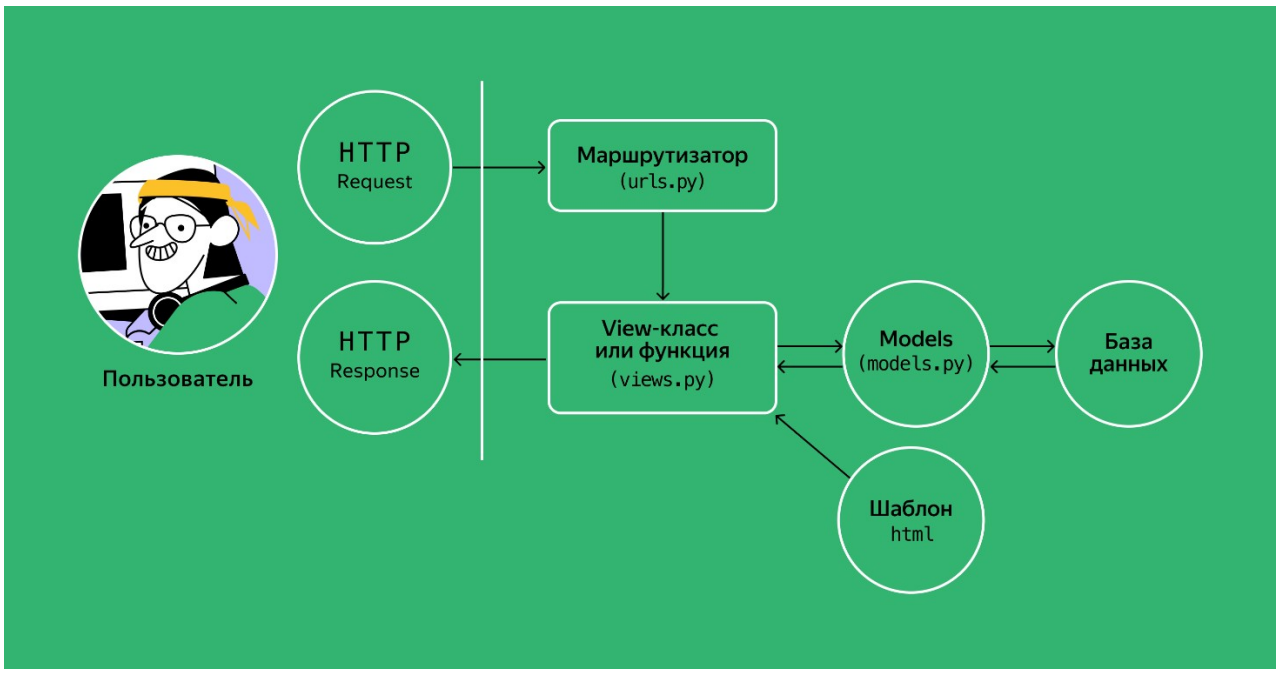
Преобразование форматов. Сериализаторы

Прежде, чем взяться за создание REST API на основе Django REST framework — разберёмся, чем взаимодействие программы и человека отличается от взаимодействия между программами.

Интерфейс для людей: веб-страницы

Django-проекты, с которыми вы уже работали, предназначены для взаимодействия с человеком. Обработка запросов в этих проектах происходит так:

- в приложение прилетает HTTP-запрос,
- запрос передаётся в *urls.py* и, в зависимости от запрошенного адреса, отправляется в соответствующую view-функцию или view-класс;
- view-функция или view-класс при необходимости взаимодействует с базой данных: добавляет или запрашивает данные;
- функция `render` встраивает данные, сгенерированные во view-классе или функции, в HTML-шаблон и создаёт HTML-страницу;
- готовая HTML-страница отдаётся в качестве HTTP-ответа.



В результате клиент получает информацию из БД, неразрывно связанную с её оформлением: данные встроены в HTML-страницу, а HTML — это способ представить информацию в формате, удобном для чтения человеком: это «интерфейс для людей».

Интерфейс для программ: API

В свою очередь, API — это интерфейс для программ, именно программы будут обмениваться информацией друг с другом, HTML-шаблоны в этой системе не понадобятся.

При этом программы, которые будут обмениваться данными, могут быть написаны на разных языках программирования. Однако благодаря API они без проблем смогут общаться между собой, обмениваясь данными в удобном и едином для всех формате (например — в формате JSON или XML).

Таким образом, чтобы описанное взаимодействие получилось, данные постоянно должны преобразовываться из формата в формат: из типов данных конкретного языка программирования в выбранный формат обмена данными и обратно. Такие преобразования получили названия: **сериализация** и **десериализация**.

В качестве примера взаимодействующих программ можно взять блог-платформу и мобильное приложение, созданное для этого сервиса.

Серверная часть и мобильное приложение будут взаимодействовать через API: приложение отправляет запросы на сервер, к API, передавая или получая данные.

Разработчик API сам выбирает формат, в котором API возвращает данные; установим, что API будет работать с JSON.

Сериализация

Первая задача: пользователь хочет прочесть какой-то конкретный пост из своей ленты друзей. Для этого должны быть выполнены такие операции:

- Мобильное приложение отправит GET-запрос к API Yatube; в запросе будет передан id поста.
- API сделает запрос к базе данных проекта в надежде получить оттуда экземпляр класса `Post` с заданным id.
- Если пост с таким id есть в БД, то программа получит объект класса Post (этот объект содержит несколько полей разных типов данных).
- В качестве ответа на вопрос нужно вернуть данные в формате JSON. Преобразование Python-объекта в JSON происходит в два шага:
  - сложный объект (например, экземпляр модели) преобразуется в словарь, содержащий простые типы данных Python;
  - получившийся Python-словарь конвертируется («рендерится») в JSON.
- JSON отправляется в HTTP-ответе в мобильное приложение.

Например, из БД получен объект класса Post, который можно было бы описать так:

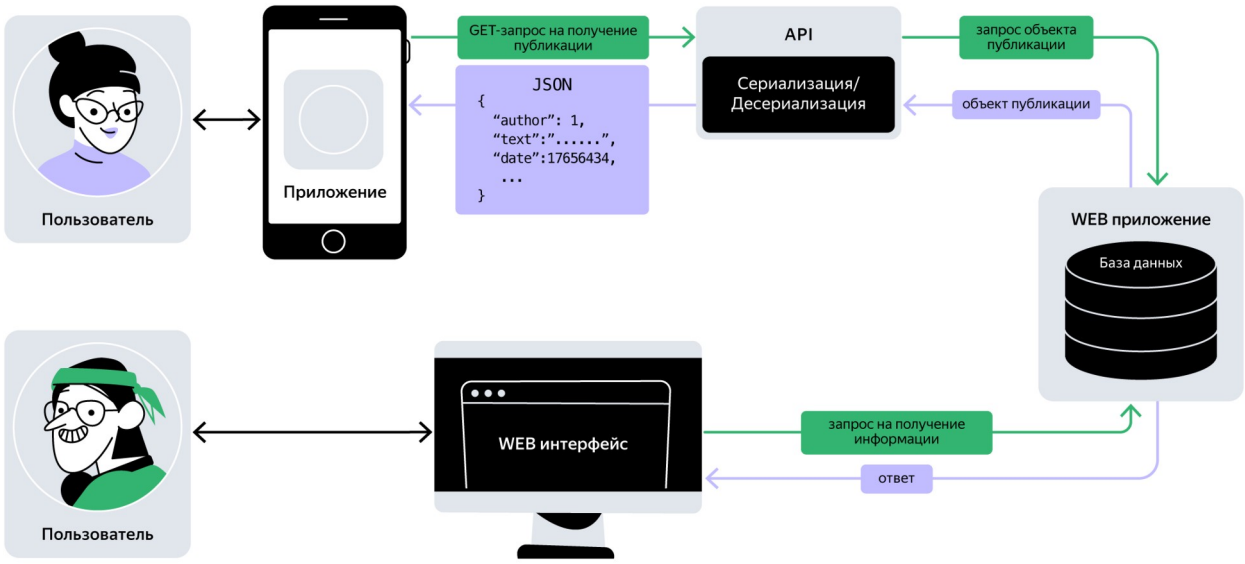
Скопировать код
PYTHON

```
post = Post(
    id=87,
    author='Робинзон Крузо',
    text='23 ноября. Закончил работу над лопатой и корытом.',
    pub_date='1659-11-23T18:02:33.123543Z'
)
```

После сериализации этого объекта будет создан такой JSON:

Скопировать код
JSON

```
{
    "id": 87,
    "author": "Робинзон Крузо",
    "text": "23 ноября. Закончил работу над лопатой и корытом.",
    "pub_date": "1659-11-23T18:02:33.123543Z"
}
```



Валидация и десериализация

Вторая задача: пользователь решил через мобильное приложение добавить в блог новую запись. Пользователь набрал текст и нажал кнопку «Отправить».

Из мобильного приложения будет отправлен POST-запрос к API, и при получении запроса API выполнит действия, подобные описанным, но в обратном направлении: преобразует JSON из запроса в Python-объект. В процесс добавится дополнительный шаг — проверка данных на корректность, **валидация**.

Будут выполнены такие операции:

- Мобильное приложение сформирует и отправит POST-запрос к API проекта; в теле запроса будет передан JSON, содержащий всю необходимую информацию для создания нового поста.
- Полученный JSON должен быть **десериализован**. Десериализация происходит в три этапа:
  - Преобразование JSON в простые типы данных Python.
  - Валидация — проверка соответствия полученных данных ожиданиям.
  - Конвертация валидированных данных в сложные объекты Python (в queryset или объект модели).
- Если полученные данные соответствуют модели, то API отправит запрос к БД, чтобы добавить в неё новый объект, а пользователю отправит ответ с подтверждением

Если же полученные данные не пройдут валидацию, то API не будет добавлять их в базу данных, а просто отправит ответ с сообщением об ошибке.

Пример JSON, который API получит в POST-запросе:

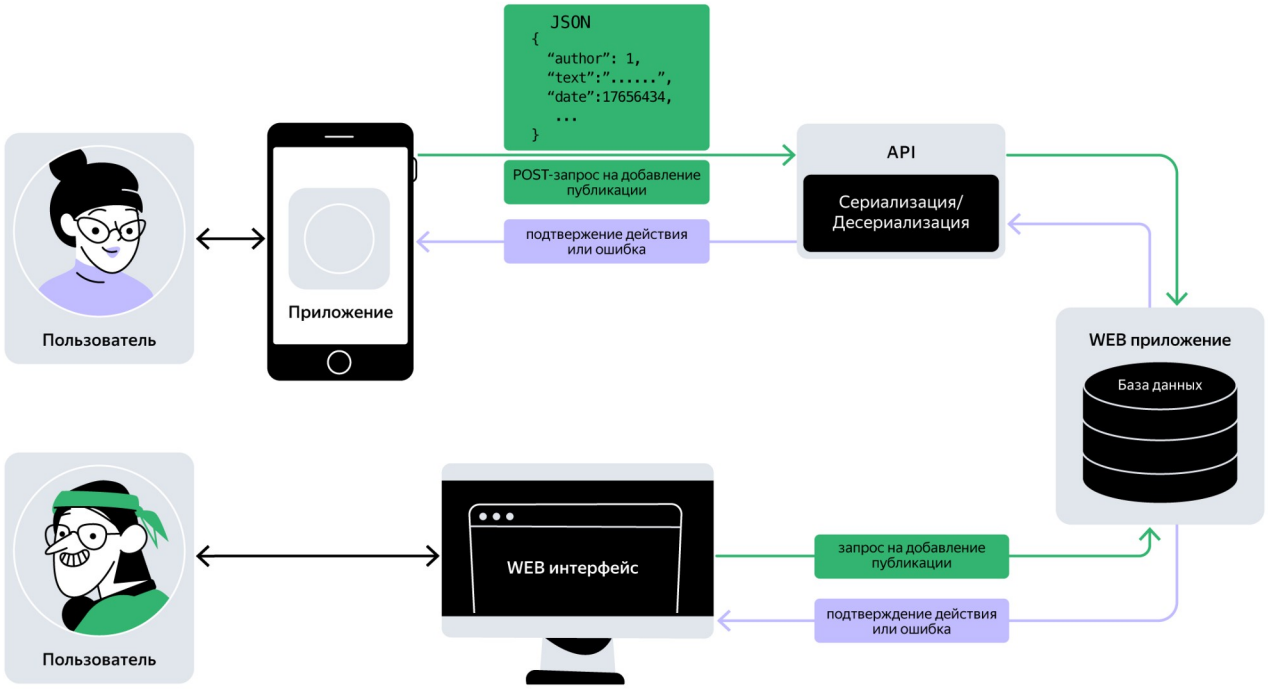
Скопировать код
JSON

```
{
    "author": "Робинзон Крузо",
    "text": "24 декабря. Всю ночь и весь день шёл проливной дождь.",
    "pub_date": "1659-12-24T21:14:56.123543Z"
}
```

JSON будет десериализован в объект Python, который можно описать так:

Скопировать код
PYTHON

```
post = Post(
    author='Робинзон Крузо',
    text='24 декабря. Всю ночь и весь день шёл проливной дождь.',
    pub_date='1659-12-24T21:14:56.123543Z'
)
```



Классы-сериализаторы в DRF

В Django REST Framework есть классы, которые способны принимать участие во всех трёх операциях: сериализации, валидации и десериализации. Эти классы называются **сериализаторы (serializers)**.

Сериализаторы преобразуют сложные данные, такие как queryset или экземпляр модели, в простые типы данных Python, которые затем можно конвертировать («отрендерить») в JSON, XML или другие форматы обмена данными.

Сериализаторы выполняют и обратное преобразование: конвертируют данные, полученные из JSON, в сложные объекты; при этом данные проходят валидацию.

Сериализаторы могут работать с моделями Django и с обычными Python-классами.

- Для работы с обычными Python-классами сериализаторы наследуют от класса **Serializer**.
- Сериализаторы, работающие с моделями, наследуют от **ModelSerializer**.

Эти классы чем-то схожи с классами **Form** и **ModelForm**: при получении запроса эти классы валидируют полученные данные и конвертируют их в Python-объекты, а при создании ответа получают на вход Python-объект и преобразуют его в необходимый формат.

Чтобы класс-сериализатор выполнил свою работу, в нём должны быть описаны:

- поля; их названия будут служить ключами в JSON при сериализации; наличие этих полей будет проверяться в JSON при десериализации;
- типы этих полей: чтобы сериализатор мог правильно конвертировать данные, он должен «знать», какие типы данных соответствуют тому или иному полю класса или модели;
- параметры этих полей (например, максимальную и минимальную длину строки для CharField).

Сериализатор для модели: ModelSerializer

Чаще всего работа с данными в Django осуществляется через модели, и в таких случаях сериализатор наследуется от класса ModelSerializer.

При создании такого сериализатора во внутреннем классе `Meta` нужно указать модель, с которой должен работать сериализатор, и список тех полей модели, которые нужно сериализовать или десериализовать. С внутренним классом `Meta` вы уже встречались при работе с **ModelForm**, в **ModelSerializer** его назначение аналогично.

Для наследников ModelSerializer нет необходимости описывать типы полей и их параметры: сериализатор сам их определит, взяв за основу поля указанной модели.

В качестве примера опишем модель **Comment** и сериализатор для неё:

Скопировать код
PYTHON

```
...


class Comment(models.Model):
    post = models.ForeignKey(Post, on_delete=models.CASCADE)
    author = models.ForeignKey(User, on_delete=models.CASCADE)
    text = models.TextField()
```

```
    created = models.DateTimeField('created', auto_now_add=True)


...


class CommentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Comment
        # Указываем поля модели, с которыми будет работать сериализатор;
        # поля модели, не указанные в перечне, сериализатор будет игнорировать.
        # Для перечисления полей можно использовать список или кортеж.
        fields = ('id', 'post', 'author', 'text', 'created')
```

Чтобы сериализатор работал со всеми полями модели без исключения, можно указать `fields = '__all__'`.

Вместо `fields` можно применить настройку **exclude** (англ. «исключить»): в этом случае сериализатор будет работать со всеми полями модели, за исключением перечисленных.

Лучшим решением будет явно перечислять поля. Если в ходе разработки в модель будет добавлено новое поле, не предназначенное для публикации, а в сериализаторе указано `fields = '__all__'` — велик шанс, что содержимое нового поля попадёт в ответ API и будет обнародовано.

«Явное лучше неявного».

Сериализатор для класса: Serializer

Сериализатор для класса работает аналогично сериализатору для модели. Разница в том, что **ModelSerializer** берёт описание полей из модели, а в классе **Serializer** нужно явным образом описать поля, их типы и параметры.

Для примера создадим Python-класс `Book` и сериализатор для него:

Скопировать код
PYTHON

```
# Класс Book
class Book():
    def __init__(self, author, title, pub_year, genre):
        self.author = author
        self.title = title
```

```
        self.pub_year = pub_year
        self.genre = genre

# Создаём сериализатор, наследник предустановленного класса Serializer
class BookSerializer(serializers.Serializer):
    # Описываем поля и их типы
    author = serializers.CharField(max_length=256)
    title = serializers.CharField(max_length=512)
    pub_year = serializers.IntegerField()
    genre = serializers.CharField(64)
```

Поля в сериализаторе обычно называют так же, как и поля того класса, с которым предстоит работать.

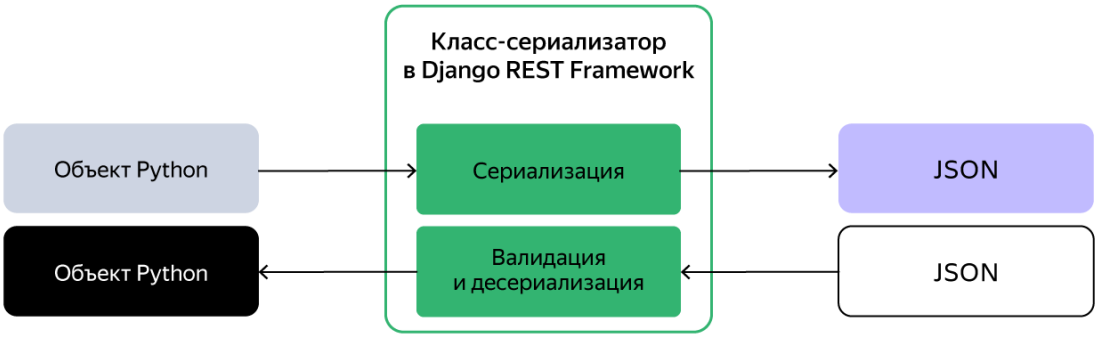
По умолчанию сериализатор будет искать в переданном объекте поля с такими же названиями, как и его собственные. Эти же имена послужат ключами в результирующем словаре после сериализации данных.

Если имена полей в сериализаторе и в объекте, переданном в сериализатор, будут различаться — возникнет ошибка `AttributeError`.

Полный перечень типов полей для сериализаторов доступен в официальной документации.

Действия разные, но сериализатор один

Один и тот же класс-сериализатор в DRF можно применять как для сериализации, так и для десериализации данных. Логика его работы будет выбрана автоматически, в зависимости от того, какой именно объект передан в сериализатор.



Если в конструктор сериализатора передать экземпляр класса, то будет запущен процесс сериализации. А если в качестве параметра передать данные из JSON-объекта, то будет запущен процесс десериализации.

Код классов-сериализаторов принято выносить в отдельный файл *serializers.py*.

Как вам урок?

Перейти к заданию

Задание

Создайте сериализатор для модели **Post** в проекте Yatube.

Импортируйте фреймворк `rest_framework.serializers` и на основе класса `serializers.ModelSerializer` создайте класс `PostSerializer`.

В `PostSerializer` опишите `class Meta` с двумя полями: `fields` и `model`.

В `fields` перечислите поля для вывода текста, автора и даты публикации. Сериализатор не должен обрабатывать поле id из модели **Post**.

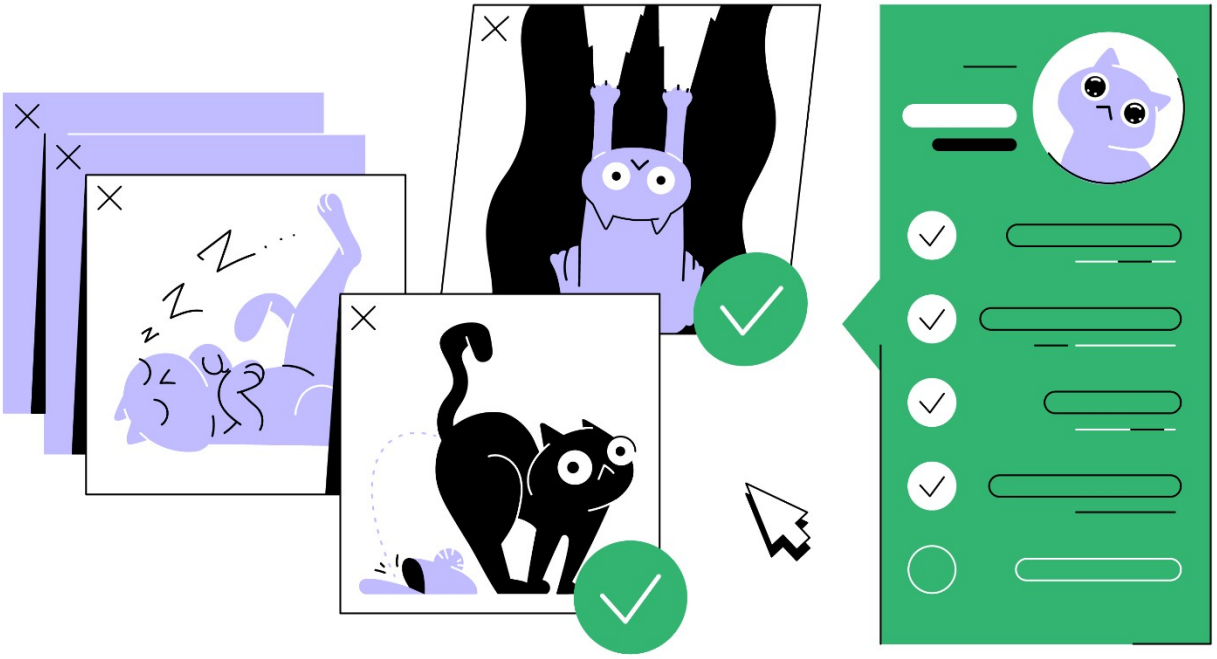
В поле `model` укажите модель, с которой будет связан сериализатор.

```
#  импортируйте в код всё необходимое


class PostSerializer(ModelSerializer):
    class Meta:
        ...
```

Учебный проект Kittygram

Разобраться с базовыми принципами работы с API вам поможет Django-проект **Kittygram: управление котиками**.



В этом проекте упрощено всё, что можно было упростить: он подготовлен для изучения базовых принципов API. Сложный полноценный проект ожидает вас впереди, а пока потренируемся на кошках.

Прочтите README, клонируйте репозиторий, разверните и активируйте виртуальное окружение, установите зависимости из файла *requirements.txt*, выполните миграции.

Обратите внимание на библиотеку `djangorestframework`, именно она поможет быстро и гибко настроить API проекта. Загляните в файл настроек *settings.py*: проверьте, что приложение *cats* и библиотека *rest_framework* добавлены в **INSTALLED_APPS**.

А что внутри?

В **Kittygram** описана простая модель — `Cat`. В ней есть три поля:

Скопировать код

PYTHON

```python
# kittygram/cats/models.py
class Cat(models.Model):
    name = models.CharField(max_length=16)
    color = models.CharField(max_length=16)
    birth_year = models.IntegerField(blank=True, null=True)

    def __str__(self):
        return self.name
```

В файле urls.py настроена обработка адреса `cats/`: запросы к этому URL направляются во view-функцию `cat_list()` приложения cats:

Скопировать код

PYTHON

```python
# kittygram/kittygram/urls.py
from django.urls import path

from .views import cat_list

urlpatterns = [
    path('cats/', cat_list),
]
```

Файл views.py содержит единственную view-функцию `cat_list()`. Обратите внимание на декоратор `@api_view`, именно он настраивает обычную view-функцию для работы с API: например, в нём указываются типы запросов, которые должна обрабатывать view-функция. В следующем уроке мы разберёмся, что это за декоратор и чем ещё он хорош.

Скопировать код

PYTHON

```python
# kittygram/cats/views.py
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status

from .models import Cat
from .serializers import CatSerializer

# View-функция cat_list() будет обрабатывать только запросы GET и POST,
# запросы других типов будут отклонены,
```

```
# так что в теле функции их можно не обрабатывать
@api_view(['GET', 'POST'])
def cat_list(request):
    if request.method == 'POST':
        serializer = CatSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
    cats = Cat.objects.all()
    serializer = CatSerializer(cats, many=True)
    return Response(serializer.data)
```

Откройте в редакторе файл serializers.py, там описан сериализатор `CatSerializer`:

Скопировать код
PYTHON

```
# kittygram/cats/serializers.py

from rest_framework import serializers

from .models import Cat


class CatSerializer(serializers.ModelSerializer):
    class Meta:
        model = Cat
        fields = '__all__'
```

Сериализатор CatSerializer связывает модель и view-класс: он берёт информацию о типах данных из модели, указанной в поле `model`. Остаётся только указать в `fields` те поля, которые надо сериализовать, — и система заработает.

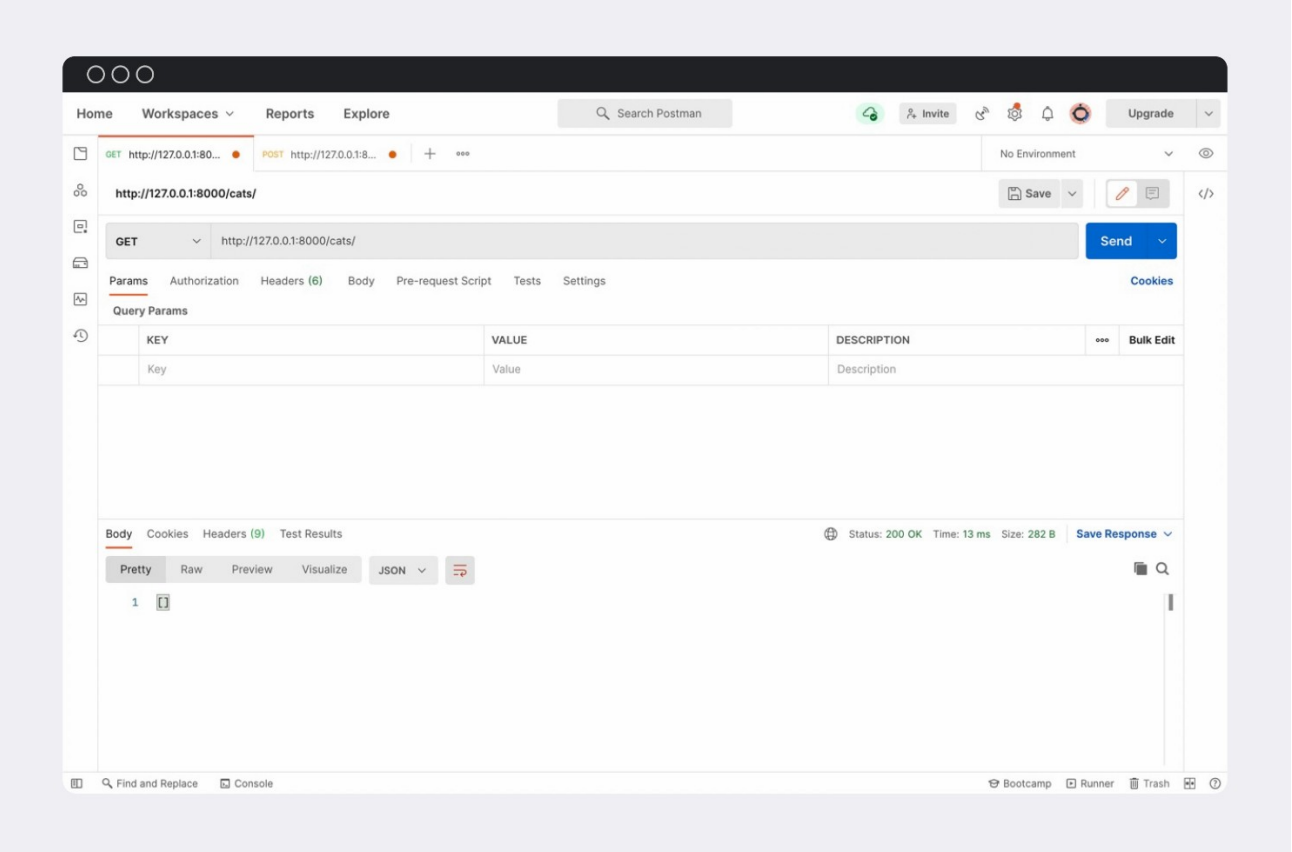
Проверяем работу Kittygram

После развёртывания проект Kittygram полностью готов к работе. Запустите его и сделайте к нему запрос через Postman.

1. Запустите приложение Kittygram командой `python manage.py runserver`.

2. Запустите приложение Postman, в качестве целевого URL укажите `http://127.0.0.1:8000/cats/`.
3. Отправьте GET-запрос.

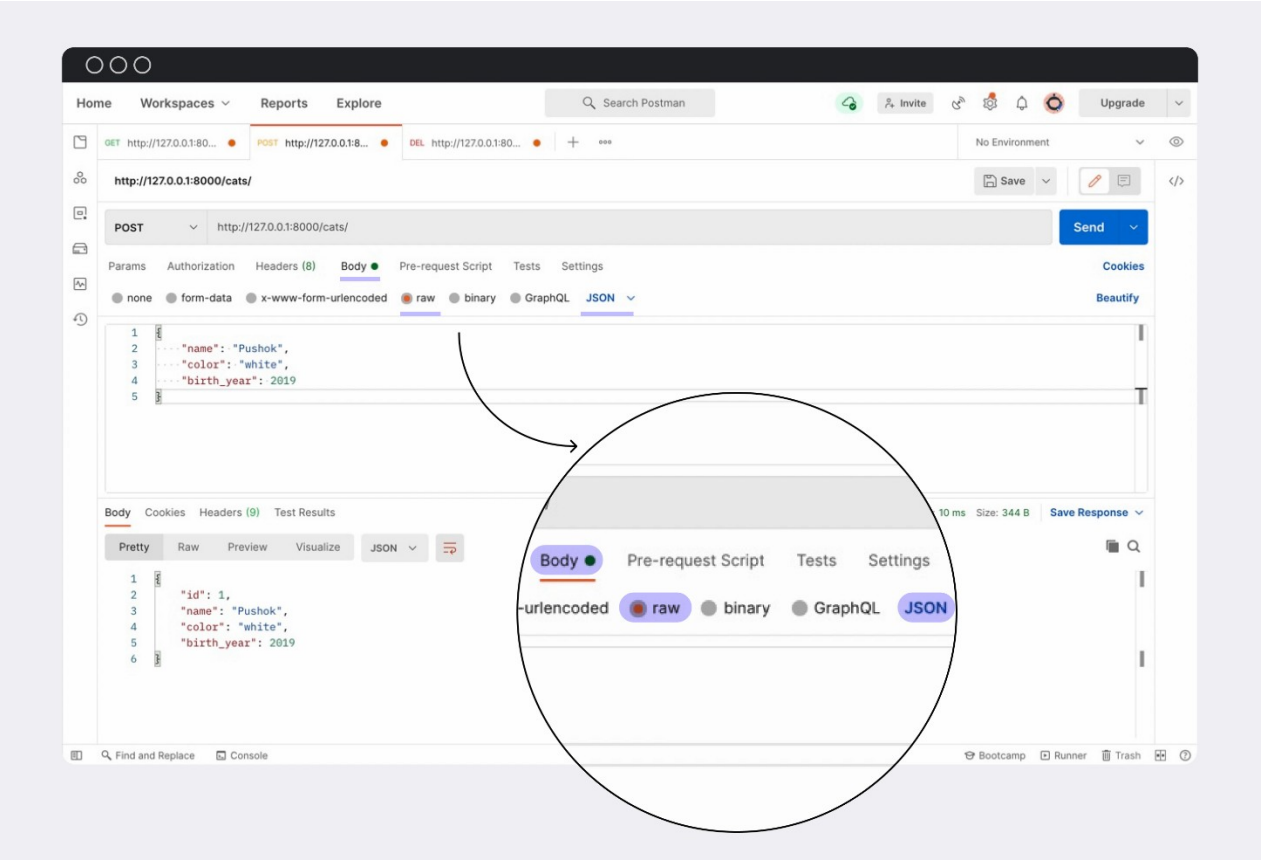
В ответ вернётся пустой JSON-объект. Это нормально, ведь в базе данных пока пусто:



Первый котик в Kittygram

Чтобы создать в БД первую запись с котиком, выполните POST-запрос к проекту и передайте поля `name`, `color` и `birth_year`:

Обратите внимание, что поля нужно передавать в теле (англ. *body*) POST-запроса. А в Postman в соответствующем разделе должны быть выбраны опции **raw** и **JSON**.



Проверьте ответ: если запрос обработан корректно, должен вернуться статус-код 201 Created и JSON с только что созданным объектом.

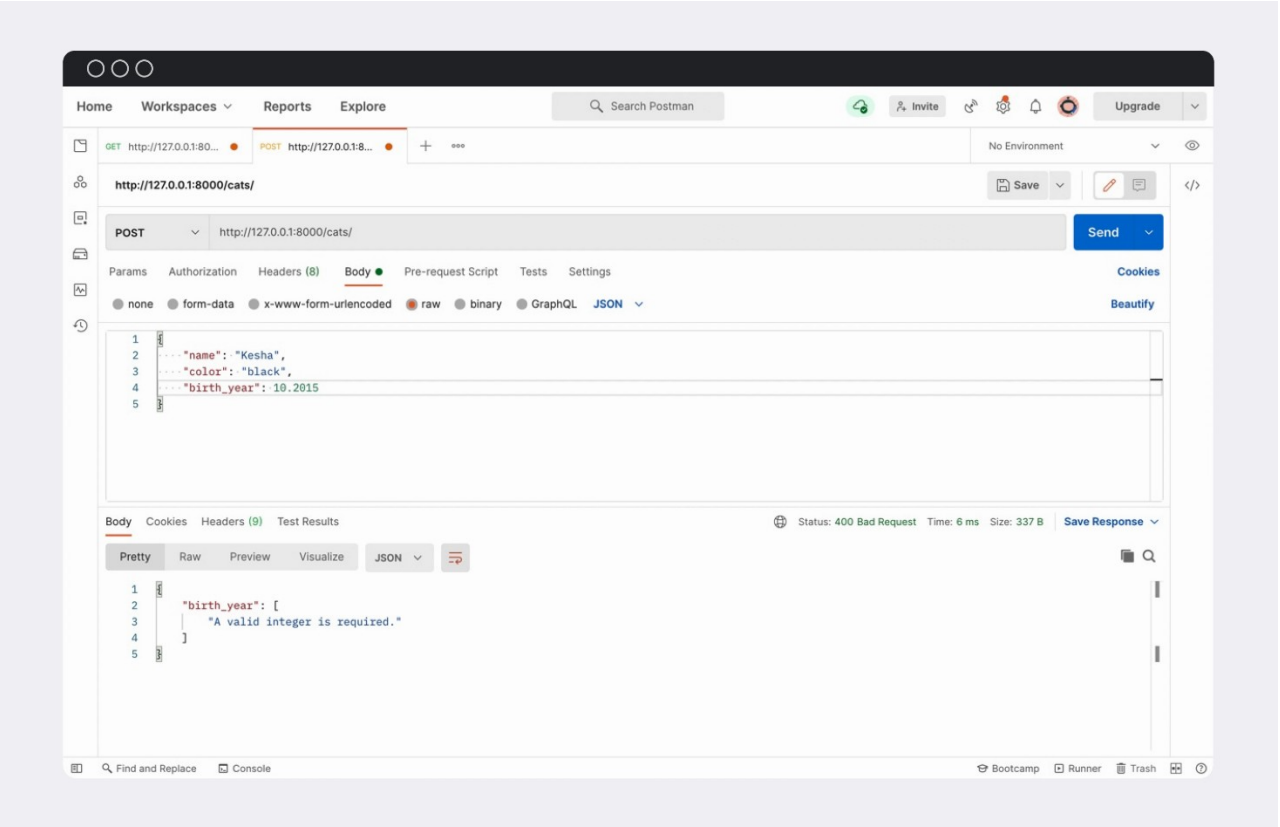
Получилось? Отлично!

Сделайте ещё один POST-запрос, создайте запись для второго котика.

Повторно выполните GET-запрос на тот же URL, в ответ вам вернётся список из существующих в базе данных записей.

Время делать ошибки: 400 Bad Request

Проверьте, что будет, если передать данные в неправильном формате: например, вместо целочисленного значения в поле `birth_year` передайте данные в формате `месяц.год`.



При валидации данных сериализатор обнаружит ошибку, и в ответ вернётся вот такой JSON:

Скопировать код
PYTHON

```
{
    "birth_year": [
        "A valid integer is required."
    ]
}
```

Обратите внимание на статус-код ответа: **400 Bad Request**. Поле `birth_year` ожидает данные в формате *Integer* (целое число), а полученные в запросе данные можно привести только к float. В результате запрос не прошёл валидацию и запись не была создана.

Измените запрос: укажите год в формате *Integer* и повторите попытку. На этот раз всё должно пройти гладко.

Ошибка 405 Method Not Allowed

В декораторе view-функции `cat_list()` указано, что она должна обрабатывать запросы GET или POST: можно только создавать и запрашивать данные.

Скопировать код
PYTHON
```python
@api_view(['GET', 'POST'])
def cat_list(request):
    ...
```

При попытке удалить запись методом DELETE вернётся сообщение об ошибке: **405 Method Not Allowed**.

Скопировать код
JSON
```json
{
    "detail": "Method \"DELETE\" not allowed."
}
```

Настройка полей в ответе

При запросе к API пользователю не понадобится id котика. Хорошо бы убрать это поле из JSON-ответа.

Список полей, передаваемых в ответ, описывается в сериализаторе, в поле `fields`. Вместо записи `fields = '__all__'` явно перечислите обрабатываемые поля.

Скопировать код
PYTHON
```python
# Обновлённый kittygram/cats/serializers.py
from .models import Cat
from rest_framework import serializers


class CatSerializer(serializers.ModelSerializer):
    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year')
```

Сделайте GET-запрос на эндпоинт `http://127.0.0.1:8000/cats/` и проверьте, что всё работает и ответ не изменился.

Уберите `id` из списка полей, и ваш API перестанет отправлять ненужные данные клиенту.

Скопировать код
PYTHON

```python
# Обновлённый kittygram/cats/serializers.py
from .models import Cat
from rest_framework import serializers


class CatSerializer(serializers.ModelSerializer):
    class Meta:
        model = Cat
        fields = ('name', 'color', 'birth_year')
```

Ещё раз сделайте GET-запрос для получения полного списка котиков и посмотрите, как изменилась выдача:

Скопировать код
PYTHON

```
...
{
    "name": "Леопольд",
    "color": "Рыжий",
    "birth_year": 1975
}
...
```

Отлично! Мы избавились от ненужного поля в выдаче.

Как вам урок?

К следующему уроку

View-функции API

В Django REST Framework запрос к API передаётся нужному представлению в соответствии с адресами, перечисленными в файле *urls.py*.

Для обработки запросов к API могут использоваться представления как в виде **функций**, так и в виде **классов**. Функции, как правило, позволяют лучше понять как работает код; вот с них и начнём.

View-функции API

Напишем простую view-функцию API с именем `hello()`, которая при GET-запросе будет возвращать вот такой ответ в формате JSON:

Скопировать код
JSON

```
{
    "message": "Это был GET-запрос!"
}
```

При POST-запросе эта view-функция будет возвращать JSON, содержащий те же данные, которые были отправлены в запросе: что пришло — то и возвращаем:

Скопировать код
JSON

```
{
    "message": "Получены данные",
    "data": {
        <JSON из тела POST-запроса>
    }
}
```

С view-функциями вы знакомы по опыту работы с Django-приложениями; view-функции API в Django REST framework применяются и выглядят сходным образом.

Декоратор @api_view

Для «настройки» view-функции на работу с API в Django REST framework есть декоратор **@api_view**. В качестве аргумента декоратору передают список типов HTTP-запросов, которые должна обрабатывать эта функция:

Скопировать код
PYTHON

```
@api_view([<разрешённые типы HTTP-запросов>])
def hello(request):
...
```

Ещё одно отличие view-функции API в Django REST framework состоит в том, что они возвращают специальный объект класса **Response**; в этот объект в качестве аргумента передаётся Python-словарь, данные из которого и должны быть отправлены в ответ на запрос в JSON формате.

Опишем API-функцию `hello()`:

Скопировать код
PYTHON

```
from rest_framework.decorators import api_view  # Импортировали декоратор
from rest_framework.response import Response  # Импортировали класс Response


@api_view(['GET', 'POST'])  # Применили декоратор и указали разрешённые методы
def hello(request):
    # По задумке, в ответ на POST-запрос нужно вернуть JSON с теми данными,
    # которые получены в запросе.
    # Для этого в объект Response() передаём словарь request.data.
    if request.method == 'POST':
        return Response({'message': 'Получены данные', 'data': request.data})

    # В ответ на GET-запрос нужно вернуть JSON
    # Он тоже будет создан из словаря, переданного в Response()
    return Response({'message': 'Это был GET-запрос!'})
```

Сейчас в декораторе `@api_view` указаны два разрешённых метода — GET и POST. В ответ на запросы других типов view-функция вернёт ответ *“Error 405 Method Not Allowed”* — это предусмотрено в декораторе @api_view.

В ответ view-функции добавлены ключи `message` и `data`, имена ключей могут быть любыми строками.

View-функция `hello()` достаточно проста — она просто возвращает полученные данные, не обрабатывая и даже не проверяя их: что передали — на том и спасибо.

В реальном мире view-функции обычно сложнее, например они могут создавать новые записи в БД или наоборот: в ответ на запрос возвращать данные, полученные из БД.

На примере view-функции `cat_list()` из проекта **Kittygram** разберёмся, как работает этот механизм.

View-функция API: обработка POST-запроса с одним объектом

Начнём с создания записи в базе данных об одном котике через API.

Модель **Cat** содержит несколько полей:

- имя котика: `name`,
- его цвет: `color`,
- его год рождения: `birth_year`.

Чтобы добавить в БД запись о новом котике, нужно отправить POST-запрос на эндпоинт `cats/`. В теле POST-запроса должен быть передан объект в формате JSON; содержание этого объекта должно соответствовать модели Cat.

JSON, переданный в POST-запросе, может быть таким:

Скопировать код
PYTHON

```
# Тело POST-запроса
{
    "name": "Стёпа",
    "color": "белый",
    "birth_year": 1971
}
```

Прежде чем сохранить полученные данные в БД, их нужно десериализовать: удостовериться, что они соответствуют ожиданиям модели **Cat** и конвертировать из простых типов данных в объект.

Для этого потребуется сериализатор; он уже описан в проекте, это класс **CatSerializer**.

Скопировать код
PYTHON
```python
# kittygram/cats/serializers.py
from rest_framework import serializers

from .models import Cat



class CatSerializer(serializers.ModelSerializer):
    class Meta:
        model = Cat
        # В прошлом уроке fields = '__all__' изменили на:
        fields = ('name', 'color', 'birth_year')
```

Чтобы подключить сериализатор к обработке данных, во view-функции `cat_list()` нужно создать экземпляр класса CatSerializer и передать в него данные из тела POST-запроса.

Данные в запросе приходят в формате JSON, преобразуются в Python-словарь, доступ к которому можно получить через объект `request.data`. Этот словарь и передаётся в сериализатор через именованный параметр `data`.

Скопировать код
PYTHON
```python
# kittygram/cats/views.py
from .serializers import CatSerializer


@api_view(['GET', 'POST'])
def cat_list(request):
    # Обработчик для POST-запросов.
    if request.method == 'POST':
        serializer = CatSerializer(data=request.data)


# GET-запросы разрешены в декораторе, но для них нет обработчика,
# поэтому в ответ на GET-запрос вернётся ошибка
# 500 Internal Server Error.
# Допишем этот обработчик позже.
```

Теперь данные, полученные в запросе, можно провалидировать.

Валидация входящих данных

Для валидации полученных данных надо вызвать в объекте сериализатора `serializer` метод `is_valid()`. В зависимости от результатов валидации можно среагировать на запрос по-разному:

- если валидация прошла успешно — сохраним запись в БД при помощи метода `save()` и в качестве подтверждения вернём в ответе созданный объект и статус-код, соответствующий успешному выполнению операции;
- если валидация не пройдена — вернём в ответе объект `serializer.errors`. В этом объекте сериализатор автоматически создаёт словарь с перечнем ошибок, возникших при валидации. Вместе с перечнем ошибок вернём и статус-код, соответствующий неудачному результату выполнения операции.

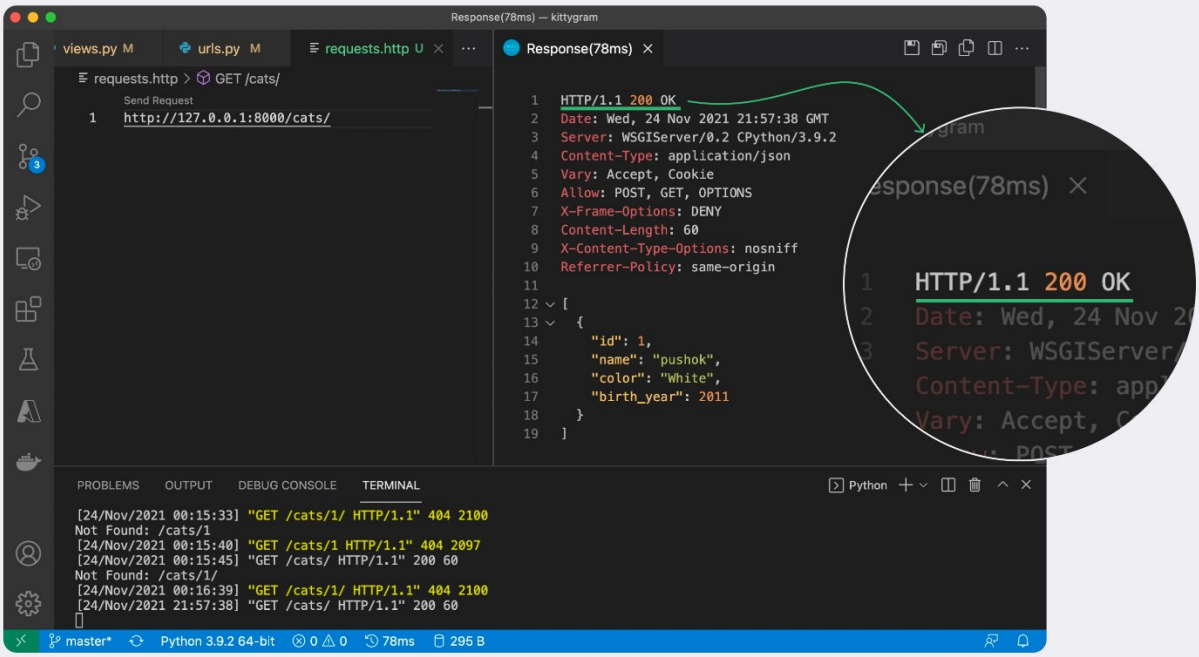
При разработке Rest API хорошей практикой считается всегда возвращать соответствующий ситуации статус-код ответа.

Если в коде не указать статус ответа — он всё равно будет отправлен автоматически; однако в таком случае может вернуться код, не отражающий реальное состояние дел.

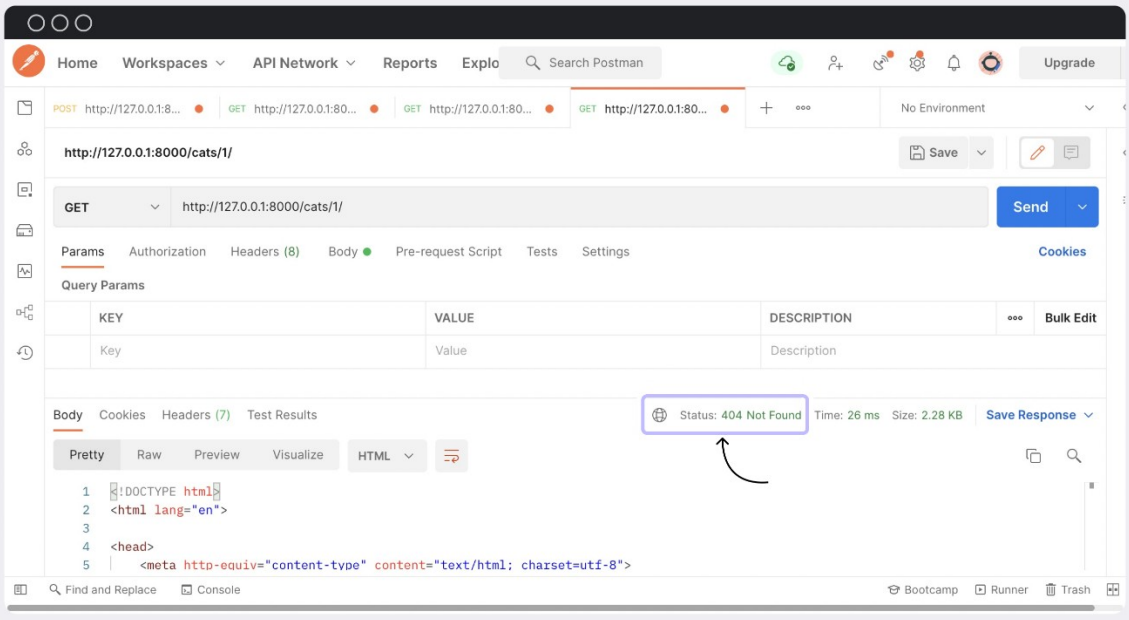
Список возможных кодов и их описание можно посмотреть здесь, в документации Django REST framework.

Выбирайте и указывайте любой подходящий к ситуации статус-код; какой бы статус вы не указали — он будет успешно отправлен в ответе. Даже если вместо статуса 200 вы отправите код 404 — ничего не сломается. Но лучше так не шутить: не все клиенты будут рады.

Посмотрите на пример ответа API: первая строка — это Status-Line, в ней передаётся информация об общем результате запроса. В начале строки — версия протокола, за ней — код и текстовое описание статуса ответа.



В интерфейсе Postman строка Status-Line отображается в общей сводке о запросе:



Добавим в код валидацию данных. Теперь view-функция, которая добавляет в БД новую запись, будет выглядеть так:

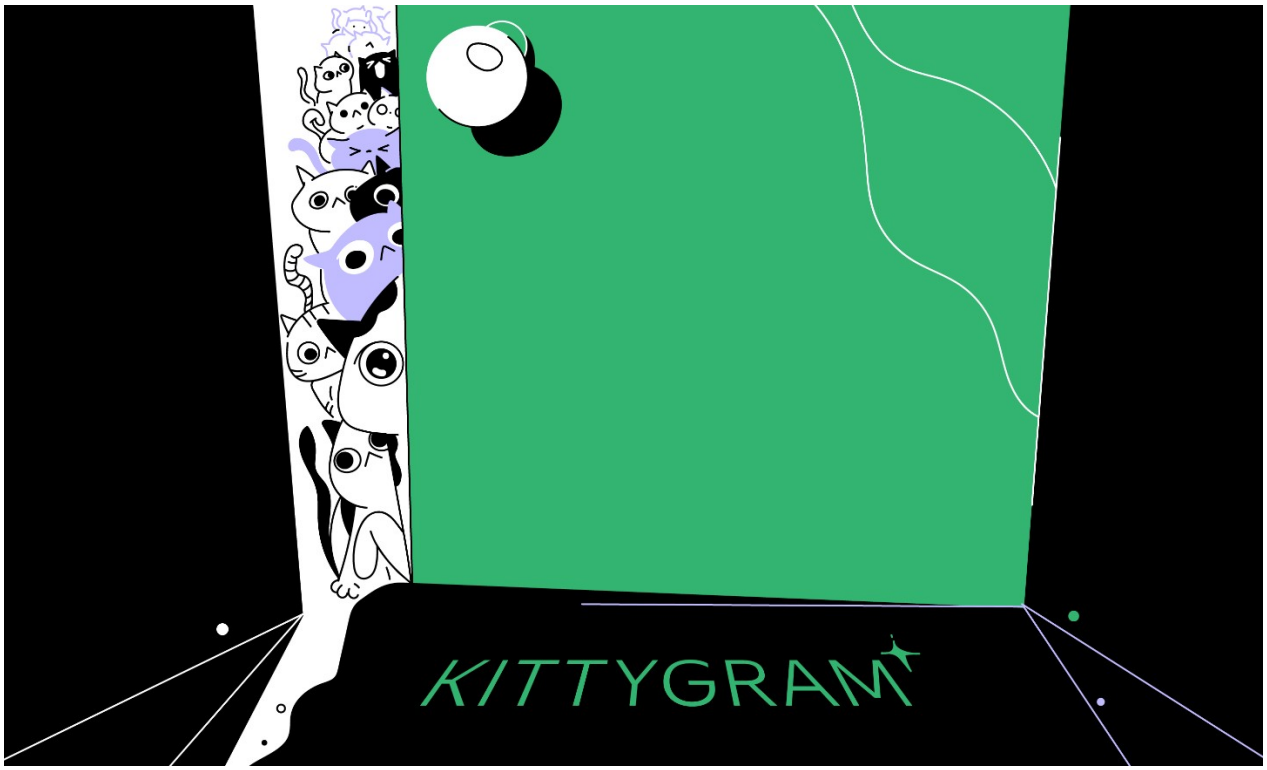
Скопировать код
PYTHON

```
@api_view(['GET', 'POST'])
def cat_list(request):
```

```
    if request.method == 'POST':
        # Создаём объект сериализатора
        # и передаём в него данные из POST-запроса
        serializer = CatSerializer(data=request.data)
        if serializer.is_valid():
            # Если полученные данные валидны —
            # сохраняем данные в базу через save().
            serializer.save()
            # Возвращаем JSON со всеми данными нового объекта
            # и статус-код 201
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        # Если данные не прошли валидацию —
        # возвращаем информацию об ошибках и соответствующий статус-код:
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

View-функция API: обработка POST-запроса со списком объектов

Следующая задача — научить view-функцию принимать и обрабатывать список объектов. Такой подход будет востребован: если владелец котофермы захочет добавить в базу всех своих котов, ему не придётся отправлять сотню отдельных запросов.



POST-запрос на добавление нескольких объектов будет выглядеть иначе, чем в случае с одним объектом: в теле POST-запроса будет передан не объект с полями `name`, `color` и `birth_year`, а **список** таких объектов.

Скопировать код
JSON

```
# Пример JSON со списком объектов


[
    {
        "name": "Стёпа",
        "color": "белый",
        "birth_year": 1971
    },
    {
        "name": "Мурка",
        "color": "рыжий",
        "birth_year": 2010
    },
    {
        "name": "Пушок",
        "color": "чёрный",
        "birth_year": 2018
    }
]
```

Чтобы сериализатор был готов принять список объектов, в конструктор сериализатора нужно передать именованный параметр `many=True`.

Скопировать код
PYTHON

```
serializer = CatSerializer(data=request.data, many=True)
```

Запрос со списком объектов будет обработан по тому же принципу, что и запрос с одним объектом.

Если этот параметр не указан, сериализатор не станет обрабатывать список объектов и вернёт ошибку: *"Invalid data. Expected a dictionary, but got list."*

Если в сериализаторе указан параметр `many=True`, а в запросе передан отдельный объект вместо списка объектов — сериализатор вернёт ошибку: *"Expected a list of items but got type dict."*

Обработка GET-запроса на получение списка объектов

Вернуть список объектов в ответ на соответствующий GET-запрос можно с помощью той же view-функции.

При подготовке ответа view-функция должна получить из базы данных **queryset**, в котором будут храниться запрошенные объекты модели (например, все объекты модели **Cat**).

Затем queryset нужно сериализовать. Для этого queryset передаётся первым аргументом в конструктор сериализатора; вторым аргументом будет `many=True`:

Скопировать код
PYTHON

```
# Получаем все объекты модели
cats = Cat.objects.all()
# Передаём queryset в конструктор сериализатора
serializer = CatSerializer(cats, many=True)
```

В результате view-функция API `cat_list()` примет такой вид:

Скопировать код
PYTHON

```
@api_view(['GET', 'POST'])  # Разрешены только POST- и GET-запросы
def cat_list(request):
    # В случае POST-запроса добавим список записей в БД
    if request.method == 'POST':
        serializer = CatSerializer(data=request.data, many=True)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
    # В случае GET-запроса возвращаем список всех котиков
    cats = Cat.objects.all()
    serializer = CatSerializer(cats, many=True)
    return Response(serializer.data)
```

Если при обработке GET-запроса в сериализаторе не указать параметр `many=True`, вернётся ошибка *AttributeError*.

Создать vs обновить

В результате вызова метода `save()` сериализатора может быть создана новая запись в БД, а может быть обновлена существующая запись.

Для создания новой записи в сериализатор передаются только данные из запроса; объект будет создан при вызове метода `save()`.

Скопировать код
PYTHON

```
...
serializer = CatSerializer(data=request.data)
# Если вызвать serializer.save(), будет создана новая запись в БД
```

Для обновления существующей записи первым параметром в сериализатор передаётся тот объект модели, который нужно обновить. В этом случае вызов `save()` не приведёт к созданию нового объекта.

Скопировать код
PYTHON

```
...
cat= Cat.objects.get(id=id)
serializer = CatSerializer(cat, data=request.data)
# Если вызвать serializer.save(), будет обновлён существующий экземпляр Cat
```

Сериализатор и метод PATCH

В проекте Kittygram может возникнуть необходимость обновить существующие записи. Например, котик Анатолий 2018 года рождения был серый, но его перекрасили в рыжий. Надо обновить запись в базе данных, но потребуется заменить значение лишь одного поля, не перезаписывая весь объект целиком.

Предположим, запись об Анатолии хранится в базе данных под id=15. Для частичного обновления данных следует отправить PATCH-запрос на эндпоинт http://127.0.0.1:8000/cats/15/; в теле этого запроса будет описание лишь одного поля:

Скопировать код

```
JSON
{
  "color": "Рыжий"
}
```

По умолчанию сериализатор ожидает получить значения всех полей, перечисленных в его параметре `fields`. Если же сериализатор получит не все значения — это вызовет ошибку *400 Bad Request "This field is required."*: «в запросе не передано обязательное поле!».

Причиной такой ошибки может стать PATCH-запрос.

Если при создании экземпляра сериализатора указать аргумент `partial=True` — отсутствие в запросе обязательных полей не приведёт к ошибке.

```
Скопировать код
PYTHON
serializer = CatSerializer(cat, data=request.data, partial=True)
```

Теперь котов можно перекрашивать хоть каждый день — ошибок в сериализаторе это не вызовет.

Разделение обязанностей между view-функциями API

Классический API для целевой модели — это, как правило, реализация шести операций:

- создание нового объекта;
- получение информации об объекте;
- удаление объекта;
- замещение объекта (целиком);
- изменение одного или нескольких полей объекта;
- получение списка объектов.

Хорошая практика — сгруппировать эти операции в две view-функции.

Первая view-функция добавляет новые объекты в коллекцию или возвращает все объекты коллекции, например:

- POST-запрос на адрес `cats/` создаст новую запись о котике;
- GET-запрос на тот же адрес вернёт список всех котиков.

Вторая view-функция обрабатывает запросы для получения, изменения (полного или частичного) и удаления одиночного объекта:

- GET-запрос к адресу `cats/<pk>/` вернёт информацию о конкретном котике по его id;
- запросы PUT, PATCH или DELETE к тому же адресу перезапишут, изменят или удалят существующую запись о котике.

В результате для всех шести действий в *urls.py* потребуется описать лишь два эндпоинта:

- `cats/`,
- `cats/<int:pk>/`.

  А ссылаться эти эндпоинты будут на две view-функции во *views.py*, например:

- `cat_list()`,
- `cat_detail()`.

Время кодить

Пришло время самостоятельно реализовать CRUD на Django REST framework с помощью view-функций API.

При работе держите в голове важные детали:

- view-функция обрабатывает только те запросы, которые указаны в декораторе `@api_view([<разрешённые HTTP-методы>])`;
- данные из POST-запроса сохраняются в словаре `request.data` и передаются в конструктор сериализатора через именованный параметр **data**: `data=request.data`;
- вместе с ответом необходимо вернуть клиенту правильный статус-код.

Как вам урок?

Перейти к заданию

Задание

Опишите view-функцию API `api_posts()`, которая будет работать с объектами модели **Post** и обрабатывать только GET- и POST-запросы:

- в ответ на GET-запрос функция должна возвращать queryset со всеми объектами модели **Post**;
- при POST-запросе функция должна создавать новый объект и возвращать его.

В файл *urls.py* добавьте эндпоинт `api/v1/posts/`, который вызывает view-функцию API `api_posts`.

Заранее предусмотрим в проекте возможность дальнейшего развития: будем версионировать API. Начнём с первой версии: эндпоинты API будут выглядеть так: `api/v1/...`.

В файле *serializers.py* подготовлен сериализатор `PostSerializer`.

```
# Импортируйте в код всё необходимое


urlpatterns = [
    path( ... ),
]
```

View-классы API

Следующий этап работы с Views в **Django REST framework —** встроенные **view-классы**. У них множество преимуществ перед view-функциями:

- возможность применять готовый код для решения стандартных задач;
- наследование, которое позволяет повторно использовать уже написанный код.

Встроенные классы DRF можно условно разделить на низкоуровневые и высокоуровневые. **Низкоуровневые** содержат лишь базовую структуру класса, его скелет; разработчик сам должен описать работу класса; их применяют для решения нестандартных задач.

Типовые задачи (скажем, CRUD) удобнее решать с помощью **высокоуровневых** view-классов: в них уже заготовлены все инструменты для решения стандартных задач.

Низкоуровневые view-классы в DRF

Начнём с низкоуровневого view-класса **APIView** из модуля `rest_framework.views`.

Если view-класс унаследован от класса `APIView`, то при получении GET-запроса в классе будет вызван метод `get()`, а при получении POST-запроса — метод `post()`. Такие методы описаны для всех типов запросов, но по умолчанию эти методы не выполняют никаких действий, их нужно описывать самостоятельно.

Скопировать код
PYTHON

```python
# Скелет есть, а кода нет. Надо самостоятельно описать необходимые методы.
class MyAPIView(APIView):
    def get(self, request):
        ...

    def post(self, request):
        ...
```

```
    def put(self, request):
        ...

    def patch(self, request):
        ...

    def delete(self, request):
        ...
```

В целом этот класс работает так же, как и view-функции.

Практика: редактируем вместе

Измените проект Kittygram: вместо view-функции опишите view-класс.

Импортируйте класс **APIView** из *rest_framework.views*, создайте класс-наследник и переопределите в нём методы `get()` и `post()`. Код почти ничем не будет отличаться от того, что был во view-функции, но будет написан в объектно-ориентированном стиле.

Скопировать код
PYTHON

```python
# Обновлённый views.py
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

from .models import Cat
from .serializers import CatSerializer


class APICat(APIView):
    def get(self, request):
        cats = Cat.objects.all()
        serializer = CatSerializer(cats, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = CatSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
```

```
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Чтобы всё заработало, исправьте код в urls.py, ведь синтаксис вызова view-классов отличается от синтаксиса вызова view-функций.

Скопировать код
PYTHON

```python
# urls.py
from django.urls import path

from cats.views import APICat

urlpatterns = [
    path('cats/', APICat.as_view()),
]
```

Как и при работе со view-функциями, все операции CRUD при использовании view-классов принято разделять на 2 группы: в одном view-классе описывается создание нового объекта и запрос всех объектов (например класс APICat), а в другом классе — получение/изменение/удаление определённого объекта (например класс APICatDetail).

Внесите изменения в Kittygram, запустите проект и поработайте с запросами через Postman: API будет работать так же, как и со view-функциями.

Generic Views: высокоуровневые view-классы

Для типовых действий, например, для вывода списка объектов или для запроса объекта по id удобнее использовать **высокоуровневые** view-классы, «дженерики» (англ. *Generic Views*): в них уже реализованы все механизмы, необходимые для решения задачи.

Некоторые из **Generic Views** выполняют строго определённую задачу (например, обрабатывают только один тип запросов), другие — более универсальны и могут «переключаться» на разные задачи в зависимости от HTTP-метода, которым был отправлен запрос.

В дженериках задают всего два поля: `queryset` (набор записей, который будет обрабатываться в классе) и `serializer_class` (сериализатор,

который будет преобразовывать объекты в формат JSON). В DRF все *Generic Views* объединены в модуле `rest_framework.generics`. Полный список можно посмотреть в документации DRF.

Рефакторинг кода: пишем Kittygram на Generic Views

Операции, описанные во view-классе проекта Kittygram, типичны для любого API. Есть смысл переписать код, заменив низкоуровневый view-класс на дженерики.

Для работы возьмём два класса и на них реализуем все шесть операций классического API:

- комбинированный класс `ListCreateAPIView`: он возвращает всю коллекцию объектов (например, всех котиков) или может создать новую запись в БД;
- комбинированный класс `RetrieveUpdateDestroyAPIView`: его работа — возвращать, обновлять или удалять объекты модели по одному.

Это практическая работа: вносите изменения, описанные в этом уроке, в проект Kittygram, развёрнутый на вашем компьютере.

Импортируйте в код всё необходимое: `generics` из `rest_framework`, модель и сериализатор; затем опишите дженерики:

Скопировать код
PYTHON

```python
# Обновлённый views.py
from rest_framework import generics

from .models import Cat
from .serializers import CatSerializer


class CatList(generics.ListCreateAPIView):
    queryset = Cat.objects.all()
    serializer_class = CatSerializer
```

```
class CatDetail(generics.RetrieveUpdateDestroyAPIView):
    queryset = Cat.objects.all()
    serializer_class = CatSerializer
```

Измените вызов view-класса в urls.py:

Скопировать код
PYTHON

```
# Обновлённый urls.py
from django.urls import path

from cats.views import CatList, CatDetail

urlpatterns = [
    path('cats/', CatList.as_view()),
    path('cats/<int:pk>/', CatDetail.as_view()),
]
```

Теперь проект Kittygram поддерживает весь API CRUD для модели Cat (а не только получение списка всех котиков и добавление нового котика). При минимальном количестве изменений в коде мы сделали API для модели Cat в Kittygram! Да и кода стало меньше; а где меньше кода, там меньше ошибок.

В коде явным образом не описана обработка разных типов запросов: всё происходит «под капотом» view-классов. Внесите изменения в свой локальный проект Kittygram и отправьте запросы разного типа через Postman: убедитесь в работоспособности проекта.

Специализированные Generic Views

В коде Kittygram view-классы унаследованы от **комбинированных** дженериков: они выполняют все операции CRUD. Для решения нашей задачи именно комбинированные дженерики подходят лучше всего. Но в некоторых случаях применение комбинированных view-классов будет избыточным или даже опасным.

Например, при создании API «только для чтения» (как знакомый вам StarWarsAPI) лучше подключить специализированный `ListAPIView`, который выполняет ровно одно действие: выводит список объектов в

ответ на GET-запрос. Это лучше и с точки зрения безопасности, и с точки зрения отсутствия избыточного кода.

Есть ещё несколько специализированных view-классов DRF:

- `RetrieveAPIView` — возвращает один объект (обрабатывает только GET-запросы);
- `CreateAPIView` — создаёт новый объект (обрабатывает только POST-запросы);
- `UpdateAPIView` — изменяет объект (обрабатывает только PUT- и PATCH-запросы);
- `DestroyAPIView` — удаляет объект (обрабатывает только DELETE-запросы).

Эти классы описываются в коде точно так же, как и комбинированные view-классы модуля **rest_framework.generics**.

Что в итоге

Работа с view-классами в DRF отличается от работы с привычными view-классами в Django только сериализацией и отсутствием HTML-шаблонов.

Удобная шпаргалка по классам DRF. Добавьте эту страницу в закладки и заглядывайте туда регулярно: с каждым разом этот справочник будет становиться всё понятнее и полезнее.

Как вам урок?

Перейти к заданию

Задание

Напишите view-класс `APIPost()`, унаследовав его от APIView.

Он будет работать с queryset, содержащим все объекты модели **Post**. При POST-запросе этот класс должен создавать новый объект модели **Post** и возвращать его, а по GET-запросу должен возвращаться сериализованный список всех объектов модели **Post**.

```
#  Импортируйте в код всё необходимое
```

```
urlpatterns = [
    path( ... ),
]
```

Вьюсеты и роутеры

Предустановленный класс для работы с моделями: Viewset

В прошлом уроке вы реализовали все типовые операции CRUD, для этого потребовалось лишь два класса-дженерика и два эндпоинта. Шесть операций упакованы в два класса — неплохо, но можно лучше.

Всякий раз, когда снова потребуется реализовать CRUD, придётся снова и снова описывать тот же набор из аналогичных классов. А почему бы не описать всё в одном классе?

И когда у кого-то возник этот вопрос — появились **вьюсеты**.

Вьюсет (англ. *viewset*, «набор представлений») — это высокоуровневый view-класс, реализующий все операции CRUD; он может вернуть объект или список объектов, создать, изменить или удалить объекты.

Во вьюсеты встроена обработка разных типов запросов, работа с сериализаторами и моделями, фильтрация и пагинация результатов, возврат ошибок. Не нужно ничего придумывать: всё работает «из коробки».

В библиотеке `rest_framework` есть несколько разных вьюсетов, они хранятся в пакете `viewsets`.

Начнём с самого популярного вьюсета. Встречайте: **ModelViewSet**.

Универсальный ModelViewSet

Класс **ModelViewSet** может выполнять любые операции CRUD с моделью. От разработчика не требуется описывать методы для чтения и записи данных для модели: эти операции уже реализованы.

В классе, наследующемся от `ModelViewSet`, обязательно должны быть описаны два поля:

- в поле `queryset` задаётся выборка объектов модели, с которой будет работать вьюсет;
- в поле `serializer_class` указывается, какой сериализатор будет применён для валидации и сериализации.

И во второй раз перепишем Kittygram: вместо дженериков применим вьюсеты.

Это практическая работа: вносите изменения, описанные в этом уроке, в проект Kittygram, развёрнутый на вашем компьютере.

В начале работы нужно импортировать пакет `viewsets` и создать класс, наследующийся от **ModelViewSet**.

Скопировать код
PYTHON

```python
# views.py
from rest_framework import viewsets

from .models import Cat
from .serializers import CatSerializer


class CatViewSet(viewsets.ModelViewSet):
    queryset = Cat.objects.all()
    serializer_class = CatSerializer
```

Всё.

Класс, который обработает все шесть типичных действий, готов.

Класс `ModelViewSet` предоставляет отличный набор инструментов для всех востребованных операций при работе с моделями. В большинстве стандартных ситуаций он будет работать «из коробки».

Выгода очевидна.

Класс ReadOnlyModelViewSet: только чтение

В пакете rest_framework.viewsets есть похожий на `ModelViewSet`, но ограниченный в правах класс `ReadOnlyModelViewSet`. Он может только получать данные модели, а записывать и изменять — не может.

Этот класс полезен в ситуациях, когда требуется только выдавать данные по запросу, без возможности их изменить. В остальном `ReadOnlyModelViewSet` работает точно так же, как и `ModelViewSet`.

Роутеры

При работе с view-классами и дженериками каждый эндпоинт отдельно описывается в *urls.py*. Но для вьюсетов есть более удобный и экономичный инструмент — **роутеры** (англ. *routers*).

С помощью роутера для заданных вьюсетов создаются эндпоинты по маске адреса:

- `URL-префикс/` и
- `URL-префикс/<int:pk>`.

В DRF есть два стандартных роутера: **SimpleRouter** и **DefaultRouter**. Они очень похожи, начнём с первого.

Добавьте роутеры в Kittygram. В файл *urls.py* импортируйте класс `SimpleRouter` и создайте экземпляр этого класса.

Скопировать код
PYTHON

```python
# urls.py
from rest_framework.routers import SimpleRouter


router = SimpleRouter()
```

Чтобы роутер создал необходимый набор эндпоинтов, необходимо вызвать его метод register() (говорят «зарегистрировать эндпоинты»). В качестве аргументов этот метод принимает URL-префикс и название вьюсета, для которого создаётся набор эндпоинтов.

Скопировать код
PYTHON

```python
router.register('cats', CatViewSet)
```

После регистрации надо включить новые эндпоинты в список `urlpatterns`: перечень эндпоинтов будет доступен в `router.urls`.

Создание эндпоинтов через router выглядит так:

Скопировать код
PYTHON

```python
# urls.py
from rest_framework.routers import SimpleRouter

from django.urls import include, path

from cats.views import CatViewSet

# Создаётся роутер
router = SimpleRouter()
# Вызываем метод .register с нужными параметрами
router.register('cats', CatViewSet)
# В роутере можно зарегистрировать любое количество пар "URL, viewset":
# например
# router.register('owners', OwnerViewSet)
# Но нам это пока не нужно

urlpatterns = [
    # Все зарегистрированные в router пути доступны в router.urls
    # Включим их в головной urls.py
    path('', include(router.urls)),
]
```

Только что созданный роутер сгенерирует два эндпоинта:

- `cats/`,
- `cats/<int:pk>/`.

Теперь через эти эндпоинты будут доступны любые операции с моделью:

- POST-запрос на `cats/` создаст новую запись.
- Запросы PUT, PATCH или DELETE к адресу `cats/<pk>/` изменят или удалят существующую запись.
- GET-запрос на те же адреса вернёт список объектов или один объект.

Параметр name в эндпоинтах

В *urls.py* для каждого маршрута можно указать необязательный параметр `name`. Если этот параметр определён, то во view-функциях или view-классах через функцию `reverse()` можно получить соответствующие URL — это очень удобно и соответствует принципу DRY.

На практике вы уже работали с этими функциями при написании тестов для Django-проекта; при тестировании API `name` в эндпоинтах вам тоже понадобятся.

Роутеры сами автоматически создают `name` для каждого эндпоинта, его значение создаётся

- из имени модели, с которой работает вьюсет,
- и суффикса:
  - *-list* (для эндпоинта, работающего с коллекцией объектов)
  - или *-detail* (для эндпоинта, работающего отдельным объектом).

Например, для роутера, созданного для вьюсета `CatViewSet()`.

Скопировать код
PYTHON

```
class CatViewSet(viewsets.ModelViewSet):
    queryset = Cat.objects.all()
    serializer_class = CatSerializer
```

имена эндпоинтов будут такими:

Скопировать код
PYTHON

```
urlpatterns = [
    # Здесь имя "cat" взято из queryset,
    # с которым работает вьюсет CatViewSet
    path('cat/', ..., name='cat-list'),
    path('cat/<int:pk>/', ..., name='cat-detail'),
]
```

При работе с вьюсетами и роутерами вы можете столкнуться с ошибкой «не определён аргумент basename»:

Скопировать код
BASH

```
'basename' argument not specified, and could not automatically determine the name
from the viewset, as it does not have a '.queryset' attribute.
```

Речь идёт о необязательном аргументе роутера `basename`: в нём можно вручную указать префикс для параметра `name` в эндпоинтах, созданных роутером.

Например, можно переопределить префикс *cat* в `name='cat-list'` и `name='cat-detail'`

Скопировать код
PYTHON

```
router.register('cats', CatViewSet, basename='tiger')
```

В результате `name` для эндпоинтов будут начинаться с tiger:

Скопировать код
PYTHON

```
urlpatterns = [
    path('cat/', ..., name='tiger-list'),
    path('cat/<int:pk>/', ..., name='tiger-detail'),
]
```

Однако есть случаи, когда параметр `basename` обязательно должен быть указан. Это необходимо в тех случаях, когда queryset однозначно не задан во вьюсете, а определён через метод `get_queryset()`.

Скопировать код
PYTHON

```
# Если бы пользователи могли оставлять комментарии к котикам,
# то эндпоинт для работы с комментариями выглядел бы примерно так:
# cats/{cat_id}/comments/


class CommentViewSet(viewsets.ModelViewSet):
    serializer_class = CommentSerializer
    # queryset во вьюсете не указываем
    # Нам тут нужны не все комментарии, а только связанные с котом с id=cat_id
    # Поэтому нужно переопределить метод get_queryset и применить фильтр
    def get_queryset(self):
        # Получаем id котика из эндпоинта
        cat_id = self.kwargs.get("cat_id")
        # И отбираем только нужные комментарии
        new_queryset = Comment.objects.filter(cat=cat_id)
        return new_queryset
```

В подобных ситуациях создать `name` автоматически не получится, и параметр `basename` придётся указать явным образом.

SimpleRouter vs DefaultRouter

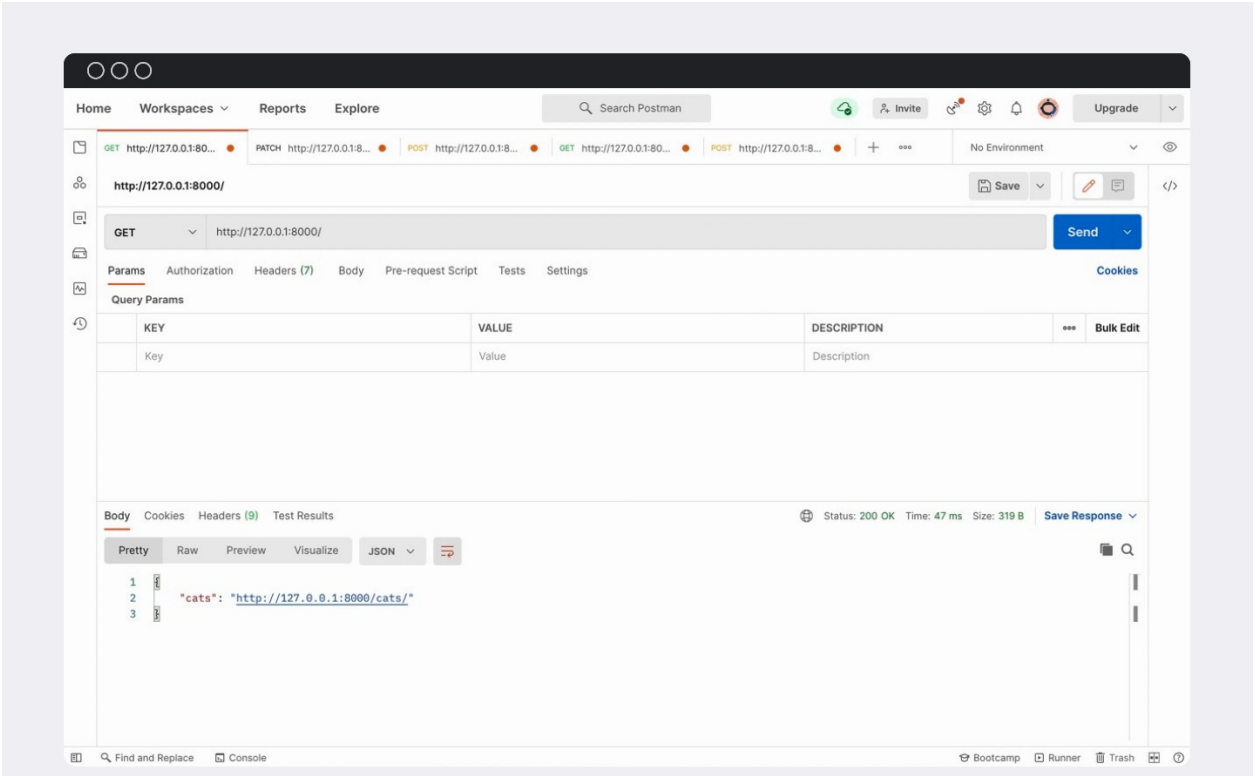
DefaultRouter — это расширенная версия SimpleRouter: он умеет всё то же, что и SimpleRouter, а в дополнение ко всему генерирует корневой эндпоинт `/`, GET-запрос к которому вернёт список ссылок на все ресурсы, доступные в API.

У этого эндпоинта тоже есть `name`: `api-root`. Маршрут для него мог бы выглядеть примерно так:

Скопировать код
PYTHON

```
urlpatterns = [
    ...
    path('', ..., name='api-root'),
]
```

Если применён DefaultRouter, то в ответ на GET-запрос к адресу http://127.0.0.1:8000/ вернётся список ссылок на доступные ресурсы.



При работе с документированием вам потребуется получить коллекцию ссылок на все ресурсы API, так что далее в курсе будем работать именно с **DefaultRouter**.

Сделайте паузу — сохраните в закладки шпаргалку с базой по сериализации, вьюсетам и роутерам. С ней будет проще решать тренажёрные задачи и выполнять финальные проекты.

Как вам урок?

Перейти к заданию

Задание

Напишите view-класс `PostViewSet()`, унаследовавшись от `ModelViewSet`, и создайте роутер для него.

Созданный роутер должен генерировать два эндпоинта:

- `api/v1/posts/`,
- `api/v1/posts/<int:pk>/`.

Через эти эндпоинты должны быть доступны любые операции с моделью Post:

- POST-запрос на `api/v1/posts/` создаст новую запись.
- Запросы PUT, PATCH или DELETE к адресу `api/v1/posts/<pk>/` изменят или удалят существующую запись.
- GET-запрос на те же адреса вернёт список объектов или один объект.

```
#  Импортируйте в код всё необходимое


...
urlpatterns = [
    path( ... ),
]
```

Сериализаторы для связанных моделей

Мы верим, что в прошлых уроках у вас всё получилось, и проект Kittygram теперь реализован на вьюсетах.

Ваш локальный Kittygram должен выглядеть примерно так, как наша «полуторная» версия, **Kittygram plus**. Синхронизируем версии: скачайте репозиторий с Kittygram plus и разверните его на компьютере. Дальше будем работать с ним.

Заодно сравните свой проект с нашим, это может быть интересно.

Дорабатываем Kittygram

В учебном проекте Kittygram описана лишь простая модель Cat; сериализаторы для таких моделей тоже довольно просты.

Но в реальных проектах моделей больше, они сложнее и практически всегда связаны друг с другом. Для таких структур придётся настраивать сериализаторы и вьюсеты более детально, забираясь им «под капот».

Ну что ж, полезли.

Начнём с сериализаторов: разберёмся со связанными и вложенными сериализаторами, узнаем, как «на лету» модифицировать данные в ответе.

Приручение котиков

Пора немного «прокачать» проект Kittygram в части бизнес-логики. Пусть у каждого котика будет владелец (*owner*), который станет его кормить и гладить.

Это практическая работа: вносите изменения, описанные в этом уроке, в проект Kittygram plus, развёрнутый на вашем компьютере.

Для начала создайте модель, в которой будут храниться данные о хозяевах-котоводах (имени и фамилии владельца будет достаточно):

Скопировать код
PYTHON

```python
from django.db import models


class Owner(models.Model):
    first_name = models.CharField(max_length=128)
    last_name = models.CharField(max_length=128)

    def __str__(self):
        return f'{self.first_name} {self.last_name}'

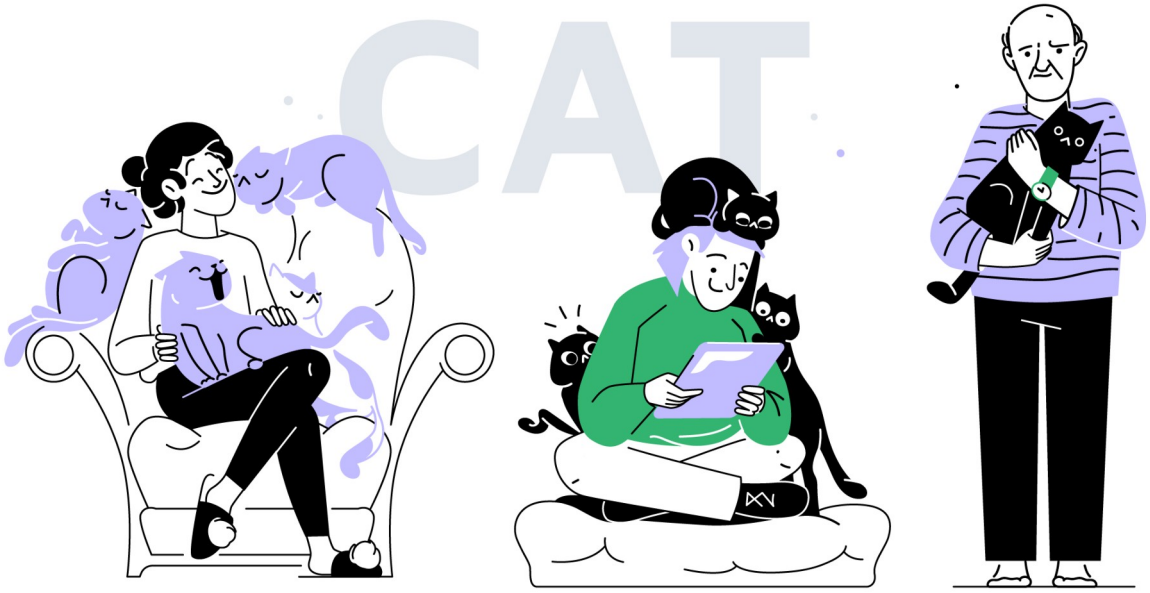

class Cat(models.Model):
    ...
```

Добавьте в модель **Cat** поле `owner`, оно будет связано с моделью **Owner**.

Связи в таблицах бывают разного типа:

- один-к-одному (OneToOne);
- один-ко-многим (ForeignKey);
- многие-ко-многим (ManyToMany).

В нашем случае у каждого владельца может быть много котиков, но у каждого котика может быть только один владелец. Такая связь называется «один-ко-многим».



Свяжите модель **Cat** через поле `owner` с моделью Owner:

Скопировать код
PYTHON

```python
# cats/models.py
from django.db import models


class Owner(models.Model):
    first_name = models.CharField(max_length=128)
    last_name = models.CharField(max_length=128)

    def __str__(self):
        return f'{self.first_name} {self.last_name}'


class Cat(models.Model):
    name = models.CharField(max_length=16)
    color = models.CharField(max_length=16)
    birth_year = models.IntegerField()
    # Новое поле в модели:
    owner = models.ForeignKey(
        Owner, related_name='cats', on_delete=models.CASCADE)

    def __str__(self):
        return self.name
```

Миграции с приключениями

После изменения моделей создайте миграции: `python3 manage.py makemigrations`. Но не всё так просто — при создании миграций возникнет вопрос:

Скопировать код
BASH

```
You are trying to add a non-nullable field 'owner' to cat without a default; we can't do that (the database needs something to populate existing rows).
Please select a fix:
 1) Provide a one-off default now (will be set on all existing rows with a null value for this column)
 2) Quit, and let me add a default in models.py
Select an option:
```

Суть конфликта проста. При миграции в БД будет создано новое поле `owner`; если бы в базе были записи — это поле было бы создано и для них. Поле `owner` — обязательное, значит, в существующих записях оно должно быть заполнено. «И чем же мы заполним поле `owner`, если в базе вдруг уже есть записи?» — спрашивает Django.

В момент создания миграций Django не знает, есть ли в модели Cat записи, и задаёт этот вопрос на всякий случай: если записи есть — что тогда делать?

Выберите первый вариант ответа: «Если в модели Cat существуют записи — заполни в них поле owner одним и тем же значением!» Пусть этим значением будет, например, `1`.

Нам это ничем не грозит: база данных пуста, никакие поля не придётся заполнять. А вот миграции успешно сформируются.

Скопировать код
BASH

```
Select an option: 1
Please enter the default value now, as valid Python
The datetime and django.utils.timezone modules are available, so you can do e.g. timezone.now
Type 'exit' to exit this prompt
>>> 1
Migrations for 'cats':
  cats/migrations/0002_auto_20210605_2128.py
```

```
    - Create model Owner
    - Add field owner to cat
```

Вот теперь можно применить миграции: `python3 manage.py migrate`.

Продолжаем работу: напишите сериализатор `OwnerSerializer` для модели **Owner**:

Скопировать код
PYTHON

```python
# cats/serializers.py
from rest_framework import serializers

from .models import Cat, Owner


class CatSerializer(serializers.ModelSerializer):

    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year')


class OwnerSerializer(serializers.ModelSerializer):

    class Meta:
        model = Owner
        fields = ('first_name', 'last_name')
```

Напишите вьюсет `OwnerViewSet`:

Скопировать код
PYTHON

```python
# cats/views.py
from rest_framework import viewsets

from .models import Cat, Owner
from .serializers import CatSerializer, OwnerSerializer


class CatViewSet(viewsets.ModelViewSet):
    queryset = Cat.objects.all()
    serializer_class = CatSerializer
```

```python
class OwnerViewSet(viewsets.ModelViewSet):
    queryset = Owner.objects.all()
    serializer_class = OwnerSerializer
```

Зарегистрируйте через роутер эндпоинты для нового ресурса:

Скопировать код
PYTHON

```python
# kittygram_plus/urls.py
from rest_framework.routers import DefaultRouter

from django.urls import include, path

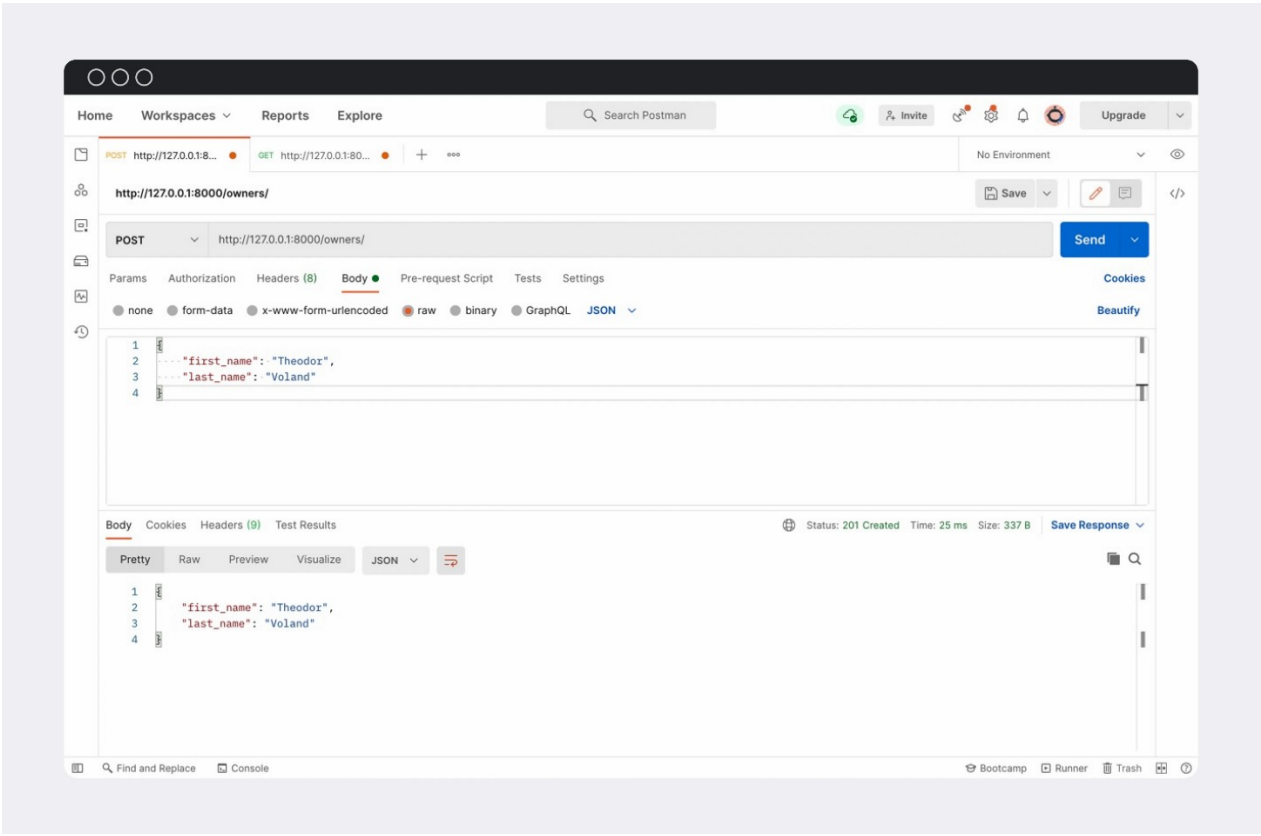
from cats.views import CatViewSet, OwnerViewSet


router = DefaultRouter()
router.register('cats', CatViewSet)
router.register('owners', OwnerViewSet)

urlpatterns = [
    path('', include(router.urls)),
]
```

Запустите веб-сервер разработки `python3 manage.py runserver` и добавьте через POST-запрос на эндпоинт `owners/` хотя бы одного хозяина котиков — он потребуется нам для дальнейших экспериментов.

Кстати, у нас мог бы зарегистрироваться Пабло Пикассо, он, судя по картинам, был неравнодушен к котикам. И Воланда можно пригласить: у него в услужении был большой чёрный кот.



Сериализаторы для связанных моделей

В модели **Owner** нет поля `cats`, но эта модель связана с моделью **Cat** через related_name `'cats'`. Сериализаторы могут работать с моделями, которые связаны друг с другом: можно указать имя `cats` в качестве поля сериализатора. Добавьте это поле в `OwnerSerializer`:

Скопировать код
PYTHON

```
class OwnerSerializer(serializers.ModelSerializer):


    class Meta:
        model = Owner
        fields = ('first_name', 'last_name', 'cats')
```

В модели Cat поле Owner вы уже создали, его надо добавить в сериализатор `CatSerializer`:

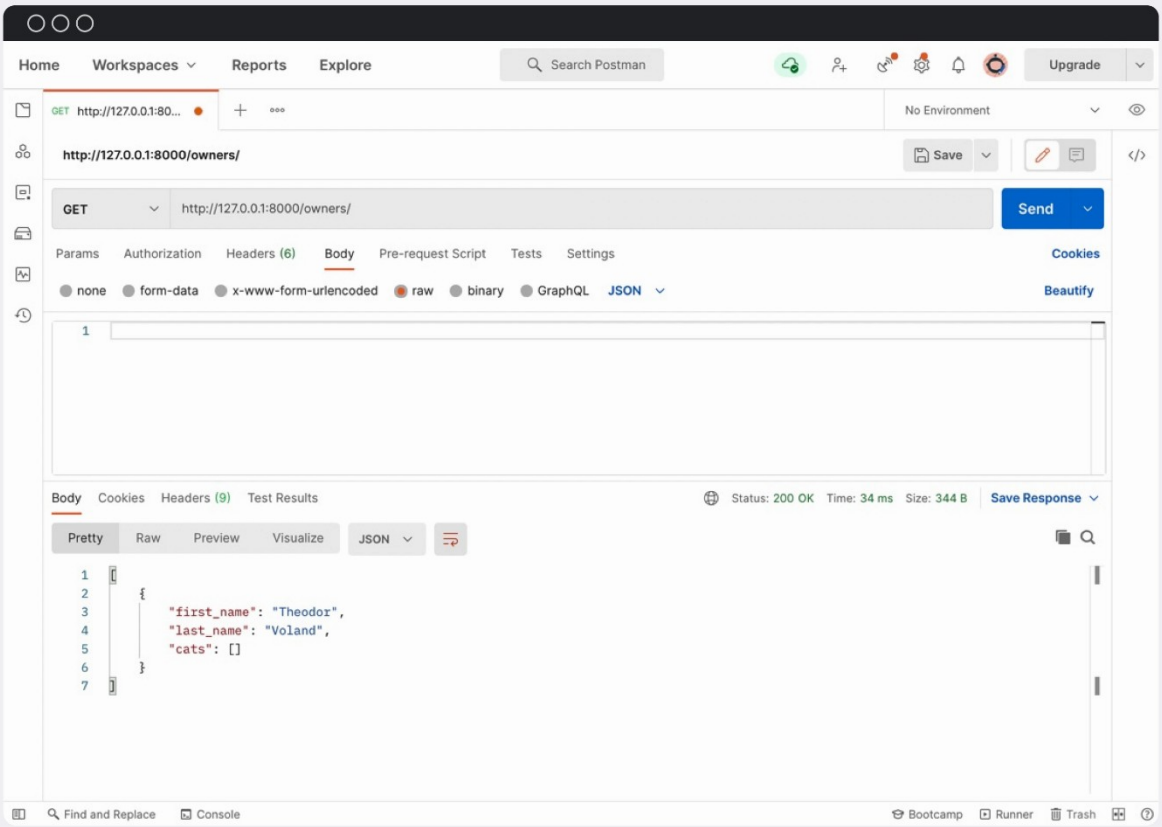
Скопировать код
PYTHON

```
class CatSerializer(serializers.ModelSerializer):


    class Meta:
```

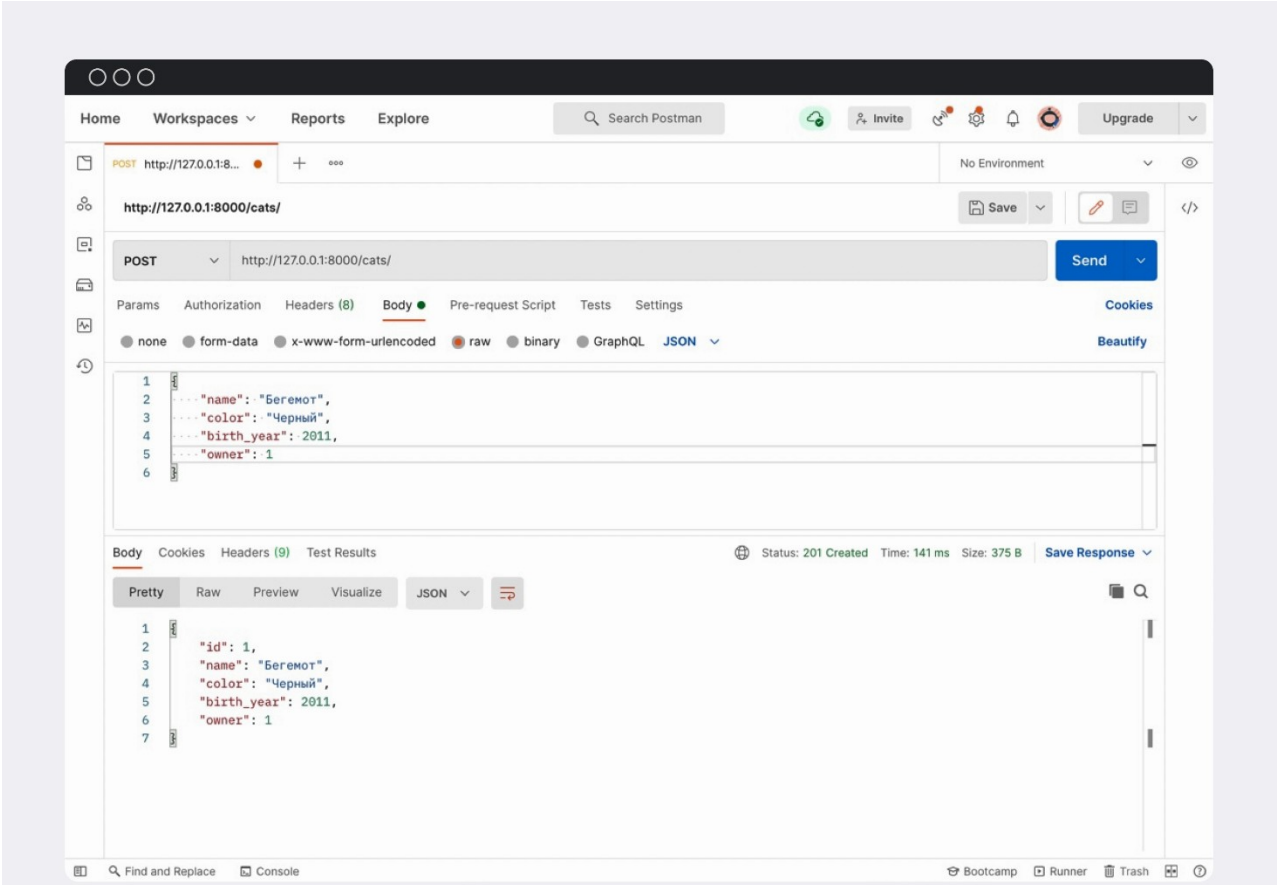
```
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year', 'owner')
```

Если теперь GET-запросом к эндпоинту owners/ запросить список всех хозяев котиков, то в ответе появится новое поле cats, но оно будет пустым: в базе нет ни одного котика.



POST-запросами на эндпоинт `cats/` добавьте в базу данных несколько котиков.

В теле запроса в поле *owner* передавайте id хозяина: если в базе данных всего один хозяин, его id равен 1.



По умолчанию для связанных полей модели сериализатор будет использовать тип `PrimaryKeyRelatedField`; этот тип поля в сериализаторе оперирует первичными ключами (id) связанного объекта.

Поэтому в POST-запросе мы указывали именно id хозяина, а не его `first_name` или `last_name`. А при GET-запросе к эндпоинту *owners/* в поле `cats` для каждого хозяина будет возвращаться список id связанных с ним котиков.

Скопировать код
PYTHON

```
[
    {
        "first_name": "Theodor",
        "last_name": "Voland",
        "cats": [
            1,
            2
        ]
    }
]
```

Для начала неплохо, но совсем не информативно: по id про объект ничего не узнаешь. Нужно изменить дефолтное поведение сериализатора и вернуть вместо id связанного объекта какую-то другую информацию об объекте.

В классе **Cat** описан метод `__str__`: для строкового представления объектов модели **Cat** используется содержимое поля `name`. Настроим сериализатор так, чтобы вместо непонятного id возвращалось строковое представление объекта.

Нам не трудно, а пользователю приятно.

Тип поля StringRelatedField

В сериализаторе `OwnerSerializer` переопределите тип поля `cats` с дефолтного `PrimaryKeyRelatedField` на `StringRelatedField`.

Роль `StringRelatedField` — получить строковые представления связанных объектов и передать их в указанное поле вместо id.

Скопировать код
PYTHON

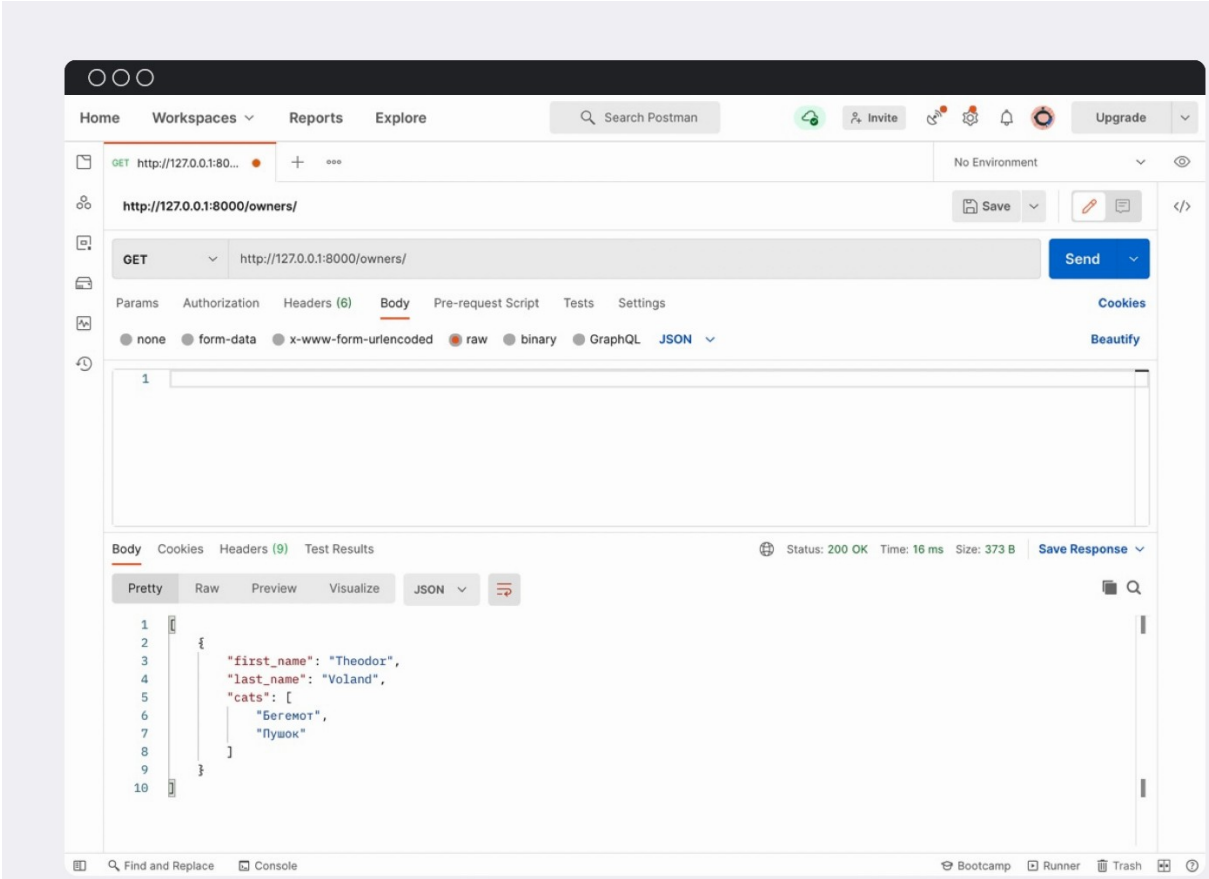
```
class OwnerSerializer(serializers.ModelSerializer):
    cats = serializers.StringRelatedField(many=True, read_only=True)

    class Meta:
        model = Owner
        fields = ('first_name', 'last_name', 'cats')
```

Обратите внимание, при указании типа поля были переданы аргументы `many=True` и `read_only=True`.

- Для поля `cats` в модели **Owner** установлена связь **«один-ко-многим»** (у одного хозяина может быть много котиков), следовательно, полю `cats` в сериализаторе надо разрешить обработку списка объектов. Для этого в нём указан аргумент `many=True`.
- Поля с типом StringRelatedField не поддерживают операции записи, поэтому для них всегда должен быть указан параметр `read_only=True`.

Снова сделайте GET-запрос к эндпоинту *owners/* и посмотрите, как изменился ответ:



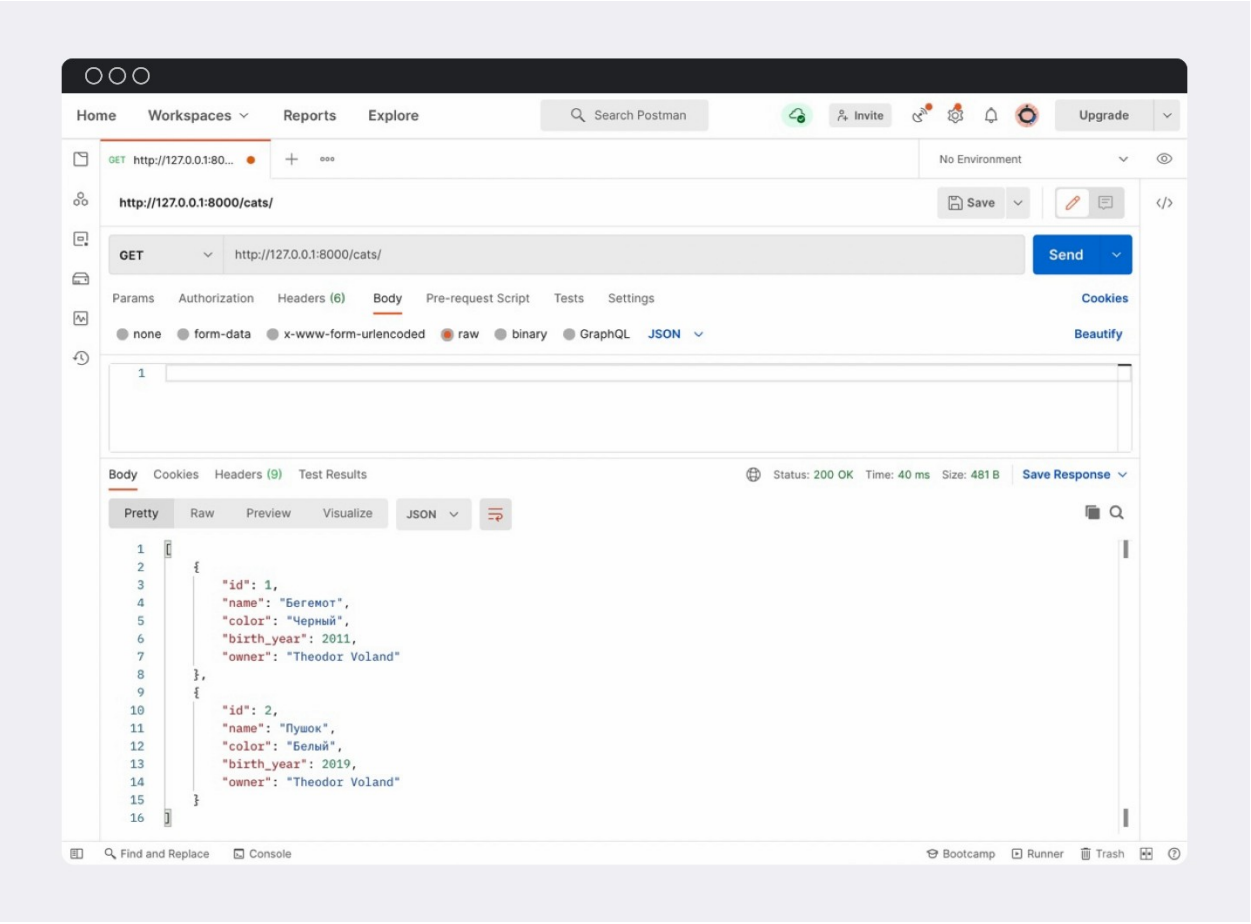
Теперь точно так же измените код сериализатора `CatSerializer`: переопределите поле `owner`. При запросе к эндпоинту cats/ в ответе должен отображаться не id хозяина котика, а строковое представление объекта модели Owner; параметр `many=True` в этом поле не нужен, ведь у котика может быть только один хозяин.

Скопировать код
PYTHON

```
class CatSerializer(serializers.ModelSerializer):
    owner = serializers.StringRelatedField(read_only=True)

    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year', 'owner')
```

Сделайте запрос на эндпоинт cats/. Если вы всё сделали правильно, то в каждом объекте котика в поле `owner` будет возвращаться **строковое представление** связанного с котиком объекта **Owner**:



Важно помнить, что если у указанной модели изменится строковое представление, то в случае использования StringRelatedField эти изменения отразятся и на ответах API.

Другие типы related-полей

Помимо `PrimaryKeyRelatedField` и `StringRelatedField` в сериализаторах можно использовать и другие типы связанных полей. Загляните в шпаргалку, там перечислены самые популярные из них. А весь список можно традиционно найти в документации.

Новые достижения

Кот не кот, если не натворил что-нибудь за день. Прокачаем проект: привяжем к каждому коту список его достижений.

Создайте модель **Achievement** (англ. «достижение»). В ней будут храниться описания подвигов, которые так любят совершать все

представители кошачьих: «уронил ёлку», «прогрыз провод», «разбил стакан», «напал из-за угла».

В модель **Cat** добавьте новое поле — `achievements` (англ. «достижения»), оно будет связано с моделью Achievement через вспомогательную модель AchievementCat — её тоже опишите в коде.

Скопировать код
PYTHON

```python
from django.db import models


class Achievement(models.Model):
    name = models.CharField(max_length=64)

    def __str__(self):
        return self.name


class Owner(models.Model):
    first_name = models.CharField(max_length=128)
    last_name = models.CharField(max_length=128)

    def __str__(self):
        return f'{self.first_name} {self.last_name}'


class Cat(models.Model):
    name = models.CharField(max_length=16)
    color = models.CharField(max_length=16)
    birth_year = models.IntegerField()
    owner = models.ForeignKey(
        Owner, related_name='cats', on_delete=models.CASCADE)
    # Связь будет описана через вспомогательную модель AchievementCat
    achievements = models.ManyToManyField(Achievement, through='AchievementCat')

    def __str__(self):
        return self.name

# В этой модели будут связаны id котика и id его достижения
class AchievementCat(models.Model):
```

```
    achievement = models.ForeignKey(Achievement, on_delete=models.CASCADE)
    cat = models.ForeignKey(Cat, on_delete=models.CASCADE)

    def __str__(self):
        return f'{self.achievement} {self.cat}'
```

Каждое достижение может принадлежать любому количеству котиков, и каждый котик может обладать любым количеством достижений; это связь **«многие-ко-многим»**.

После изменения моделей создайте и примените миграции.

Вложенный сериализатор

В работе со связанными моделями Cat и Owner мы получали лишь ссылку на связанный объект (это был его id или что-то, замещающее id), но сам объект так и оставался недоступен. Однако список достижений котика — это именно список связанных с котиком **объектов** из модели **Achievement**. Надо их как-то добыть.

Задача состоит в том, чтобы при запросе к эндпоинту *cats/* вместе с объектом котика вернуть **список**, состоящий из привязанных к этому зверю **объектов**.

Чтобы реализовать эту идею в сериализаторе, нужно вложить один сериализатор в другой: определить в сериализаторе поле, типом которого будет другой сериализатор. Таким образом вложенный сериализатор передаст в поле родительского сериализатора список объектов.

В нашем случае родительским сериализатором будет `CatSerializer`, а вложенным — `AchievementSerializer` (но сперва его надо написать).

Разомните пальцы: напишите сериализатор для модели **Achievement**:

Скопировать код
PYTHON
```
class AchievementSerializer(serializers.ModelSerializer):

    class Meta:
        model = Achievement
        fields = ('id', 'name')
```

Теперь перенастроим `CatSerializer`: переопределим в нём поле `achievements`. По дефолту к этому полю в сериализаторе будет применён тип `PrimaryKeyRelatedField`. Но в наших планах было получить не id объектов **Achievement**, а его объекты целиком.

Назначьте типом поля `achievements` сериализатор `AchievementSerializer`. Да, так можно!

Скопировать код
PYTHON

```
class AchievementSerializer(serializers.ModelSerializer):

    class Meta:
        model = Achievement
        fields = ('id', 'name')

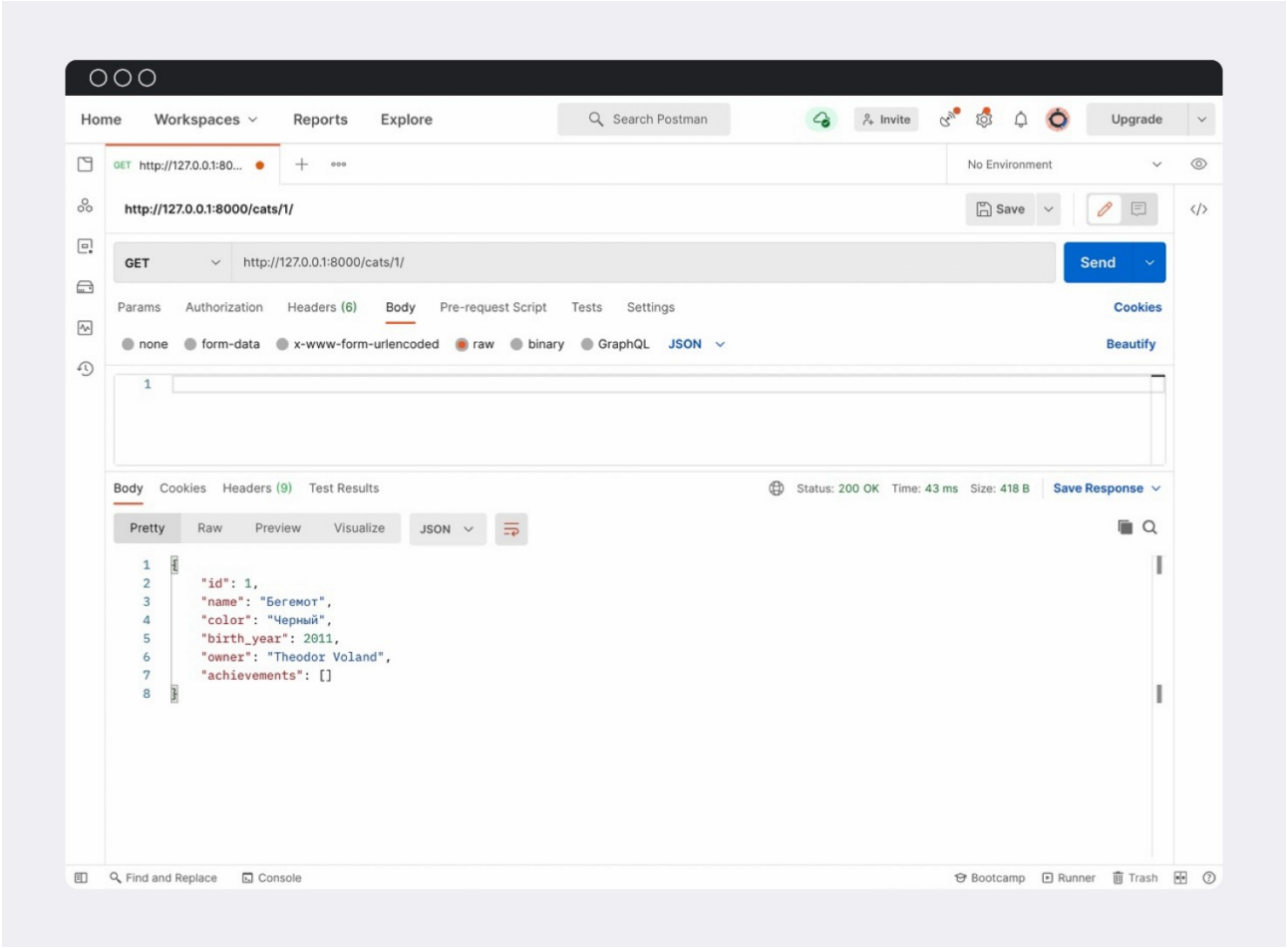

class CatSerializer(serializers.ModelSerializer):
    owner = serializers.StringRelatedField(read_only=True)
    # Переопределяем поле achievements
    achievements = AchievementSerializer(read_only=True, many=True)

    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year', 'owner', 'achievements')
```

Теперь поле `achievements` в CatSerializer будет получать объекты Achievement, сериализованные в AchievementSerializer.

Достижений у каждого котика может быть много, значит, полю achievements нужно передать аргумент many=True. И пока ограничим доступ к полю — «только для чтения», с записью разберёмся позже.

Проверьте результат: сделайте GET-запрос к конкретному котику:

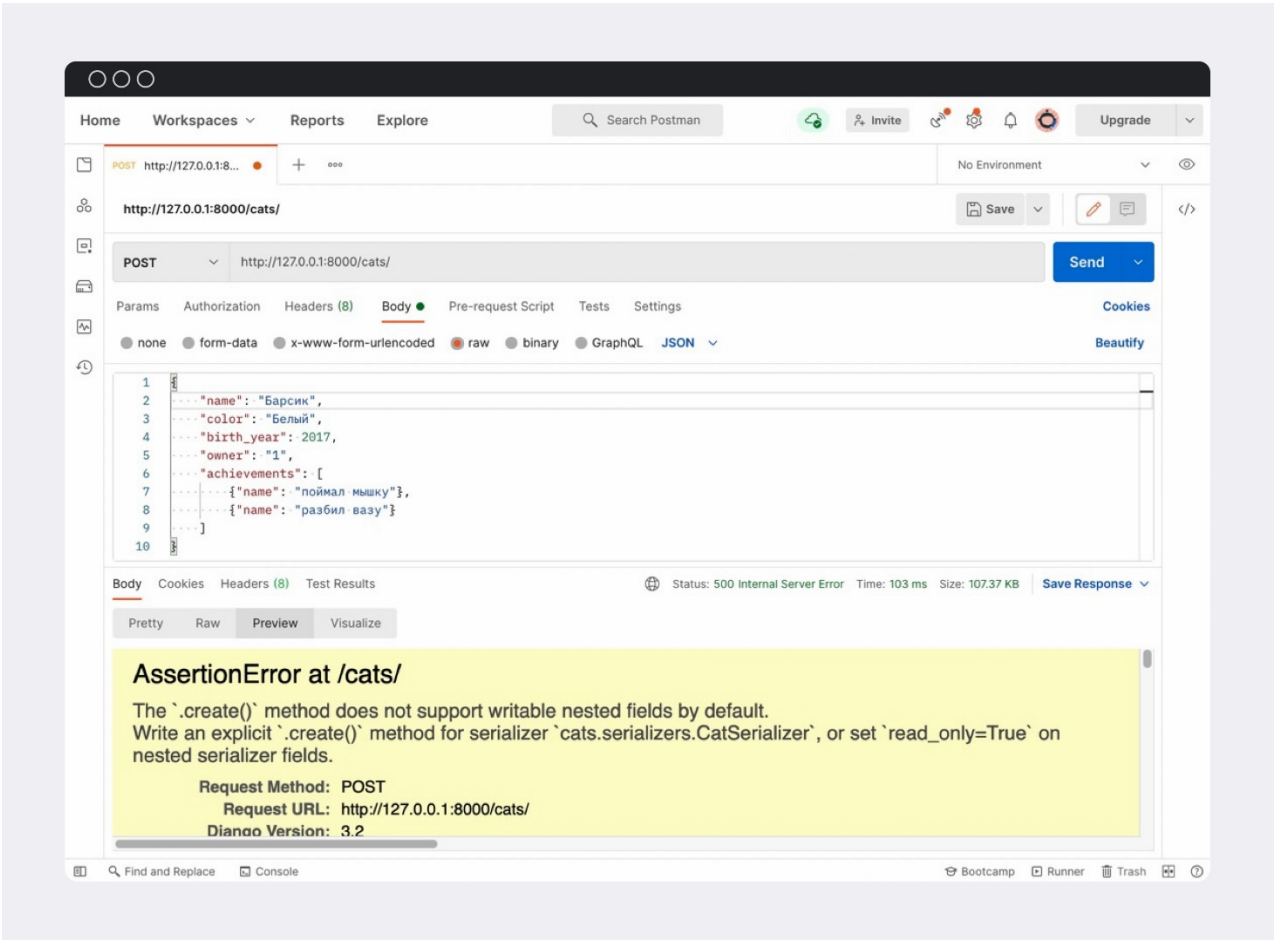


В ответе будет доступен список объектов из модели **Achievements**, привязанных к конкретному котику. Этот список пока что пуст. Пора записать в базу котиков с достижениями.

Операции записи с вложенными сериализаторами

Сейчас для поля `owner` модели **Cat** явным образом указан тип StringRelatedField; запись в него невозможна. Но в это поле потребуется записывать данные, поэтому удалите из кода строку, где это поле переопределяется: это вернёт полю его тип «по умолчанию».

Вложенные сериализаторы по умолчанию доступны только для чтения. Если из описания поля `achievements` убрать параметр `read_only=True` и отправить POST-запрос, передав данные о новом котике и список его достижений, возникнет ошибка:



В тексте ошибки будет предложено два варианта решения проблемы: либо установить для поля `achievements` аргумент `read_only=True`, либо явно описать в коде, как и куда следует сохранять полученные данные и отношения между ними — как связать котика с достижением. Установить параметр `read_only=True` — не выход: нам нужно записывать достижения в БД. Значит, надо описать, как должны сохраняться данные.

При получении такого POST-запроса

Скопировать код
PYTHON

```
{
    "name": "Барсик",
    "color": "White",
    "birth_year": 2017,
    "owner": 1,
    "achievements": [
        {"name": "поймал мышку"},
        {"name": "разбил вазу"}
    ]
}
```

порядок работы должен быть таким:

- из списка `serializer.validated_data` извлечь и сохранить в переменную элемент `achievements`: в нём хранится список достижений котика;
- в базе данных создать запись о новом котике — для этого у нас есть вся необходимая информация; достижения котика в этом не участвуют — лежат в стороне, ждут обработки;
- перебрать полученный список достижений котика и сравнить каждое из достижений с имеющимися в базе данных записями:
  - если проверяемый элемент уже есть в базе — в таблицу связей **AchievementCat** добавить связь этого достижения с новым котиком;
  - если проверяемого элемента в базе нет — в базе достижений создать новую запись и в таблицу связей **AchievementCat** добавить связь этого достижения с новым котиком.
- вернуть JSON с объектом свежесозданного котика и списком его достижений.

Чтобы настроить сохранение данных, нужно переопределить метод `create()` в сериализаторе. Опишем его и укажем явным образом, какие записи в каких таблицах нужно создать.

Скопировать код
PYTHON

```
from .models import Cat, Owner, Achievement, AchievementCat

...

class CatSerializer(serializers.ModelSerializer):
    achievements = AchievementSerializer(many=True)  # Убрали read_only=True
    # Убрали owner = serializers.StringRelatedField(read_only=True)

    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year', 'owner', 'achievements')

    ...

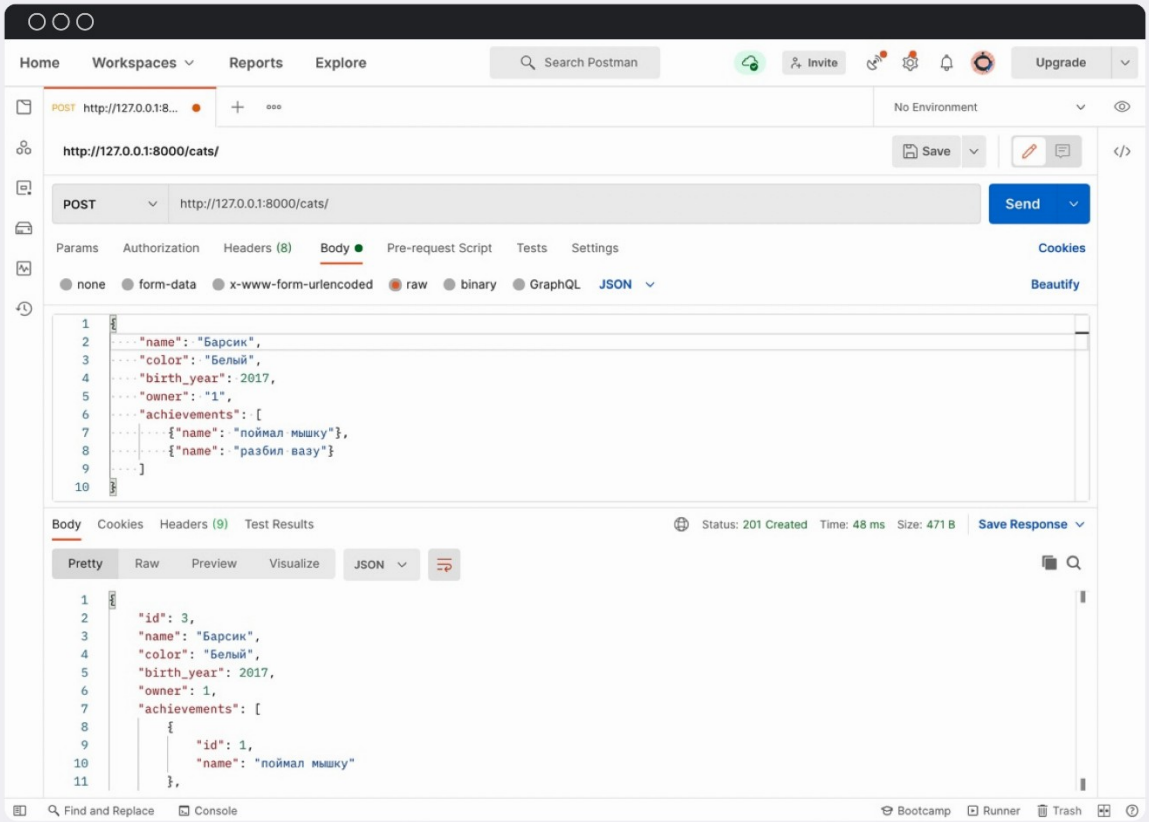
    def create(self, validated_data):
        # Уберём список достижений из словаря validated_data и сохраним его
```

```
        achievements = validated_data.pop('achievements')

        # Создадим нового котика пока без достижений, данных нам достаточно
        cat = Cat.objects.create(**validated_data)

        # Для каждого достижения из списка достижений
        for achievement in achievements:
            # Создадим новую запись или получим существующий экземпляр из БД
            current_achievement, status = Achievement.objects.get_or_create(
                **achievement)
            # Поместим ссылку на каждое достижение во вспомогательную таблицу
            # Не забыв указать к какому котику оно относится
            AchievementCat.objects.create(
                achievement=current_achievement, cat=cat)
        return cat
```

Снова сделаем запрос на добавление котика и списка его достижений:



Заработало!

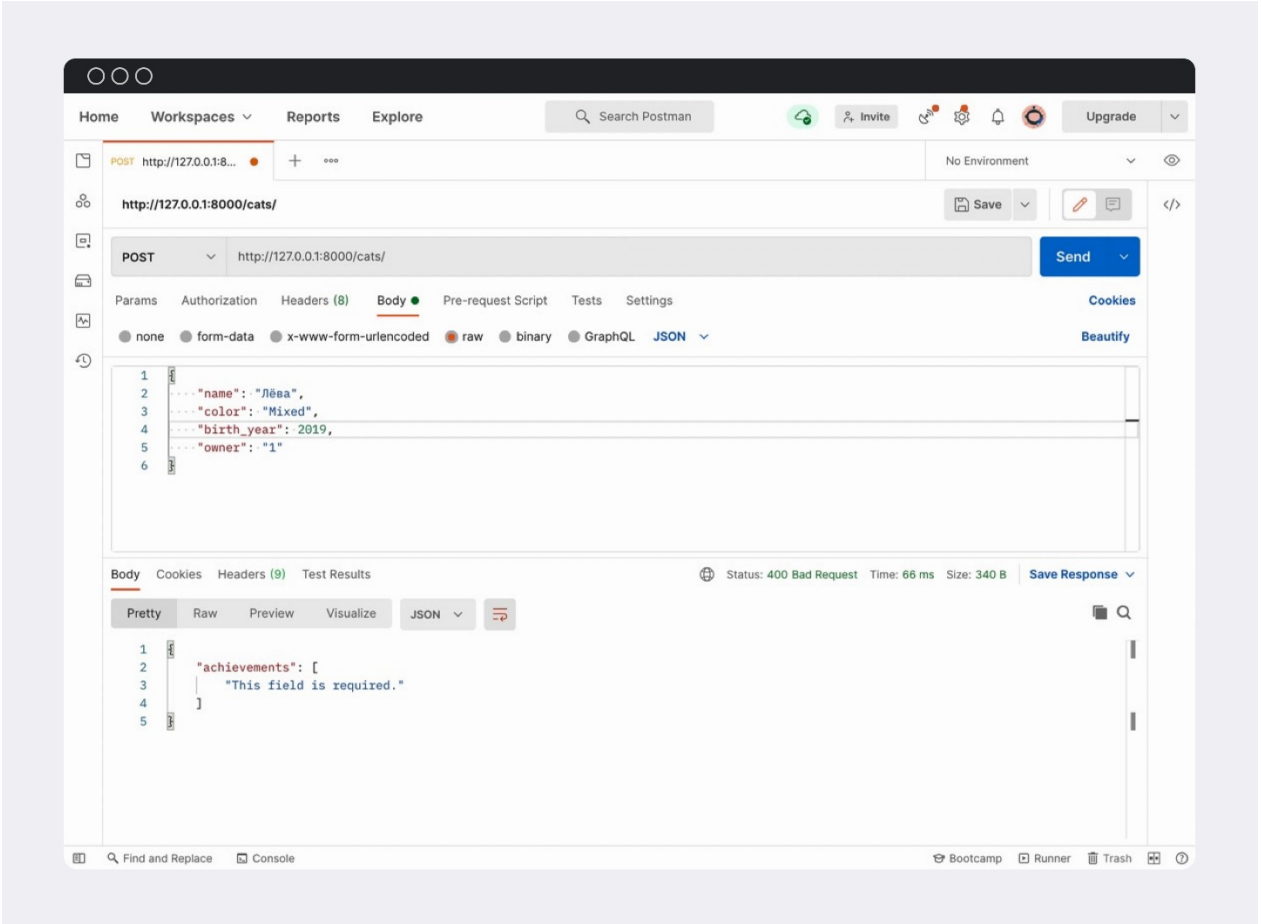
Теперь можно не только получать связанные объекты, но и отправлять их через POST-запросы — сериализатор обучен и знает, что с ними делать.

Чтобы использовать операции записи в поле со вложенным сериализатором, в родительском сериализаторе необходимо описать методы `create()` или `update()` (или сразу оба) и явно указать, как именно следует сохранять данные и связи между ними.

Исходные данные из запроса в сериализаторе

При работе с сериализаторами бывает полезен доступ к тем данным, которые были переданы в сериализатор: например, если нужно проверить, было ли передано в запросе какое-нибудь необязательное поле. Эти данные хранятся в словаре `serializer.initial_data`, и прямо сейчас они понадобятся.

Отправьте POST-запрос на добавления котика, но не указывайте список его достижений:



Вернётся ошибка: поле `achievements` является обязательным.

Если иное не определено на уровне модели или сериализатора **явным образом**, то все поля модели, перечисленные в сериализаторе, будут обязательными.

В модели **Cat** явным образом не указано, что поле `achievements` — необязательное. Сериализатор видит, что поле модели не описано как необязательное — и к собственному полю `achievements` применяет атрибут `required=True`.

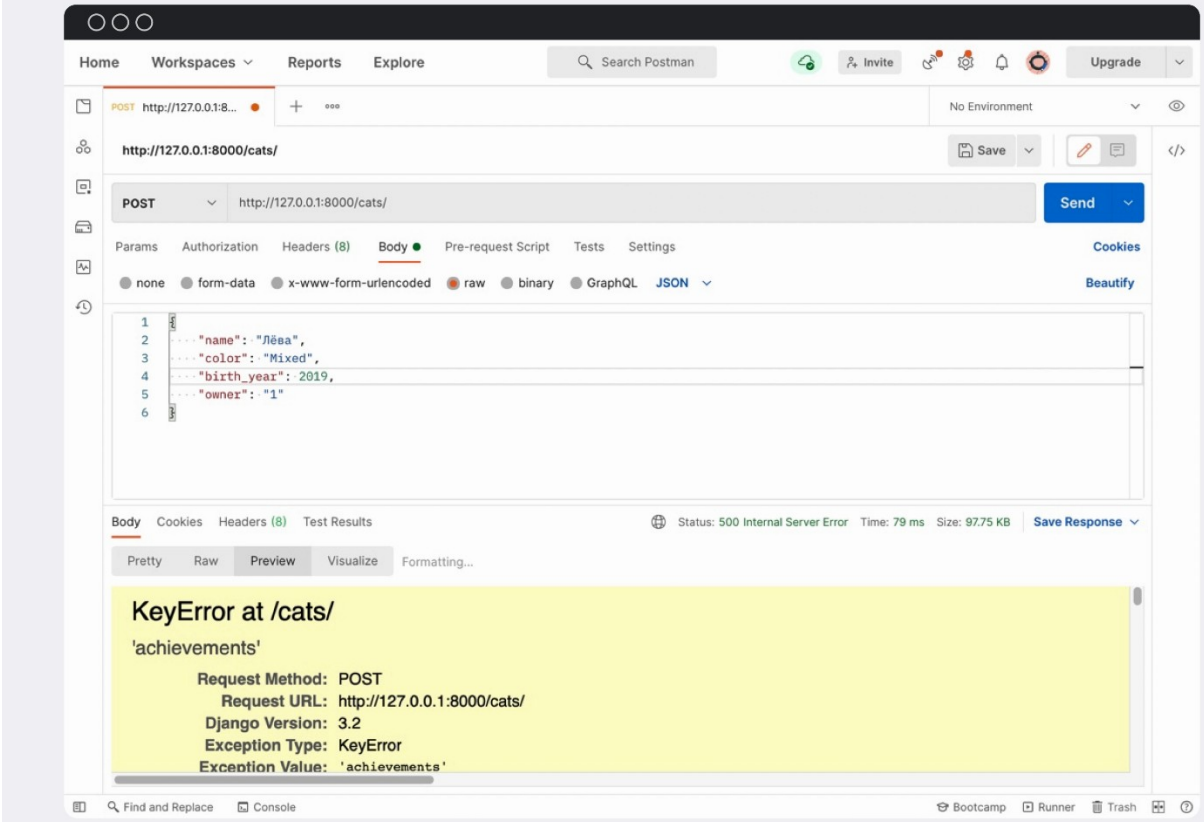
Но поле `achievements` нельзя назначать обязательным: бывают котики и без достижений, и не должно быть запрета на добавление таких котиков в Kittygram. Модель **Cat** это позволяет, но сериализатор не даёт этого сделать. Следовательно, нужно явным образом определить в сериализаторе поле `achievements` как необязательное.

Установим для поля `achievements` в сериализаторе атрибут `required` со значением `False`:

Скопировать код
PYTHON

```
class CatSerializer(serializers.ModelSerializer):
    achievements = AchievementSerializer(many=True, required=False)
    ...
```

Теперь сериализатор не будет беспокоиться, если этого поля нет в запросе, но возникнет другая ошибка:



Переопределённый метод `create()` в сериализаторе пытается сохранить данные из поля `achievements`, но поле теперь необязательное, данные в POST-запросе для сохранения не пришли — и всё сломалось.

Тут и понадобится словарь `initial_data`: в методе `create()` проверим, пришло в запросе поле `achievements` или нет, и, в зависимости от результата, будем сохранять котика с достижениями или без них.

Скопировать код
PYTHON

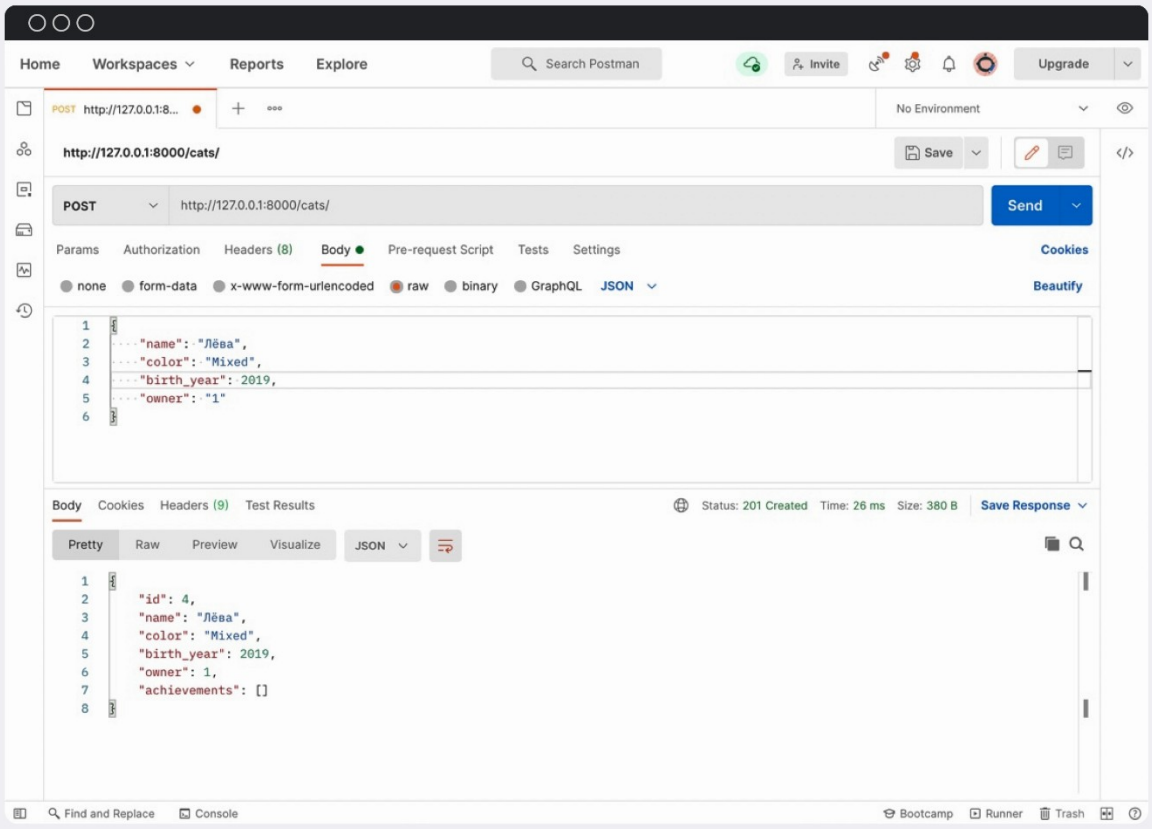
```
...

    def create(self, validated_data):
        # Если в исходном запросе не было поля achievements
        if 'achievements' not in self.initial_data:
            # То создаём запись о котике без его достижений
            cat = Cat.objects.create(**validated_data)
            return cat

        # Иначе делаем следующее:
        # Уберём список достижений из словаря validated_data и сохраним его
```

```
        achievements = validated_data.pop('achievements')
        # Сначала добавляем котика в БД
        cat = Cat.objects.create(**validated_data)
        # А потом добавляем его достижения в БД
        for achievement in achievements:
            current_achievement, status = Achievement.objects.get_or_create(
                **achievement)
            # И связываем каждое достижение с этим котиком
            AchievementCat.objects.create(
                achievement=current_achievement, cat=cat)
        return cat
```

Вот теперь всё работает как надо:



Как вам урок?

Перейти к заданию

Задание

Настройте API для Yatube так, чтобы при запросе постов возвращалась информация о группе, в которой опубликован пост. Данные о группе должны возвращаться в виде значения её поля `slug`.

Добавьте возможность при создании или изменении поста через API опционально указывать группу, передавая в теле запроса поле `slug`.

```
from rest_framework import serializers

from .models import Post, Group


class PostSerializer(serializers.ModelSerializer):
    group = ...

    class Meta:
        fields = ('id', 'text', 'author', 'image', 'pub_date', ...)
        model = Post
```

Сериализаторы: дополнительные настройки

В проекте Kittygram информация о котике содержит и год его рождения. Человеку, знакомому с календарём и арифметикой, не составит труда вычислить возраст котика. Но пользователи кошачьего API попросили разработчиков доработать сервис и явно возвращать возраст их любимцев в ответе на запрос.

Поле SerializerMethodField: обработка данных в сериализаторе

В уроке про проектирование API обсуждалось создание ответа, который содержит в числе прочего данные, которых нет в моделях. Задача с возрастом котика — тот самый случай. Давайте разберёмся, как это можно сделать, и поможет нам в этом SerializerMethodField.

SerializerMethodField — это поле для чтения, связанное с определённым методом, в котором будет вычислено значение этого поля. Метод, который будет вызван для поля `<имя_поля>`, по умолчанию должен называться `get_<имя_поля>`.

Аргументами метода `get_<имя_поля>` должны быть `self` и сериализуемый объект (он передаётся во второй аргумент):

Скопировать код
PYTHON

```
def get_something(self, obj):
    ...
```

С помощью поля SerializerMethodField можно модифицировать существующее поле или создать новое.

Добавьте в сериализатор `CatSerializer` поле `age` (его нет в модели **Cat**); содержимое этого поля будет вычисляться «на лету» в методе `get_age`:

Скопировать код
PYTHON

```
...
```

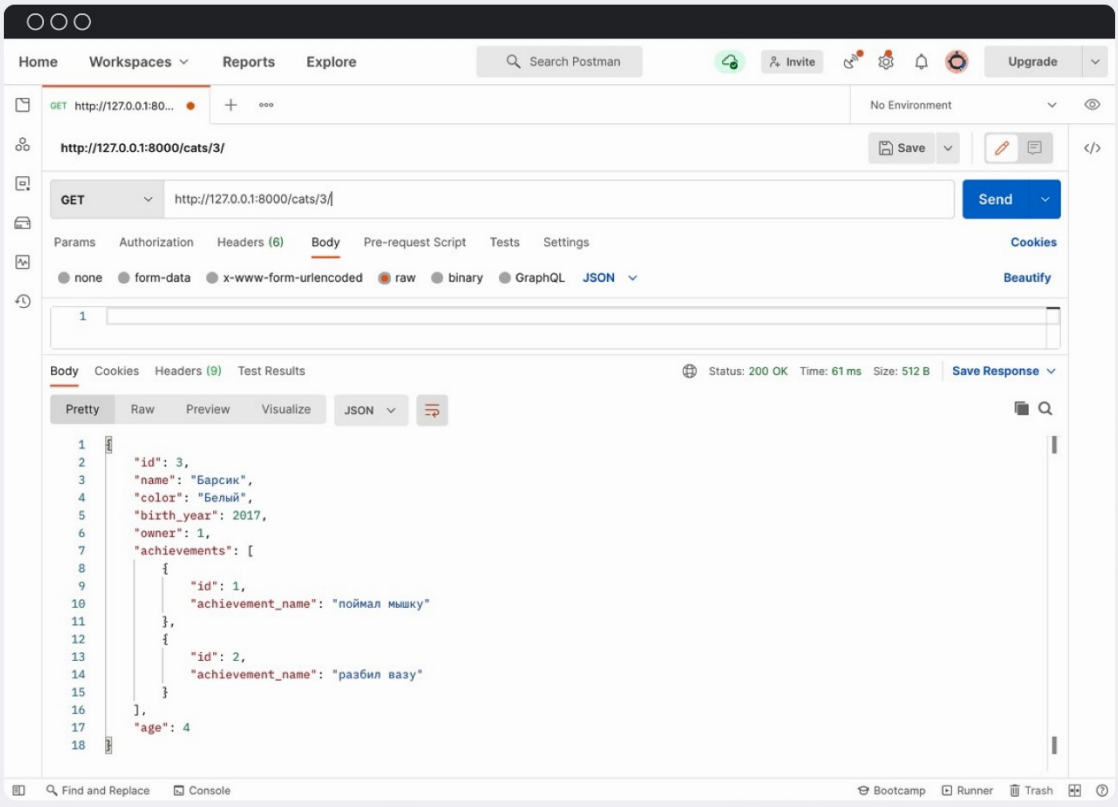
```
import datetime as dt
...

class CatSerializer(serializers.ModelSerializer):
    achievements = AchievementSerializer(many=True)
    age = serializers.SerializerMethodField()

    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year', 'owner', 'achievements',
                  'age')

    def get_age(self, obj):
        return dt.datetime.now().year - obj.birth_year
```

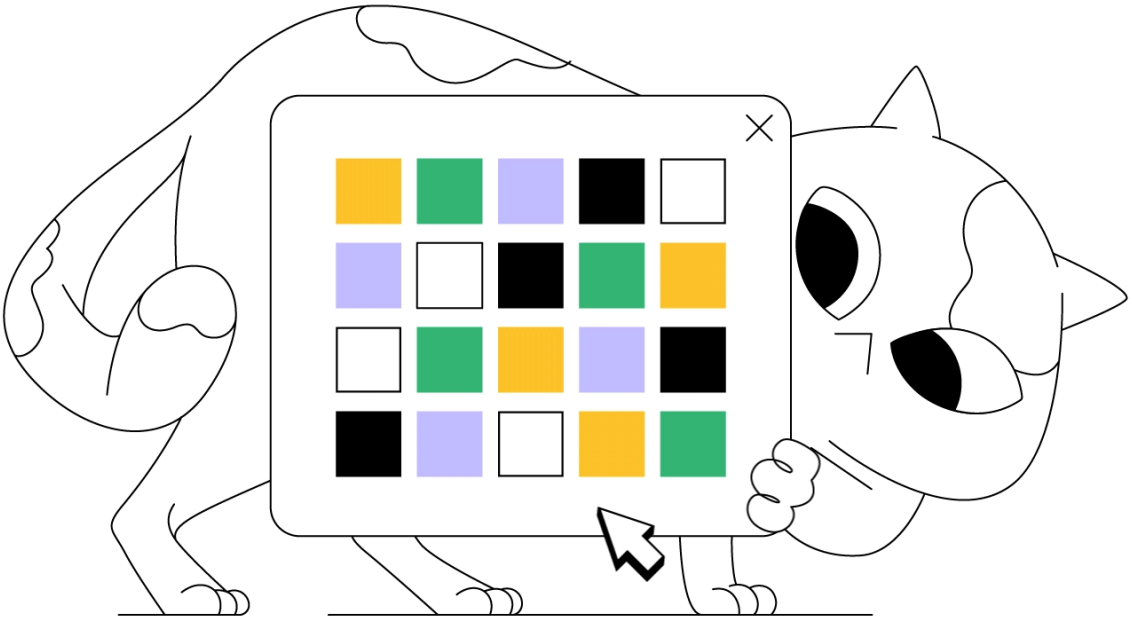
В ответе на GET-запросы котика или котиков появится новое поле:



Хорошей практикой считается не перегружать этот метод какими-либо тяжёлыми операциями.

Пользовательский тип поля для сериализатора

Разработчики фронтенда сообщили, что по просьбе пользователей сайта они разрабатывают виджет-палитру для выбора цвета котиков.



Виджет будет передавать цвет на бэкенд через API в закодированном виде, в hex-формате, например — `#ff0000`.

Коллеги попросили проверить, можно ли перенастроить бэкенд так, чтобы он мог работать с цветом в hex-формате. Отличная задача, приступим.

Сейчас в БД для каждого котика хранится название его цвета; название цвета — это обычная строка. Сериализатор для модели тоже ожидает цвет в строковом формате. Если в POST-запросе в поле `color` вместо строки с названием цвета `"красный"` придёт строка с кодом цвета `"#ff0000"`, то код будет записан в БД. А нам это не надо, это не консистентно: изначально было принято решение хранить цвета в человекочитаемом виде; не будем нарушать это правило.

Цветовая модель RGB, которую применяют фронтендеры, может отобразить более 16 млн цветов, и, разумеется, не у всех цветов есть названия. Попытки поименовать все цвета RGB, конечно, делались — так появились палитры с цветами «В меру оливково-коричневый», «Коричневый цвета кожаного седла для лошади» и, разумеется, «Цвет бедра испуганной нимфы».

Но разработчики фронтенда клятвенно обещают, что их новый виджет будет возвращать только коды именованных цветов — другие они использовать не будут.

На сегодняшний день в спецификации цветов HTML и CSS определено и поименовано более сотни цветов.

Таким образом, задача сводится к тому, что необходимо создать такое поле для сериализатора, которое

- в режиме записи конвертирует код цвета в его название,
- в режиме чтения вернёт название цвета из БД.

Существует простая и удобная библиотека webcolors, которая позволяет конвертировать код цвета в его название и наоборот. Она нам пригодится: установите её в виртуальное окружение согласно инструкции.

Теперь разберёмся с полем сериализатора. Кроме стандартных, вложенных или related типов полей в сериализаторах можно описывать и применять собственные типы; это как раз то, что нужно.

Для создания собственного типа поля сериализатора нужно описать класс для нового типа, который будет унаследован от `serializers.Field` и описать в нём два метода: `def to_representation(self, value)` (для чтения) и `def to_internal_value(self, data)` (для записи).

Опишем новый тип поля `Hex2NameColor` в *serializers.py*:

Скопировать код
PYTHON

```python
import webcolors


...


class Hex2NameColor(serializers.Field):
    # При чтении данных ничего не меняем - просто возвращаем как есть
    def to_representation(self, value):
        return value
    # При записи код цвета конвертируется в его название
    def to_internal_value(self, data):
        # Доверяй, но проверяй
        try:
```

```
            # Если имя цвета существует, то конвертируем код в название
            data = webcolors.hex_to_name(data)
        except ValueError:
            # Иначе возвращаем ошибку
            raise serializers.ValidationError('Для этого цвета нет имени')
        # Возвращаем данные в новом формате
        return data


...
```

Теперь новый тип поля `Hex2NameColor` можно присвоить полю `color` в `CatSerializer`:

Скопировать код
PYTHON

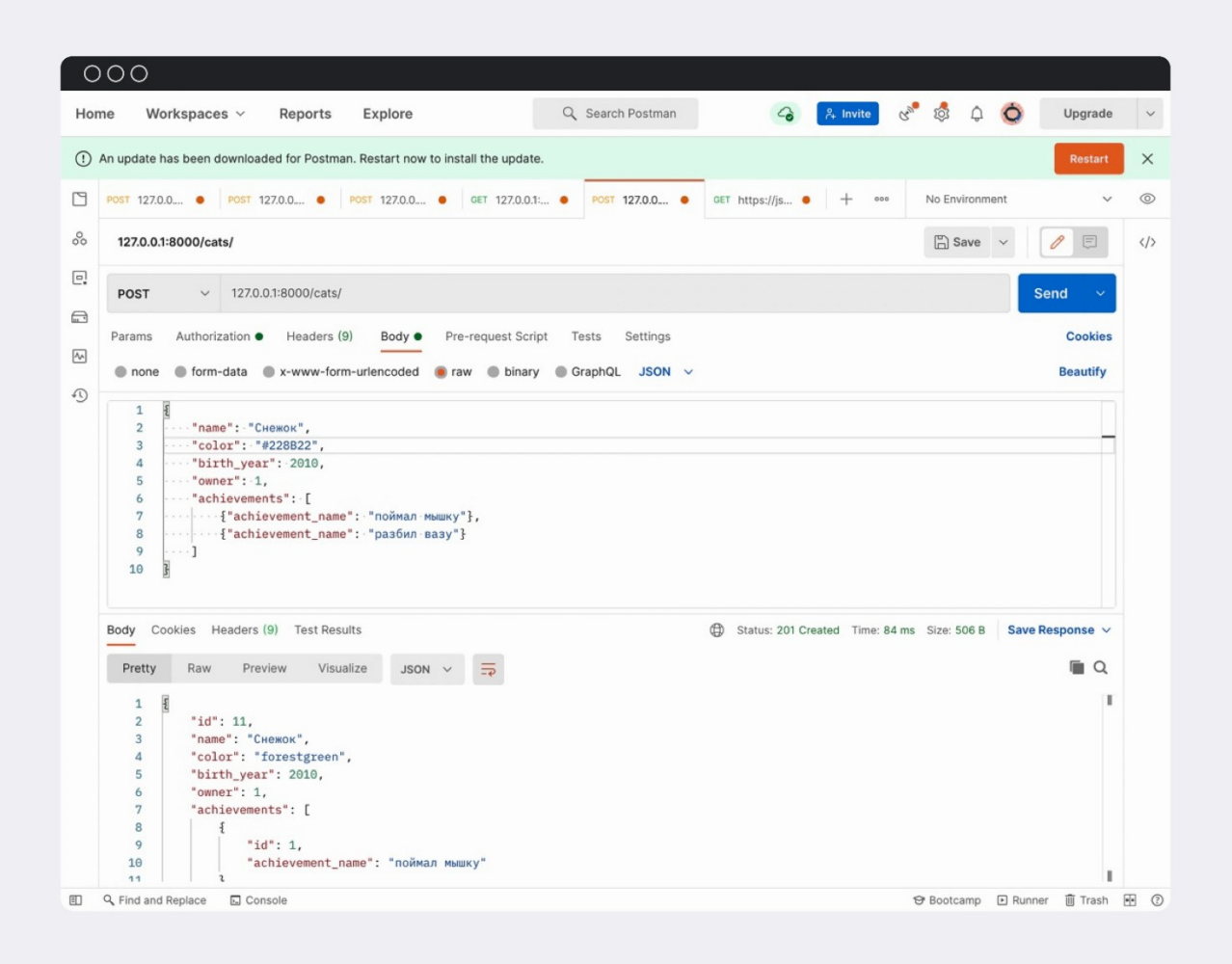
```
...

class CatSerializer(serializers.ModelSerializer):
    achievements = AchievementSerializer(many=True, required=False)
    age = serializers.SerializerMethodField()
    color = Hex2NameColor()  # Вот он - наш собственный тип поля

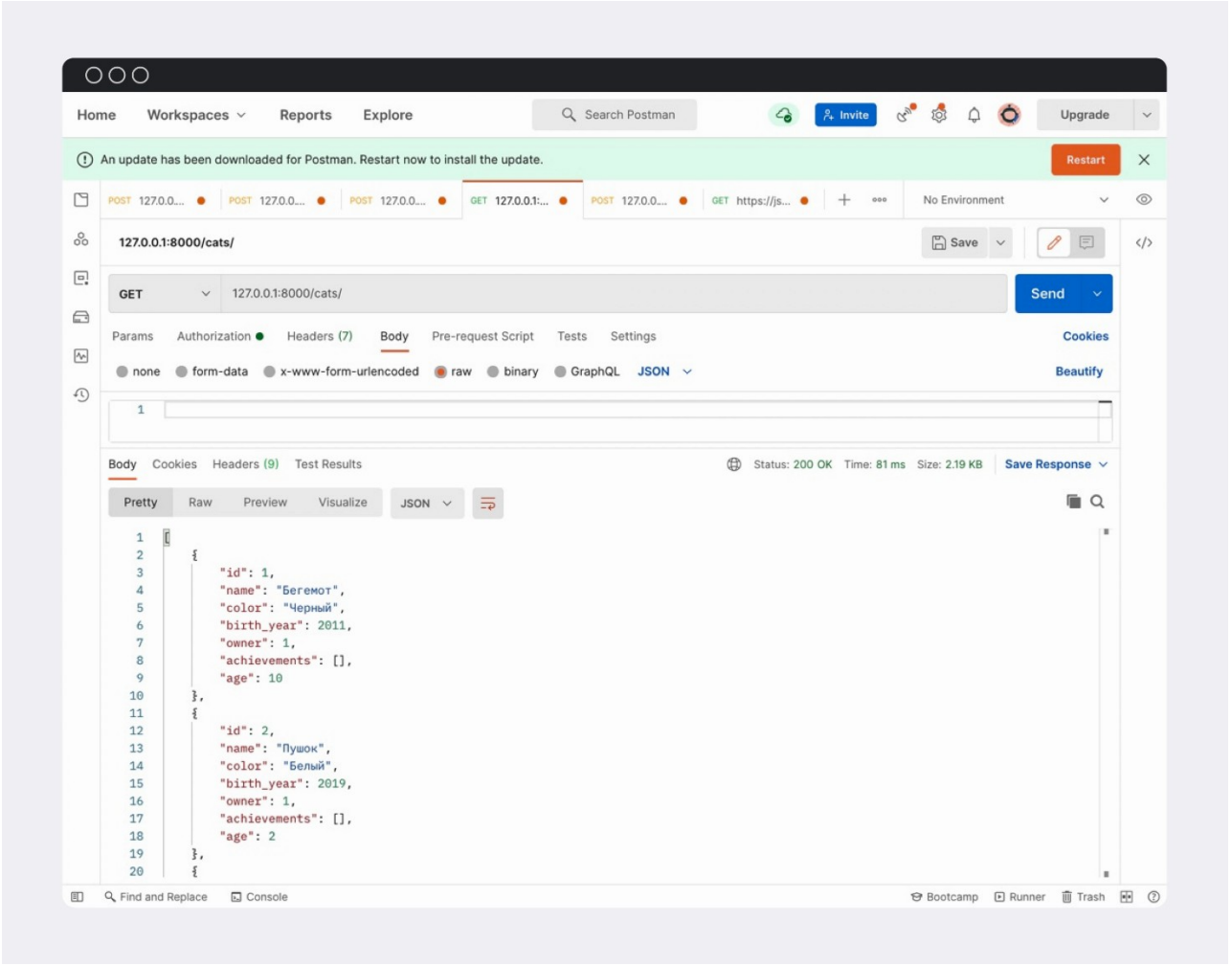
    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year', 'owner', 'achievements',
                  'age')


...
```

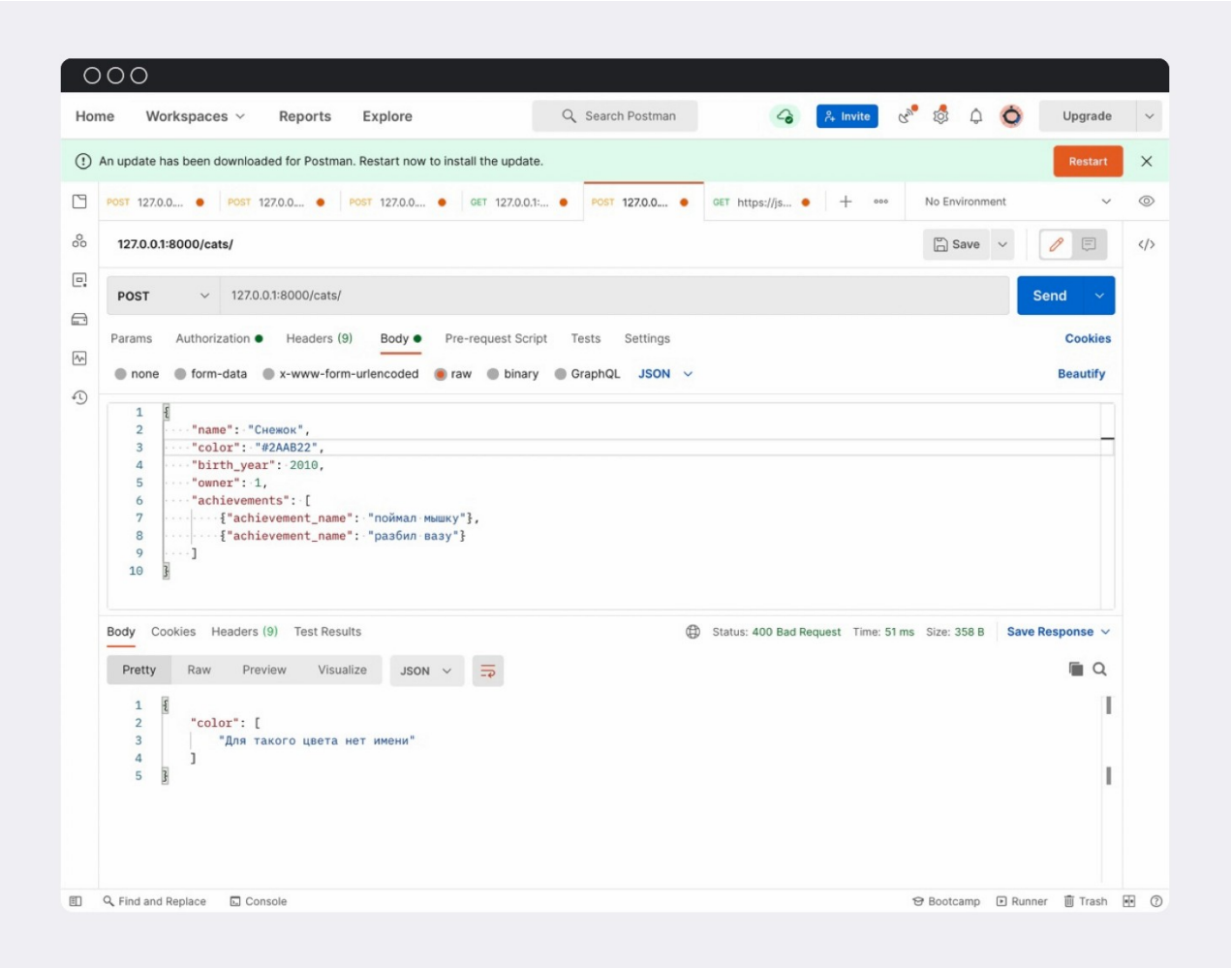
Отправим POST-запрос на добавление нового котика, укажем в качестве цвета, например, код `#228B22` и проверим результат:



Сработало! Строка с кодом конвертировалась, и теперь в БД цвет котика указан как `forestgreen`. При этом существующие записи при GET-запросе возвращаются без изменений.



А вот если в POST-запросе передать код цвета, для которого не существует названия — сериализатор вернёт ошибку для этого поля:



Тестирование прошло удачно.

Над Москвой встаёт `#287233` восход,

По мосту идёт `#ffa500` кот,

И лоточник у метро продаёт

Апельсины цвета `#f5f5dc`

Когда разработчики фронтенда закончат работу над виджетом — можно будет без проблем настроить API для работы с новым форматом цвета. А пока вернём всё как было.

Переименование полей: параметр source

В модели Achievement есть поле name, но и в других моделях есть поля с таким именем. Разобраться можно, но в ответах это имя лучше поменять на более информативное `achievement_name`.

Сериализатор, унаследованный от `ModelSerializer`, по умолчанию использует те же названия полей, что и в модели, с которой он работает. Эти же имена служат ключами в ответе API.

Необходимость изменить имена возникает достаточно часто. Эту задачу решают через переопределение поля и применение параметра **source**:

Скопировать код
PYTHON

```
new_field_name = serializers.CharField(source='old_field_name')
```

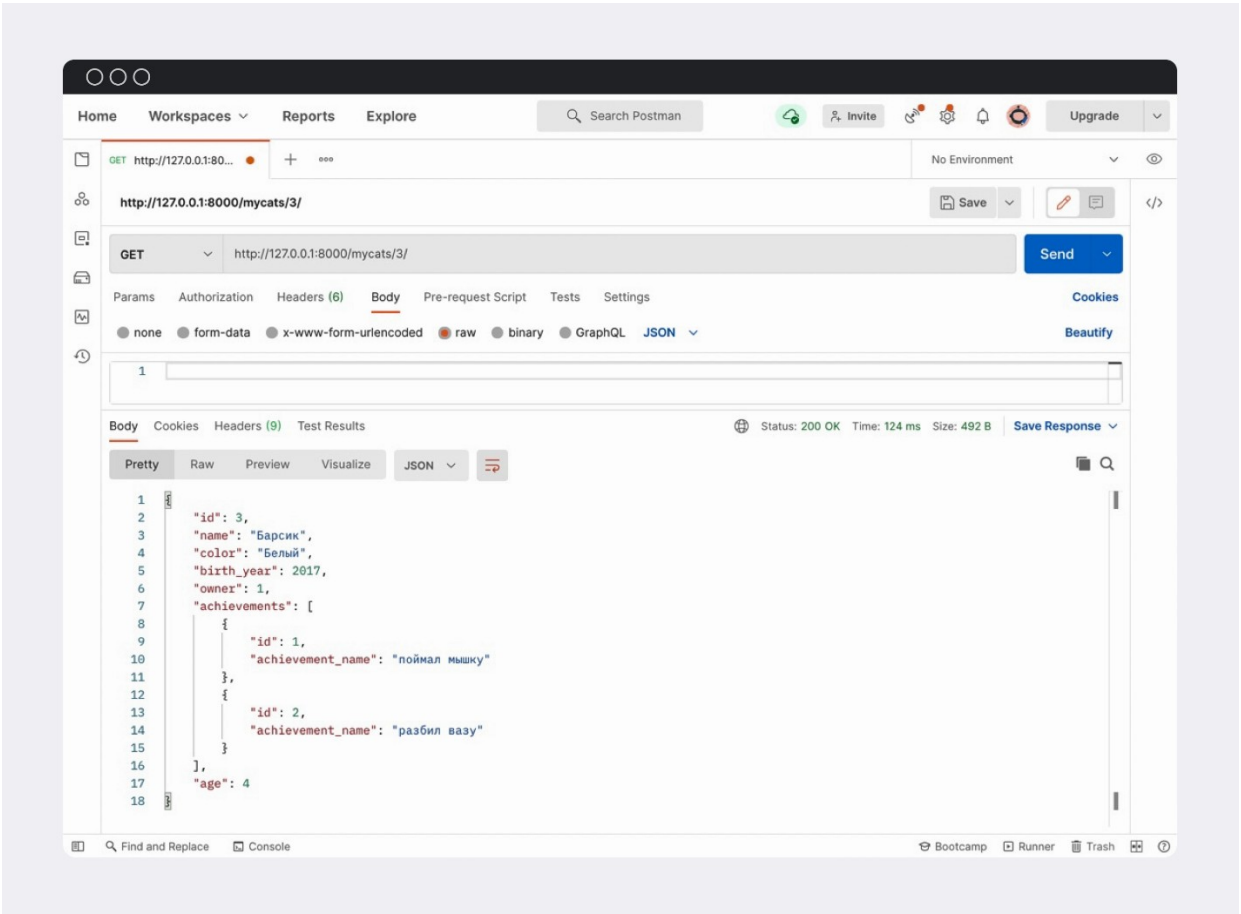
Определите новое поле achievement_name в сериализаторе, в качестве параметра передайте аргумент `source=<'оригинальное имя поля в модели'>`.

Скопировать код
PYTHON

```
class AchievementSerializer(serializers.ModelSerializer):
    achievement_name = serializers.CharField(source='name')


    class Meta:
        model = Achievement
        fields = ('id', 'achievement_name')
```

Теперь сериализатор знает, с каким полем из модели Achievement он должен работать, и соотнесёт всё правильно.



Этот подход работает при использовании сериализатора как для чтения, так и для записи.

Ограничение возможных значений поля: выбор из списка

Изменим возможности для выбора цвета котиков: набор кошачьих расцветок не особо велик, и можно заранее описать все доступные цвета. А потом можно будет организовать категоризацию по цветам.

Это можно сделать на уровне моделей:

Скопировать код
PYTHON

```python
from django.db import models



CHOICES = (
        ('Gray', 'Серый'),
        ('Black', 'Чёрный'),
        ('White', 'Белый'),
```

```
        ('Ginger', 'Рыжий'),
        ('Mixed', 'Смешанный'),
    )

...

class Cat(models.Model):
    name = models.CharField(max_length=16)
    color = models.CharField(max_length=16, choices=CHOICES)
    birth_year = models.IntegerField()
    owner = models.ForeignKey(
        Owner, related_name='cats', on_delete=models.CASCADE, blank=True,
        null=True)
    achievements = models.ManyToManyField(Achievement, through='AchievementCat')

    def __str__(self):
        return self.name

...
```

То же можно сделать и на уровне сериализатора, указав для поля `color` тип `ChoiceField` и передав в параметр `choices` список с возможными вариантами:

Скопировать код
PYTHON

```
...

from .models import Cat, Owner, Achievement, AchievementCat, CHOICES

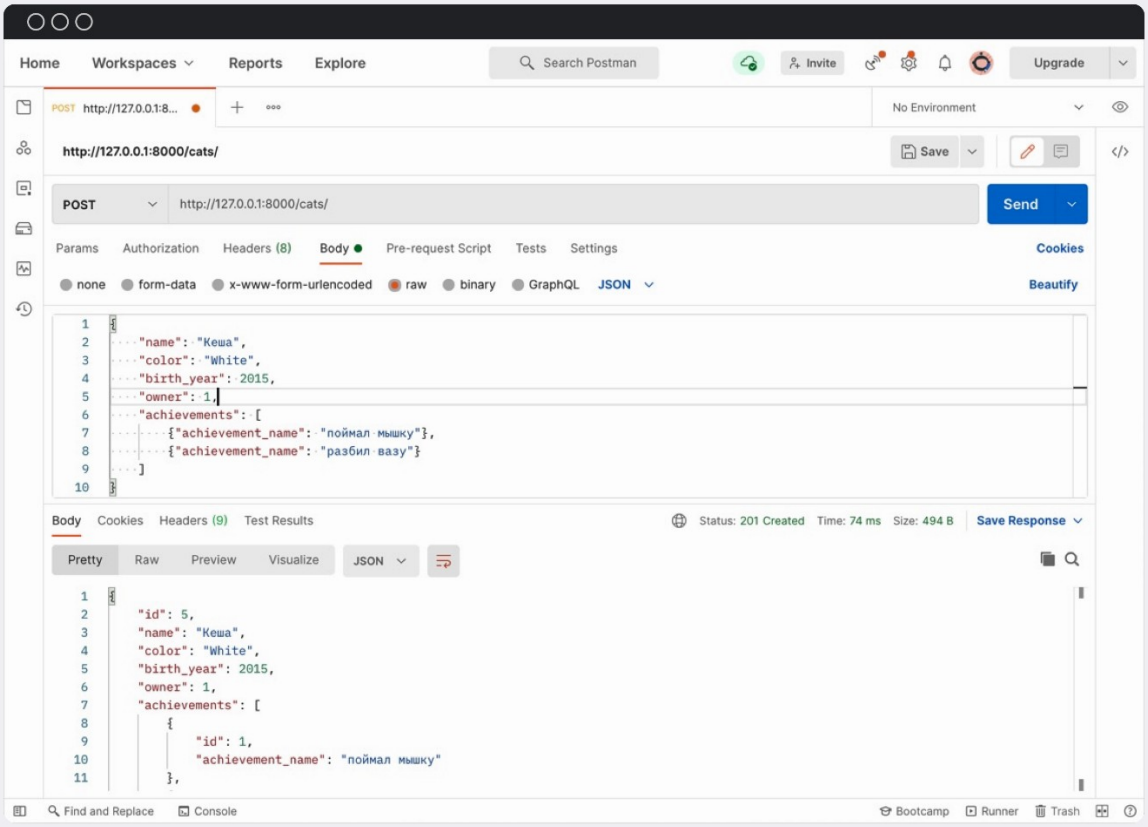
...

class CatSerializer(serializers.ModelSerializer):
    achievements = AchievementSerializer(many=True)
    age = serializers.SerializerMethodField()
    # Теперь поле примет только значение, упомянутое в списке CHOICES
    color = serializers.ChoiceField(choices=CHOICES)

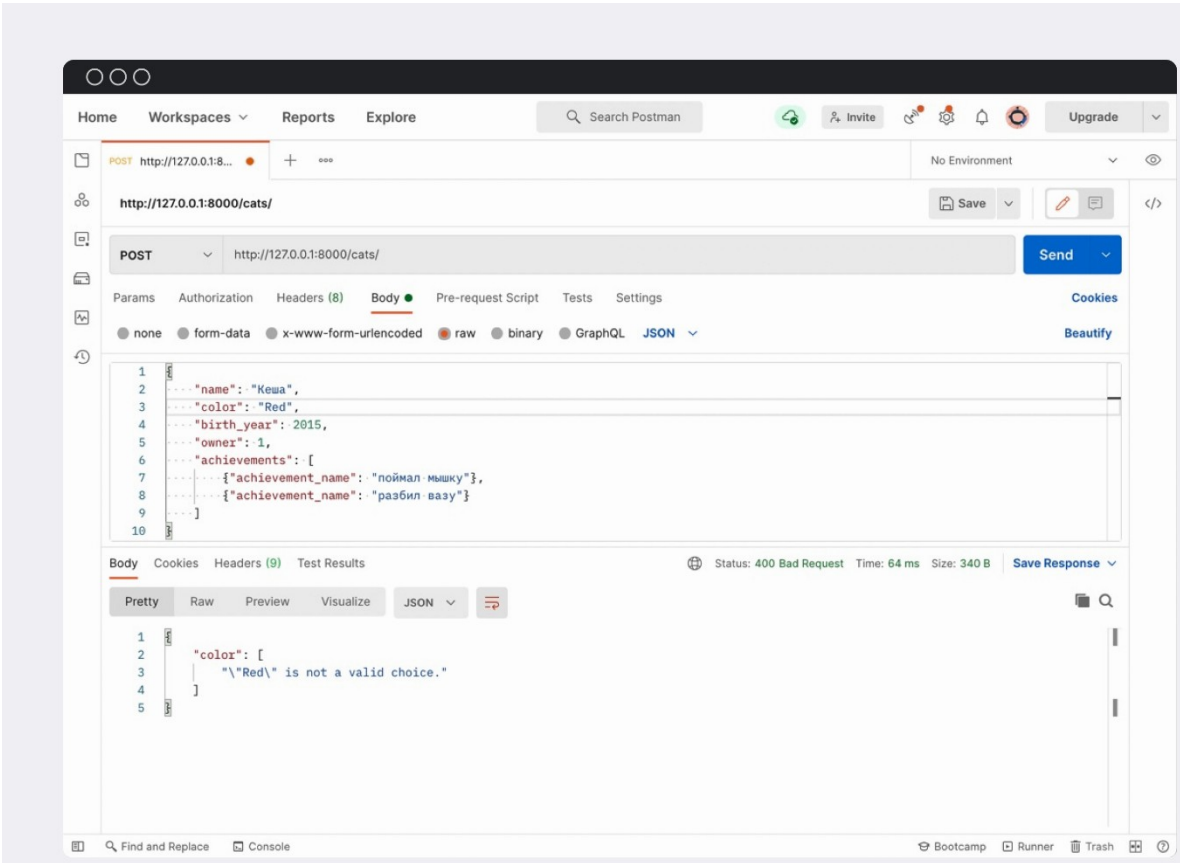
    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year', 'owner', 'achievements',
```

```
                'age')
...
```

Изменили модели — не забыли создать и применить миграции. После этого можно проверять.



Теперь работа с цветом стандартизована; запрос, содержащий в поле `color` название любого неучтённого в списке цвета, вызовет ошибку 400 Bad Request:



А вот и [шпаргалка по сериализаторам и валидаторам для связанных моделей](). Сохраняйте её в закладки, лишней точно не будет.

Как вам урок?

Перейти к заданию

Задание

Владельцы интернет-ресурсов покупают готовые статьи на биржах; стоимость зависит от количества символов в тексте. Настройте Yatube API так, чтобы в ответ на GET-запрос поста (или списка постов) вместе с информацией о посте возвращалось дополнительное поле `character_quantity`. В этом поле верните количество символов в запрошенных публикациях.

```
from rest_framework import serializers

from .models import Post


class PostSerializer(serializers.ModelSerializer):
    character_quantity = ...

    class Meta:
        fields = ('id', 'text', 'author', 'image', 'pub_date', ...)
        model = Post
```

```
# опишите метод для вычисления количества символов в посте
...
```

Регулярные выражения

Регулярные выражения (англ. *regular expressions* или *regExp*) — это самостоятельный язык, предназначенный для поиска строк и проверки их на соответствие какому-то шаблону.

Для поиска используется строка-образец (англ. *pattern*, по-русски её часто называют «шаблоном» или «маской»), определяющая правило поиска.

Символы в регулярном выражении могут:

- служить спецсимволами шаблона (например, точка в шаблоне regExp означает «любой символ в искомой строке»),
- указывать на символы, которые должны присутствовать в искомой строке (в email перед названием доменной зоны обязательно должна стоять точка, это один из признаков email).

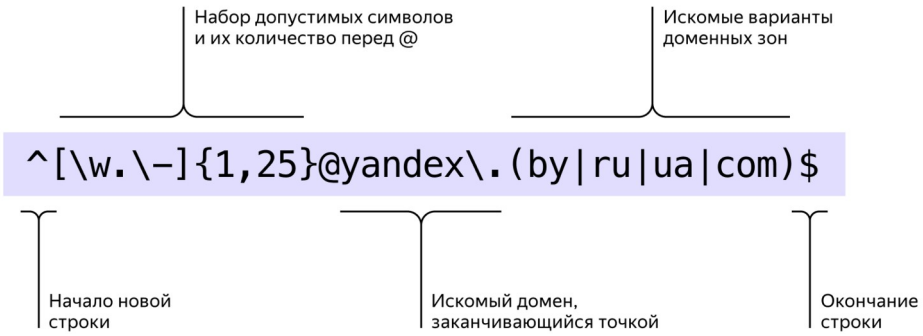
Чтобы отличить «точку-спецсимвол regExp» от «точки — части искомой строки», во втором случае символ «экранируют» — ставят перед ним обратный слеш .

Вот недавно был случай: директор компании, разрабатывающей Kittygram, попросил секретаря найти email клиента. Имя клиента директор не помнит, а про email всего-то и известно, что тот был не длиннее 25 символов, состоял из латиницы и цифр и совершенно точно — на домене yandex, но на каком точно... «Нет, не помню, найдите все похожие!»

Секретарь выгружает список клиентов (а там пятнадцать тысяч строк) и идёт с этим списком и с печенькой к разработчикам.

Разработчики с благодарностью берут печеньку, этот страшный список и пишут регулярное выражение — шаблон, которому должна соответствовать искомая строка.

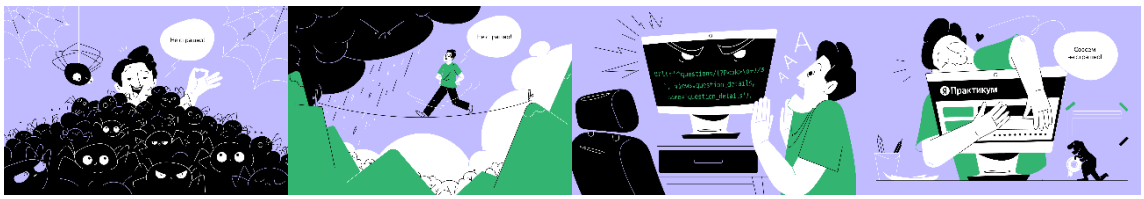
Шаблон для поиска адреса электронной почты в доменах yandex.ru, yandex.ua, yandex.by, yandex.com получился таким:



- `^` означает начало новой строки.
- `[\w.\-]` — квадратные скобки объединяют набор возможных символов, которые могут находиться в начале строки. Среди этих символов могут быть:
  - `\w` — любые словообразующие символы: буквы от a до z и от A до Z, цифры от 0 до 9, знак подчёркивания;
  - `.` — символ «точка» (в квадратных скобках её не нужно экранировать);
  - `\-` — символ «дефис» (экранирован, значит, это символ, который может встретиться в строке).

  Эти символы используются чаще всего в первой части адреса электронной почты.
- `{1,25}` указывает на то, что строка, состоящая из символов, перечисленных в квадратных скобках перед фигурными скобками, может быть длиной от 1 до 25 знаков.
- Элемент `(…)` объединяет варианты доменов, которые могут быть в строке, а разделяющий их символ `|` соответствует оператору «или».
- `$` означает окончание строки.

Разработчики честно заслужили печеньку: поиск выдал 18 адресов, и среди них директор легко нашёл нужный email. Happy end.



Специальные символы и примеры их использования:

Символ	Описание	Пример	Результат
^	Признак начала строки	^kittygram	Найдёт все строки, которые начинаются на kittygram, например: kittygram — лучший сервис для котоводов
$	Признак конца строки	kittygram$	Найдёт все строки, которые заканчиваются на kittygram, например: лучший сервис для котоводов — это kittygram
.	Точка означает любой символ, кроме перевода строки	Стёпа ..дит	Найдёт строки Стёпа кодит, Стёпа водит, Стёпа ходит и прочие подобные
|	Означает ИЛИ	Мурка|Снежок	Найдёт как Мурка, так и Снежок
?	Символ слева от ? отсутствует или представлен в одном экземпляре	kittygra?m	Найдёт kittygram и kittygrm
*	Символ слева от * может отсутствовать или повторяться сколько угодно раз	kittygra*m	Найдёт kittygram, kittygrm, kittygraam и подобные строки
+	Символ слева от + должен присутствовать хотя бы в одном экземпляре или повторяться сколько угодно раз	kittygra+m	Найдёт kittygram, kittygraam, kittygraaam и подобные строки
\	Символ экранирования или начала метасимвола	dev\/project\/ kittygram	Найдёт текст dev/project/kittygram

Помимо специальных символов есть метасимволы, которые описывают группы символов в строке, например:

Мета-символ	Описание	Пример	Результат
\w	Буква, цифра или нижнее подчёркивание	\w+	Соответствует целому слову без пробелов, например Мурка_2010
\W	НЕ буква, цифра или нижнее подчёркивание	\W\w+\W	Найдёт полное слово, которое обрамлено любыми символами, например ?Мурка!
\d	Любая цифра	\d+	Соответствует строке из цифр, например 128, 3 или 100001. С помощью \d можно, например, описать числа, состоящие из шести цифр: ^\d{6}$
\D	Любой символ НЕ цифра	^\D+$	Соответствует любому выражению, где нет цифр, например Мурка

Полный перечень специальных символов и метасимволов собран в документации Python, а потренироваться в написании регулярных выражений можно в онлайн-тренажёре, например вот тут: https://regex101.com/.

Самое главное из урока и даже немного больше есть в шпаргалке. Сохраните её в закладки. Синтаксис регулярных выражений стандартизирован и применяется не только в Python, но и в большинстве других языков программирования. В них стоит разобраться: они пригодятся в работе не раз и не два.

Как вам урок?

К следующему уроку

Вьюсеты. Расширенные возможности

Бытует мнение, что у котиков с доминирующим белым окрасом голубой цвет глаз более глубокий, чем у остальных, и это считается достоинством. Такие котики вызывают большой интерес у наших пользователей.

В связи с повышенным спросом на белых котиков требуется расширить возможности API: по специальному запросу нужно отдавать информацию о пяти котиках белого цвета. Для таких запросов уже придумали эндпоинт: `cats/recent-white-cats/`.

Задача не выглядит типичной: можно получить данные о всех котиках или о каком-то одном, но не о пяти, да ещё и определённого цвета.

Нестандартные действия во вьюсетах

Ещё раз переберём список стандартных действий (англ. *actions*) во вьюсетах:

- *create*: создание экземпляра;
- *retrieve*: получение экземпляра;
- *list*: получение списка экземпляров;
- *update*: обновление экземпляра (все поля);
- *partial_update*: обновление экземпляра (только выбранные поля);
- *destroy*: удаление экземпляра.

Для поставленной задачи ни один из них не подойдёт.

Необходимо нестандартное действие; для этого во вьюсете пишут отдельные методы, которые оборачивают в декоратор `@action` («действие»). Этот декоратор настраивает метод и создаёт эндпоинты для этих действий.

Имя такого метода-«действия» может быть произвольным.

Декоратор `@action` по умолчанию отслеживает только GET-запрос. Но если передать в декоратор параметр `methods`, то можно разрешить и другие методы запросов.

Скопировать код
PYTHON

```
@action(methods=['get', 'delete', ...]
```

В декораторе можно явным образом указать, должен ли метод работать с одним объектом или с коллекцией объектов. Для этого используется параметр `detail`, который может принимать значения `True` (разрешена работа с одним объектом) или `False` (работаем с коллекцией).

URL эндпоинта по умолчанию генерируется из двух частей: `<URL-префикс ресурса>/<название метода>/`.

Скопировать код
PYTHON

```
# К такому методу можно обратиться через эндпоинт cats/cats_for_sale/
@action()
def cats_for_sale()
    ...
```

Если URL эндпоинта не должен совпадать с именем метода, URL можно изменить: для этого нужно передать в декоратор аргумент `url_path` с необходимым значением.

Решим задачу с получением пяти белых котиков. Для этого опишем метод `recent_white_cats` для вьюсета `CatViewSet`.

Скопировать код
PYTHON

```
from rest_framework.decorators import action
from rest_framework.response import Response


...

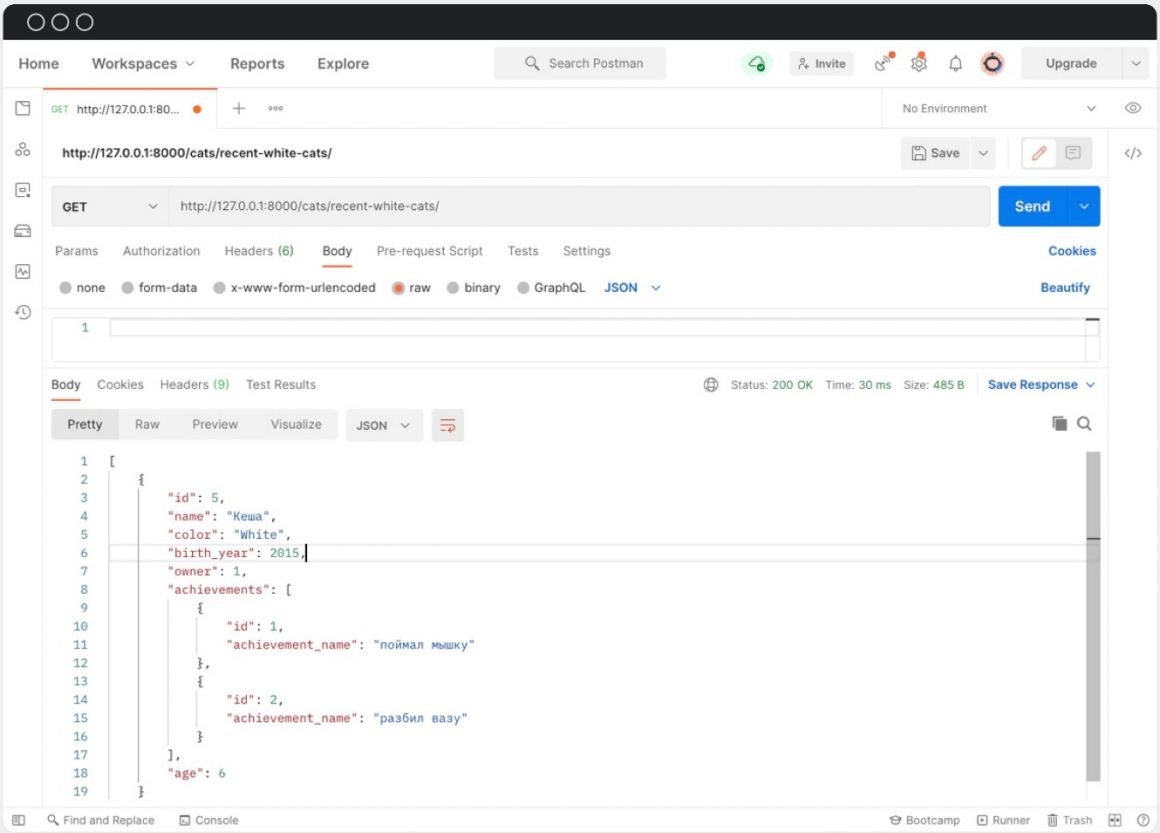

class CatViewSet(viewsets.ModelViewSet):
    queryset = Cat.objects.all()
    serializer_class = CatSerializer

    # Пишем метод, а в декораторе разрешим работу со списком объектов
    # и переопределим URL на более презентабельный
    @action(detail=False, url_path='recent-white-cats')
```

```
    def recent_white_cats(self, request):
        # Нужно получить записи о пяти котиках белого цвета
        cats = Cat.objects.filter(color='White')[:5]
        # Передадим queryset cats сериализатору
        # и разрешим работу со списком объектов
        serializer = self.get_serializer(cats, many=True)
        return Response(serializer.data)
```

Готово, проверяем.

При отправке GET-запроса на эндпоинт `cats/recent-white-cats/` должен быть получен примерно такой результат (конечно, если у вас в базе есть достаточно белых котиков):



Различные сериализаторы для одного вьюсета

В Kittygram для сериализации и десериализации каждой модели использовался лишь один сериализатор. Но если некоторые поля сериализатора доступны только для чтения, то по факту для чтения

применяется один набор полей, а для записи — другой. Это равносильно использованию разных сериализаторов.

Да, так тоже можно: для модели можно описать несколько сериализаторов и использовать их, например, в зависимости от типа запроса.

Допустим, когда

- добавляется новый котик,
- запрашивается детальная информация о конкретном котике,
- обновляется информация о конкретном котике,

будем обрабатывать все доступные поля модели, а вот если запрашивается список котиков, то необходимы только id, имя и цвет.

Опишем для этого ещё один сериализатор, который назовём `CatListSerializer`:

Скопировать код
PYTHON

```python
class CatListSerializer(serializers.ModelSerializer):
    color = serializers.ChoiceField(choices=CHOICES)

    class Meta:
        model = Cat
        fields = ('id', 'name', 'color')
```

Добавьте во вьюсет стандартный метод `get_serializer_class`: в нём можно определить, какой из доступных сериализаторов должен обрабатывать данные в зависимости от действия:

Скопировать код
PYTHON

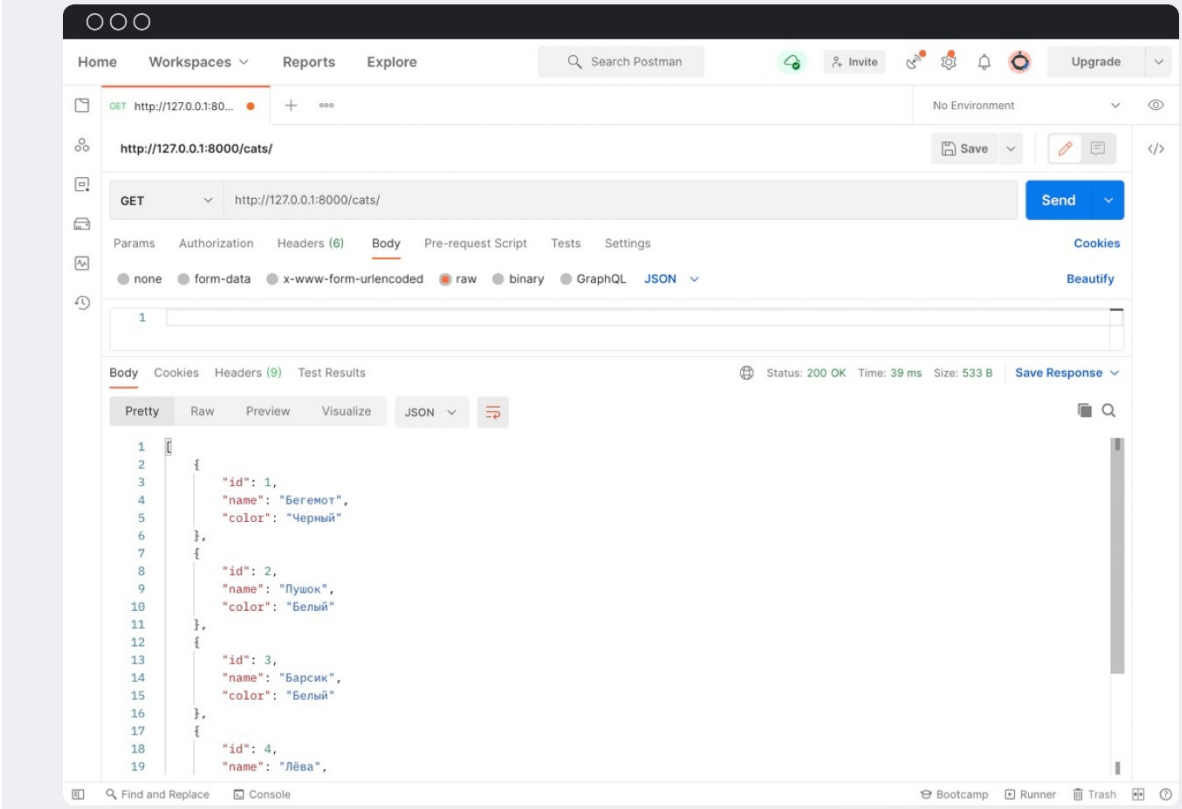
```python
class CatViewSet(viewsets.ModelViewSet):
    queryset = Cat.objects.all()
    serializer_class = CatSerializer

    ...

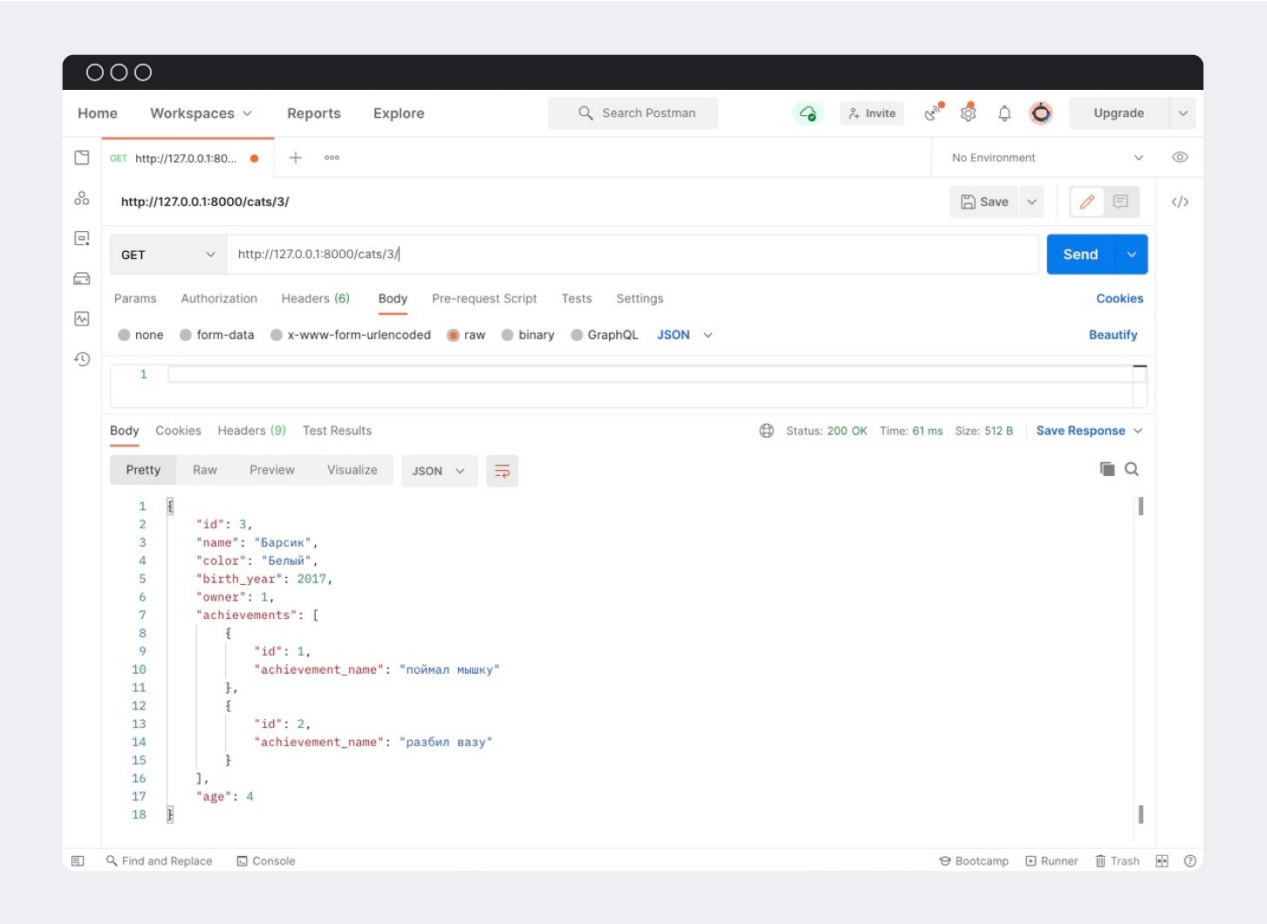
    def get_serializer_class(self):
        # Если запрошенное действие (action) — получение списка объектов ('list')
        if self.action == 'list':
```

```
            # ...то применяем CatListSerializer
            return CatListSerializer
        # А если запрошенное действие — не 'list', применяем CatSerializer
        return CatSerializer
```

Проверим результаты, отправив GET-запрос на получение списка всех котиков.



По ответу видно, что отработал новый сериализатор: всё по плану. А при запросе к конкретному котику всё работает как и раньше:



Нестандартный набор стандартных действий

Чаще всего мы наследовали вьюсеты от `ModelViewSet`. По умолчанию это даёт возможность обрабатывать шесть типичных действий:

- создание нового объекта;
- получение информации об одном объекте;
- удаление объекта;
- полное замещение существующего объекта;
- изменение одного или нескольких полей объекта;
- получение списка объектов.

Если требуется только получать данные из БД, можно унаследоваться от `ReadOnlyModelViewSet`. В этом случае доступный набор действий будет таким:

- получение информации об одном объекте;
- получение списка объектов.

А если требуется иной набор действий — например, нужно только создавать новый объект и получать информацию об одном объекте? Эта задача может быть решена с использованием миксинов (от англ. mix in, «смешивать»).

Миксины и ваш собственный базовый класс для вьюсетов

Никто не знает, что придёт в голову разработчикам и какие вьюсеты им понадобятся, поэтому готовых вьюсетов на все случаи жизни нет и быть не может. Но есть классы-«детали», из которых можно быстро собрать практически любой необходимый базовый вьюсет: **миксины**.

Чтобы самостоятельно создать базовый вьюсет с особым набором действий — нужно унаследовать его от одного или нескольких миксинов с нужными действиями и, дополнительно, от базового класса `GenericViewSet`:

Скопировать код
PYTHON

```
from rest_framework import mixins


...


# Собираем вьюсет, который будет уметь изменять или удалять отдельный объект.
# А ничего больше он уметь не будет.
class UpdateDeleteViewSet(mixins.UpdateModelMixin, mixins.DestroyModelMixin,
                          viewsets.GenericViewSet):
    pass
```

В DRF есть пять предустановленных классов миксинов, они соответствуют пяти операциям с данными:

- CreateModelMixin — создать объект (для обработки запросов POST);
- ListModelMixin — вернуть список объектов (для обработки запросов GET);
- RetrieveModelMixin — вернуть объект (для обработки запросов GET);
- UpdateModelMixin — изменить объект (для обработки запросов PUT и PATCH);

- DestroyModelMixin — удалить объект (для обработки запросов DELETE).

Опишем собственный базовый класс вьюсета: он будет создавать экземпляр объекта и получать экземпляр объекта; назовём его `CreateRetrieveViewSet`.

Скопировать код
PYTHON

```
# cats/views.py
...

from rest_framework import mixins

...

class CreateRetrieveViewSet(mixins.CreateModelMixin, mixins.RetrieveModelMixin,
                            viewsets.GenericViewSet):
    # В теле класса никакой код не нужен! Пустячок, а приятно.
    pass
```

Базовый вьюсет готов, он получился не хуже, чем `ModelViewSet`. Теперь можно унаследоваться от этого базового класса.

Опишем вьюсет `LightCatViewSet`, унаследованный от `CreateRetrieveViewSet`.

Скопировать код
PYTHON

```
class LightCatViewSet(CreateRetrieveViewSet):
    queryset = Cat.objects.all()
    serializer_class = CatSerializer
```

Зарегистрируем этот вьюсет в роутере:

Скопировать код
PYTHON

```
from cats.views import CatViewSet, LightCatViewSet, OwnerViewSet

...

router.register(r'mycats', LightCatViewSet)
```

Нужно разобраться, что за странный синтаксис в первом аргументе метода `register()`, ведь там должен быть просто префикс для URL.

Неожиданное откровение

При регистрации эндпоинтов для вьюсета в качестве URL-префикса используется не обычная строка, а **регулярное выражение**. За счёт этого URL-префикс может обрабатывать множество вариантов URL.

Скопировать код
PYTHON

```
...
# При регистрации эндпоинтов с таким URL-префиксом
router.register(r'profile/(?P<username>[\w.@+-]+)/', AnyViewSet)
# ...вьюсет AnyViewSet будет получать на обработку все запросы с адресов
# /profile/toh@/
# /profile/nik.nik/
# /profile/leo/
# ...и подобных, ограниченных маской регулярного выражения.
```

В уроке о роутерах в первый параметр метода `register()` мы передавали просто строку, но «просто строка» — это частный случай регулярного выражения, и Django эту «просто строку» тоже интерпретировал как regExp.

Префикс `r` перед строкой определяет **raw-строку** (или **r-строку**, так короче). Такую строку система будет читать как простую последовательность символов, игнорируя escape-последовательности (англ. *escape sequence)* — комбинации обратного слеша и символа.

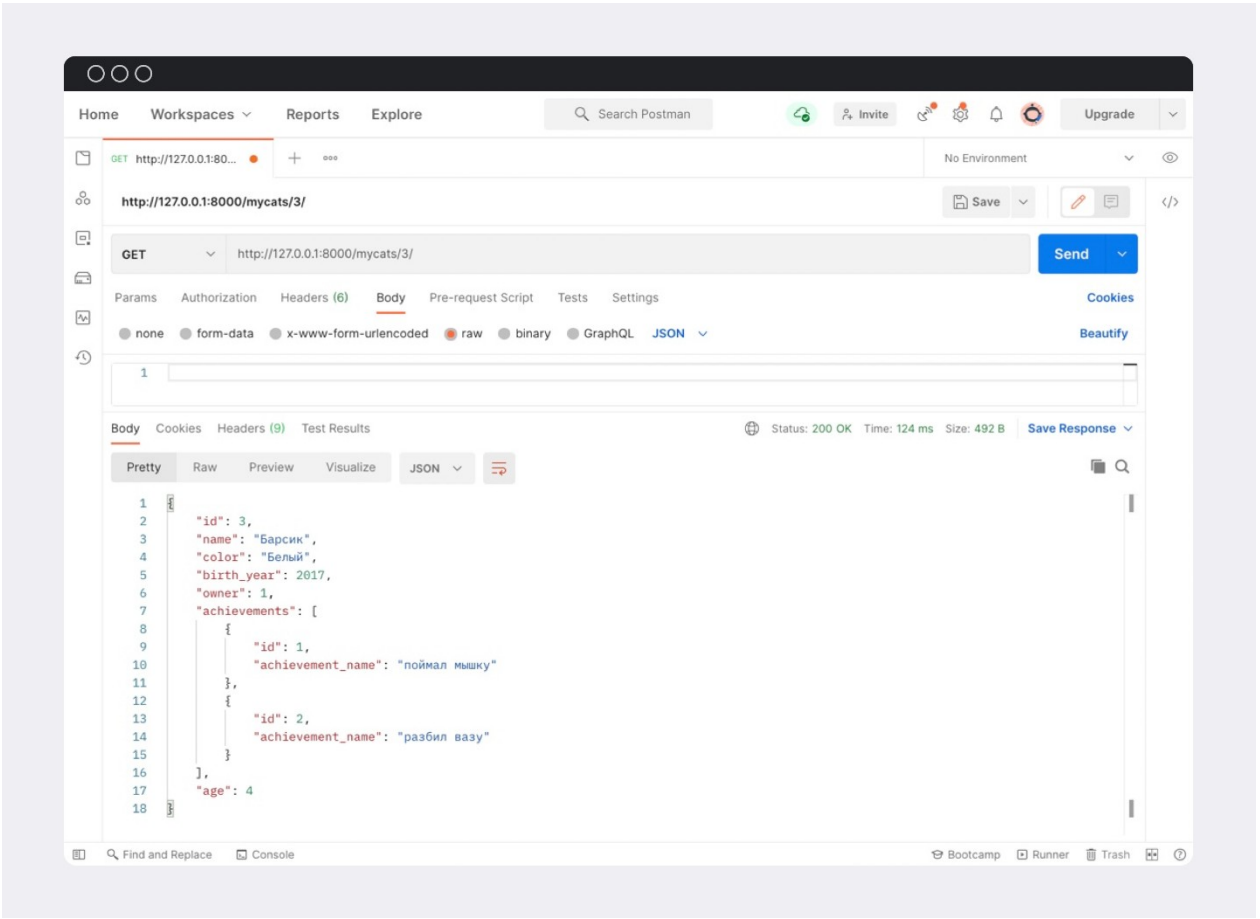
Например, escape-последовательность `\n` интерпретируется как перенос строки, но в raw-строке это будут просто два текстовых символа без всякого скрытого смысла; в результате вся r-строка будет считана как регулярное выражение.

Вернёмся к нашим миксинам

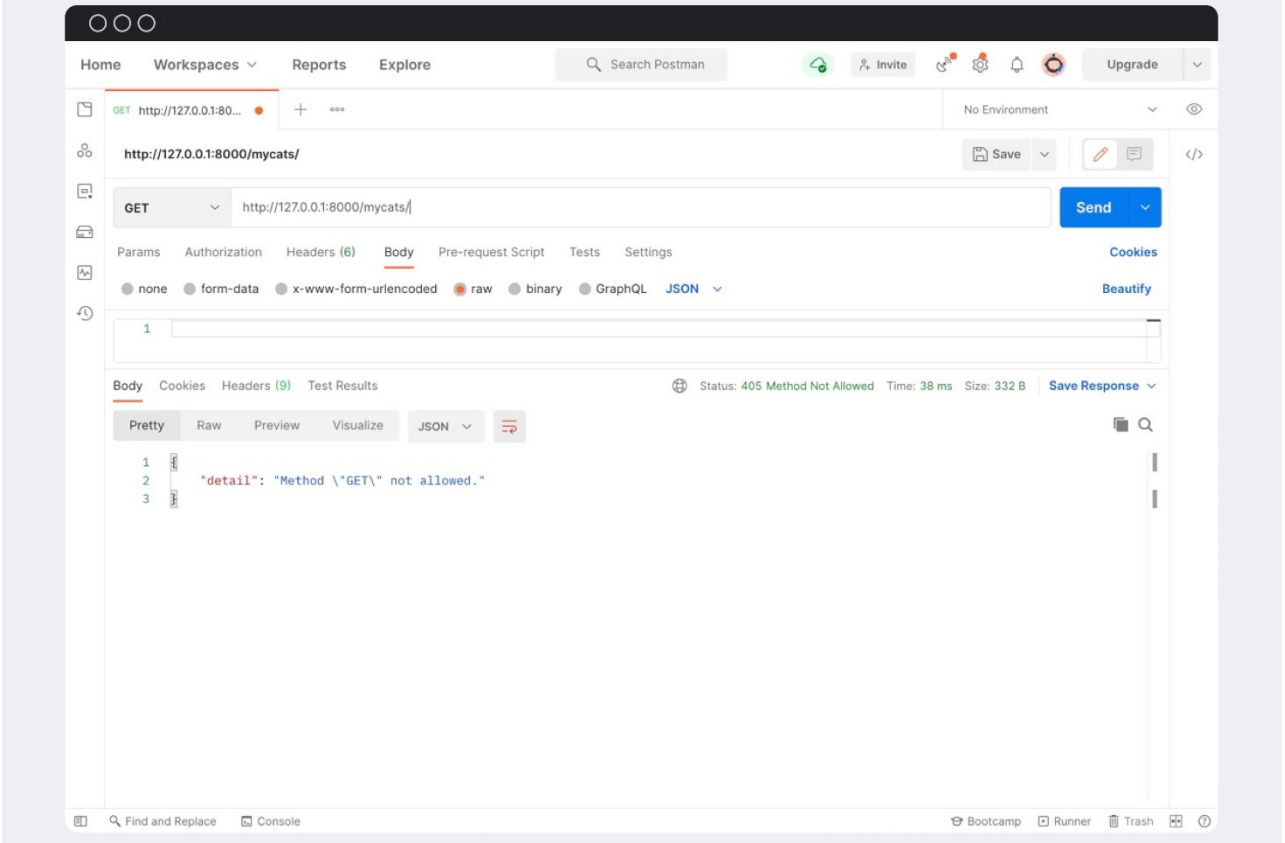
Базовый вьюсет собран, наследник описан и применён в коде.

Проверьте работу нового вьюсета: сделайте запросы на новый эндпоинт `mycats/`.

GET-запрос к конкретному котику отрабатывает на «отлично»:



А вот получить список котиков через этот эндпоинт нельзя:



Работает! Теперь можно монтировать базовые вьюсеты с любым набором действий.

Мой вьюсет — мои правила: наследование от класса ViewSet

В тех случаях, когда нужно получить больше контроля и возможностей что-то «подкрутить» во вьюсетах, можно наследоваться от базового класса `ViewSet`. Именно от него наследуется, например, класс `ModelViewSet`.

Класс `ViewSet`, в свою очередь, наследуется от `APIView`.

При работе с низкоуровневыми вьюсетами все нужные методы вам придётся описать самостоятельно.

В классе `ViewSet` есть шесть предопределённых методов:

- **list(self, request)** — для получения списка объектов из *queryset;*
- **create(self, request)** — для создания объекта в модели;
- **retrieve(self, request, pk=None)** — для получения определённого объекта из *queryset*;
- **update(self, request, pk=None)** — для перезаписи (полного обновления) определённого объекта из *queryset*;
- **partial_update(self, request, pk=None)** — для частичного обновления объекта из *queryset*;
- **destroy(self, request, pk=None)** — для удаления одного из объектов *queryset.*

Чтобы применить любой из этих методов, нужно полностью описать его; в классе `ViewSet` эти методы объявлены, но не описаны.

Например, в приложении нужно создать вьюсет, который будет получать сериализованный объект одного котика (методом `retrieve()`) и полный список всех котиков (методом `list()`). Это можно сделать так:

Скопировать код
PYTHON

```python
from rest_framework import viewsets
from rest_framework.response import Response
```

```
from django.shortcuts import get_object_or_404

from .models import Cat
from .serializers import CatSerializer


class CatViewSet(viewsets.ViewSet):
    def list(self, request):
        queryset = Cat.objects.all()
        serializer = CatSerializer(queryset, many=True)
        return Response(serializer.data)

    def retrieve(self, request, pk=None):
        queryset = Cat.objects.all()
        cat = get_object_or_404(queryset, pk=pk)
        serializer = CatSerializer(cat)
        return Response(serializer.data)
```

Похоже на то, как мы работали в APIView, с той разницей, что здесь присутствуют поля *queryset* и *serializer*.

Документацию по предустановленным методам класса `ViewSet` можно [посмотреть на официальном сайте проекта](#).

Как вам урок?

К следующему уроку

Аутентификация по токену. JWT + Djoser

Аутентификация

Аутентификация на веб-сайтах, как правило, устроена так: при входе на сайт вы один раз вводите логин и пароль, система проверяет ваши данные и, в соответствии с вашими правами, предоставляет вам доступ к сайту. При переходе по страницам этого сайта вам не приходится каждый раз заново вводить свои данные: система запоминает вас, «сохраняет ваше состояние».

Но для REST API такой вариант не подходит: отсутствие состояния (*stateless*) — это один из основных принципов REST. Этот принцип означает, что каждый запрос к серверу не должен быть связан с предыдущими запросами, как будто их и не было.

Если провести аналогию с веб-сайтами — REST предполагает, что при каждом переходе на новую страницу вам снова и снова придётся вводить логин и пароль.

Аутентификация на основе токенов

Аутентификация с помощью токенов отлично вписывается в архитектуру REST. Клиент отправляет токен, кодовую последовательность символов, в заголовке каждого запроса к серверу. Если в базе данных существует пользователь, которому выдавался этот токен, то запрос будет обработан. Если токен не соответствует ни одному пользователю — запрос будет отклонён.

При каждом последующем запросе к API клиент также должен передавать этот токен, и каждый раз будет проводиться проверка прав доступа.

Пишем свою аутентификацию

В актуальной версии Kittygram изменять данные может любой желающий. Это не лучший вариант.

Настроим проект так, чтобы отправлять запросы к сервису могли только аутентифицированные пользователи. Прикрутим к проекту механизм аутентификации по токену.

Порядок действий для пользователя будет таким:

1. Пользователь создаёт на проекте свою учётную запись, для входа в систему у него будет логин и пароль.
2. Клиент отправляет запрос на специальный эндпоинт и в запросе передаёт логин и пароль. Если в базе данных существует такой пользователь и пароль совпадает с сохранённым в базе — в ответ клиент получает токен.
3. Теперь клиент может работать с API, но при каждом запросе он должен отправлять токен.

В DRF есть готовый модуль, который предоставляет возможность аутентификации по токенам прямо «из коробки»: **Authtoken**.

Настроим этот модуль на примере Kittygram.

Фреймворк DRF в Kittygram уже подключён и настроен (установить DRF самостоятельно можно командой `pip install djangorestframework` при активированном виртуальном окружении; затем добавить приложение `rest_framework` в `INSTALLED_APPS`).

Чтобы подключить механизм получения токена и аутентификации по токену в Django REST framework, нужно выполнить несколько действий:

1. Подключить модуль authtoken: добавить строку 'rest_framework.authtoken' в список **INSTALLED_APPS**

Скопировать код
PYTHON

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
```

```
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'rest_framework.authtoken',
    'cats.apps.CatsConfig',
]
```

2. В settings.py в настройках REST_FRAMEWORK объявить новый способ аутентификации `TokenAuthentication`.
3. Запретить доступ всем неаутентифицированным пользователям: ограничение доступа настраивается с помощью **пермишенов** (англ. permissions, «разрешения»). Чтобы запретить доступ без токена, нужно добавить значение `IsAuthenticated` для ключа `DEFAULT_PERMISSION_CLASSES`:

Скопировать код
PYTHON
```
REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],

    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
    ]
}
```

4. Выполнить миграции, чтобы в базе данных создались поля для работы и хранения токена: `python manage.py migrate`.
5. Добавить маршрут для получения токена. Подключать можно в любом urls.py, мы подключим в головном:

Скопировать код
PYTHON
```
from rest_framework.authtoken import views



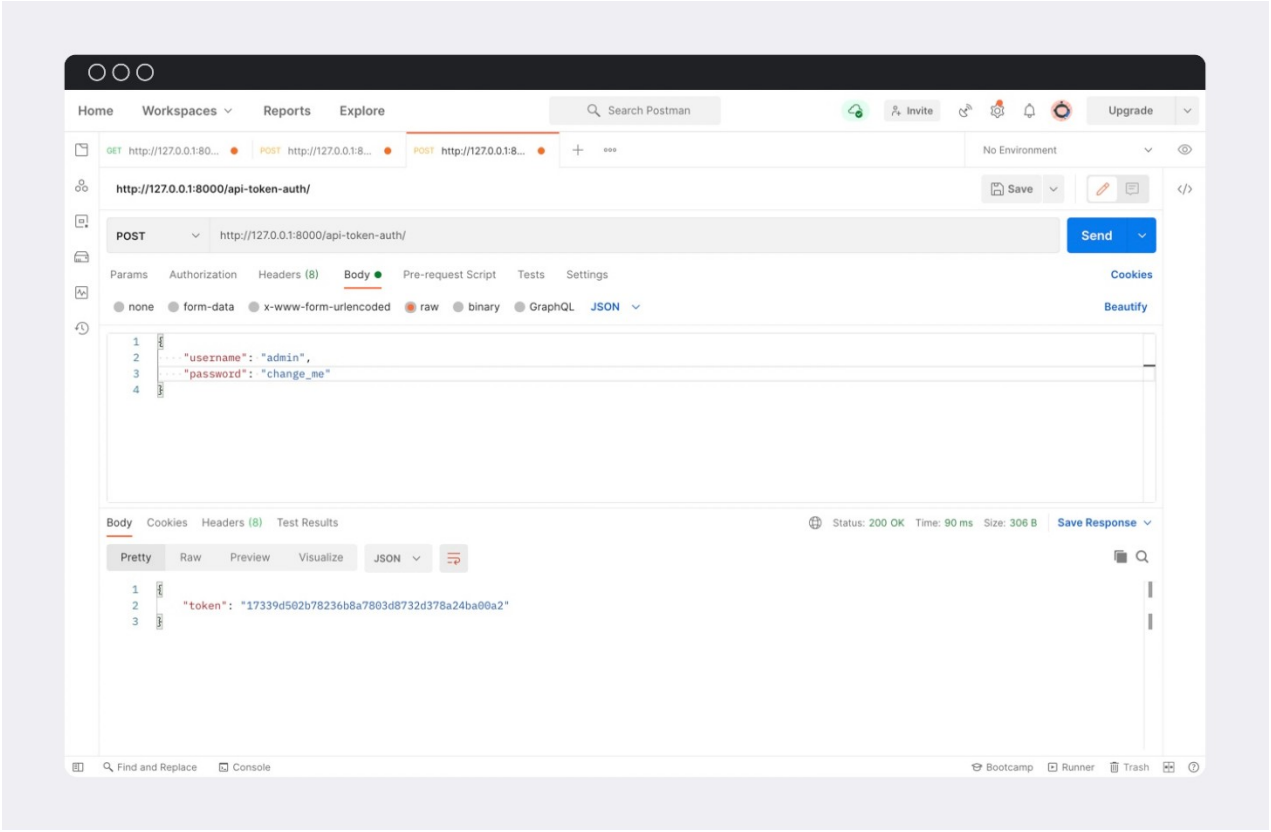
urlpatterns = [

    ...
```

```
    path('api-token-auth/', views.obtain_auth_token),
]
```

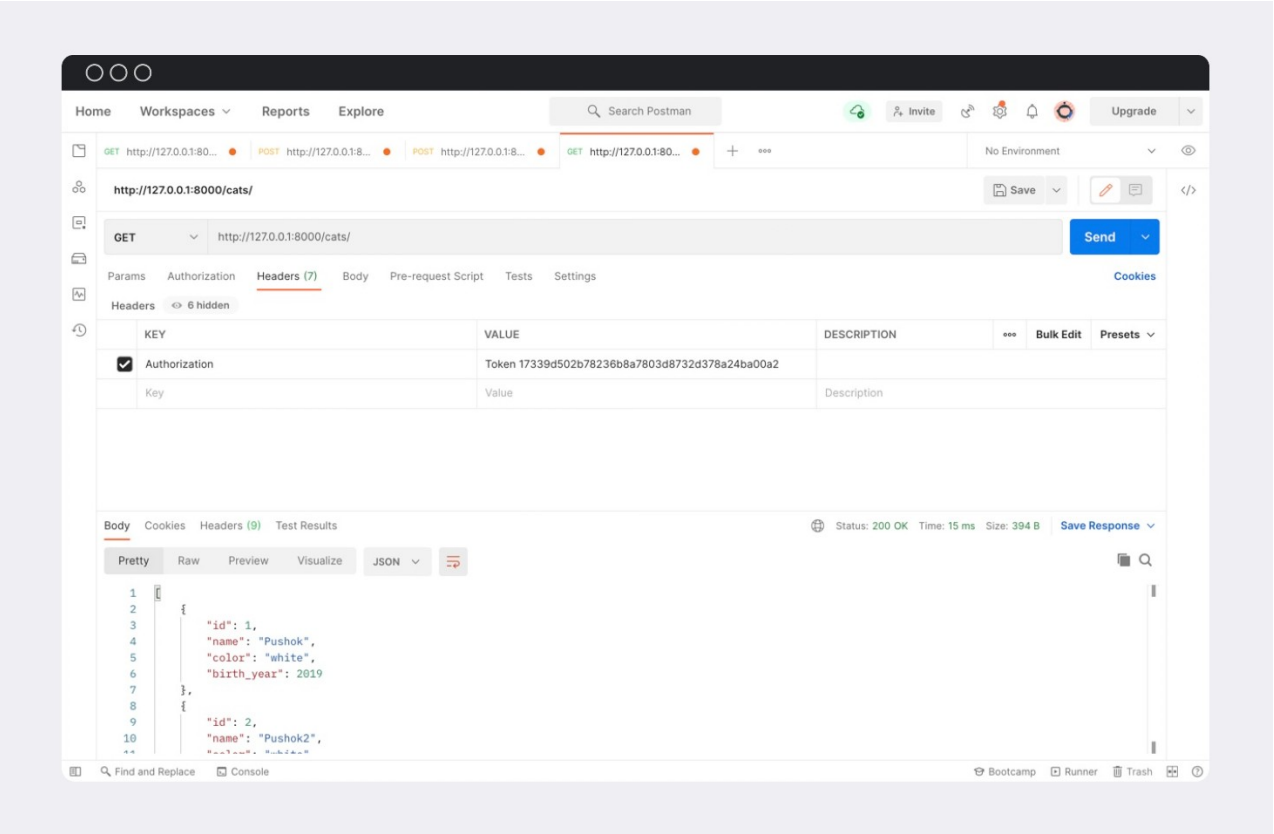
Дополнительную информацию о модуле Authtoken и о его настройках можно найти в документации.

Теперь через Postman можно отправить POST-запрос к адресу `api-token-auth/`, передать в теле запроса `username` и `password` существующего пользователя (например, суперпользователя) — и в ответ придёт заветный токен:



И что с этим делать?

Этот токен надо будет передавать в заголовке каждого запроса, в поле **Authorization**. Перед самим токеном должно стоять ключевое слово `Token` и пробел.

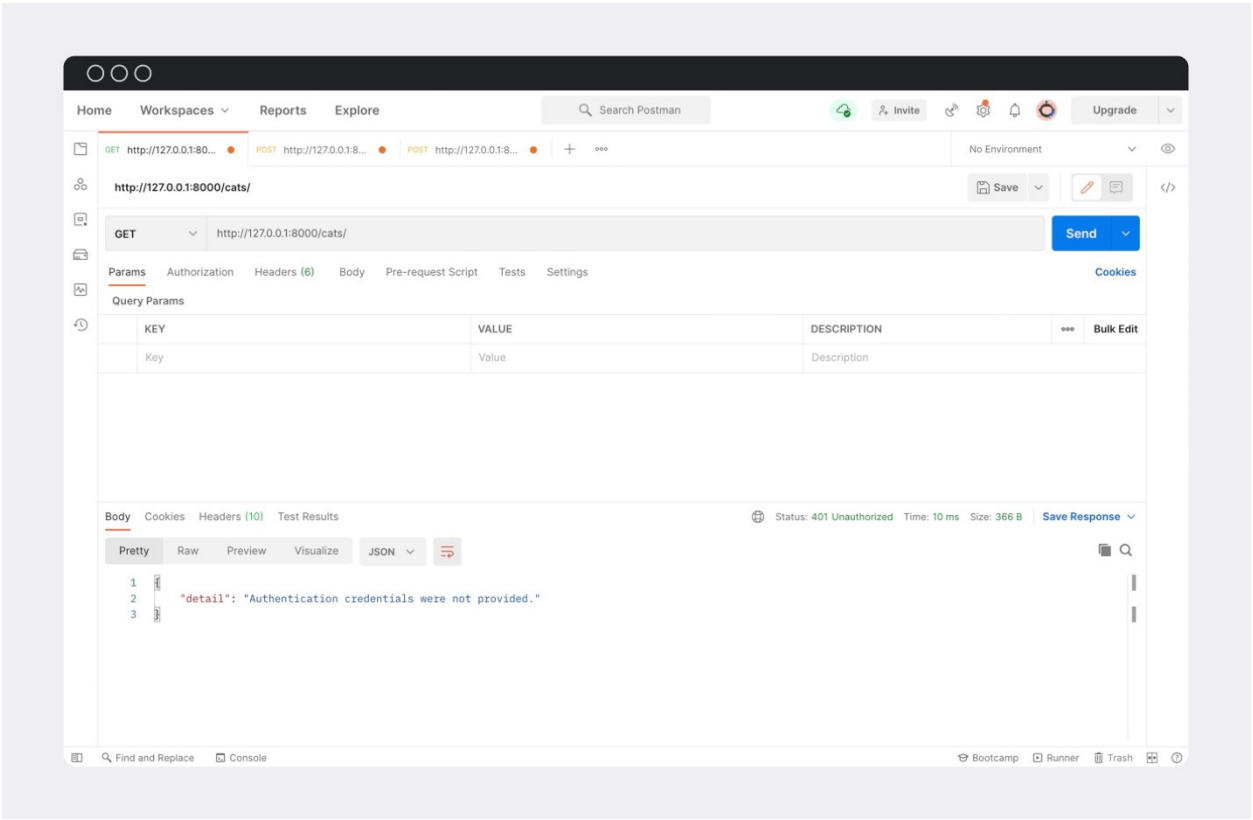


При последующих запросах к API после успешной аутентификации в объекте запроса будут доступны данные о пользователе:

- `request.user` — экземпляр пользователя;
- `request.auth` — токен (экземпляр класса `rest_framework.authtoken.models.Token`).

Этих данных не было в запросе, но **Authtoken** предоставляет их, основываясь на токене и соответствующей записи в базе данных. Их можно использовать, например, для того, чтобы сопоставить аутентифицированного пользователя с пользователем, который создавал соответствующий объект в БД.

В ответ на запрос без токена клиент получит отказ, статус-код `HTTP 401 Unauthorized`:



В случае указания недействительного токена (или токена с ошибкой) ответ будет таким:

Скопировать код
JSON

```
{
    "detail": "Invalid token."
}
```

Аутентификация по JWT-токену

Есть и более продвинутый способ аутентификации — по **JWT-токену**.

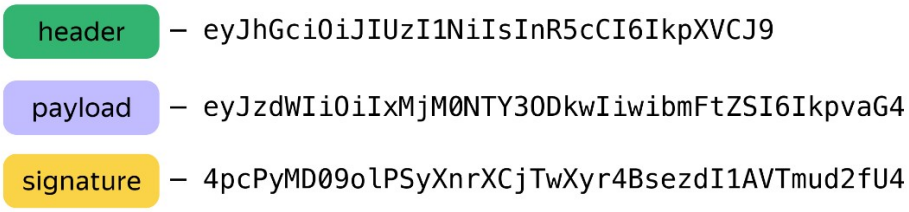
Преимущества JWT-токена в том, что прямо в нём записана информация о пользователе и сроке годности токена; системе не нужно каждый раз обращаться к базе данных, чтобы их сопоставить.

Токен, созданный по стандарту **JWT** (***J**SON **W**eb **T**oken*), состоит из трёх частей. Каждая из них записывается в формате JSON:

- **header** (англ. «заголовок») содержит служебную информацию;
- **payload** (англ. «полезная нагрузка») хранит основные данные токена;

- **signature** (англ. «подпись») — подпись, ключ безопасности для защиты информации.

После подготовки каждая из частей кодируется алгоритмом *Base64URL.* Получившиеся строки разделяются между собой точками:



Header JWT

Header, как правило, содержит два поля:

- алгоритм создания подписи — обычно применяется алгоритм *HMAC-SHA256* или *RSA;*
- тип токена — это строка `"JWT"`.

Скопировать код
JAVASCRIPT
```
{
"alg": "HS256",
"typ": "JWT"
}
```

Payload JWT

Payload хранит тип токена, timestamp со сроком его действия и информацию для аутентификации:

Скопировать код
JAVASCRIPT
```
{
  "token_type": "access",
  "exp":1578171903,
  "user_id":5
```

```
}
```

Signature JWT

Подпись гарантирует, что содержимое `header` и `payload` в токене не было изменено после создания. Специальный алгоритм генерирует подпись на основе содержимого `header` и `payload`. При кодировании этот алгоритм использует секретный ключ, который известен только серверу.

С этого момента в учебном проекте Kittygram вы будете работать именно с JWT-токенами. Для этого придётся отключить аутентификацию через Authtoken (если, конечно, вы её подключали).

Практика: подключите и настройте JWT-аутентификацию

Для работы с JWT в Django установите и подключите две библиотеки Djoser и Simple JWT:

Скопировать код
BASH

```
pip install djoser djangorestframework-simplejwt==4.7.2
```

Обновите файл settings.py:

Скопировать код
PYTHON

```
INSTALLED_APPS = (
    'django.contrib.auth',
    ...
    'rest_framework',
    'djoser',
)
```

Обратите внимание, приложение Djoser должно быть зарегистрировано **после** `django.contrib.auth` и `rest_framework`.

Добавьте новые настройки в settings.py, они сходны с настройками Authtoken:

- permission;
- способ аутентификации по умолчанию;

- минимально необходимые настройки модуля Simple JWT.

Скопировать код
PYTHON
```python
from datetime import timedelta


...

REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],

    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
}



SIMPLE_JWT = {
    # Устанавливаем срок жизни токена
   'ACCESS_TOKEN_LIFETIME': timedelta(days=1),
   'AUTH_HEADER_TYPES': ('Bearer',),
}
```

Проверьте, нет ли в проекте неприменённых миграций: `python manage.py migrate`.

Измените файл роутинга *urls.py*:

Скопировать код
PYTHON
```python
urlpatterns = [
    ...
    # Djoser создаст набор необходимых эндпоинтов.
    # базовые, для управления пользователями в Django:
    path('auth/', include('djoser.urls')),
    # JWT-эндпоинты, для управления JWT-токенами:
    path('auth/', include('djoser.urls.jwt')),
]
```

Готово. Но прежде чем продолжить — посмотрите на некоторые из эндпоинтов, которые стали доступны в API.

Эндпоинт	Описание
`/auth/users/`	Базовый: зарегистрировать нового пользователя
`/auth/users/me/`	Базовый: получить/обновить зарегистрированного пользователя
`/auth/jwt/create/`	Создать JWT-токен
`/auth/jwt/refresh/`	Получить новый JWT по истечении времени жизни ранее сгенерированного

Хорошая новость заключается в том, что djoser сделал за вас большую часть тяжёлой работы: он сгенерировал эндпоинты, управляющие токенами и пользователями через API.

Список всех эндпоинтов, которые создаёт **djoser**, есть в документации.

Для обработки добавленных эндпоинтов **djoser** использует собственные вьюсеты и сериализаторы.

Если нужно изменить набор полей сериализатора из **djoser**, то

- из `djoser.serializers` импортируется класс сериализатора, который нужно переопределить (например, `UserSerializer` или `UserCreateSerializer`; полный список сериализаторов **djoser** доступен в документации);
- описывается новый класс сериализатора (он наследуется от импортированного);
- в новом сериализаторе переопределяется набор полей, используемых по умолчанию.

Скопировать код
PYTHON

```python
from djoser.serializers import UserSerializer
...

class CustomUserSerializer(UserSerializer):
    class Meta:
```

```
        model = User
        fields = ('email', 'id', 'username', 'first_name', 'last_name')
```

Аналогично можно поступить и с вьюсетом, который использовал сериализатор.

Скопировать код
PYTHON

```python
from djoser.views import UserViewSet
...
from .serializers import CustomUserSerializer
...

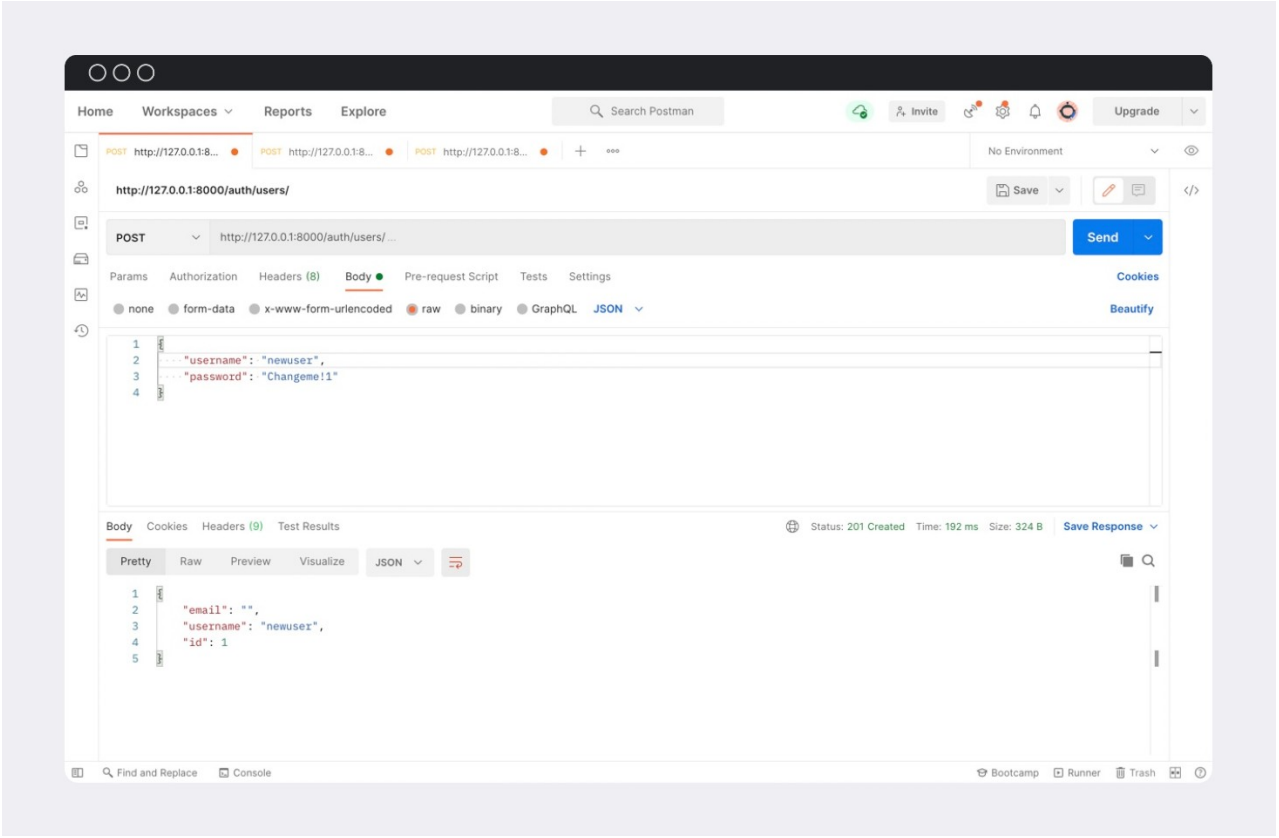

class CustomUserViewSet(UserViewSet):
    ...
```

Применение JWT на практике

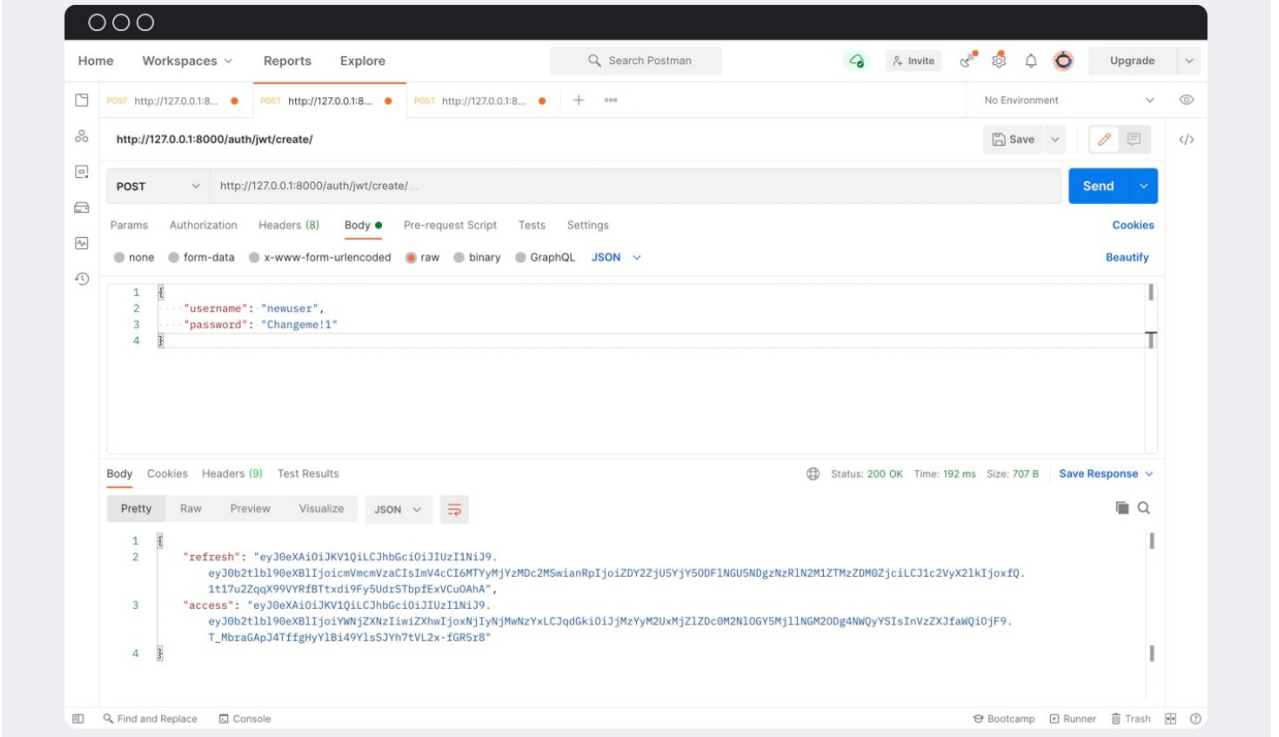
Теперь пользователя можно создать через API.

Придумайте новую пару «логин-пароль» и отправьте POST-запрос на http://127.0.0.1:8000/auth/users/, передав их в полях `username` и `password`.

Если вы всё сделали правильно, вам вернётся HTTP-ответ со статус-кодом **201 Created**:



Теперь можно получить токен: отправьте POST-запрос на эндпоинт /auth/jwt/create/, передав действующий логин и пароль в полях `username` и `password`.



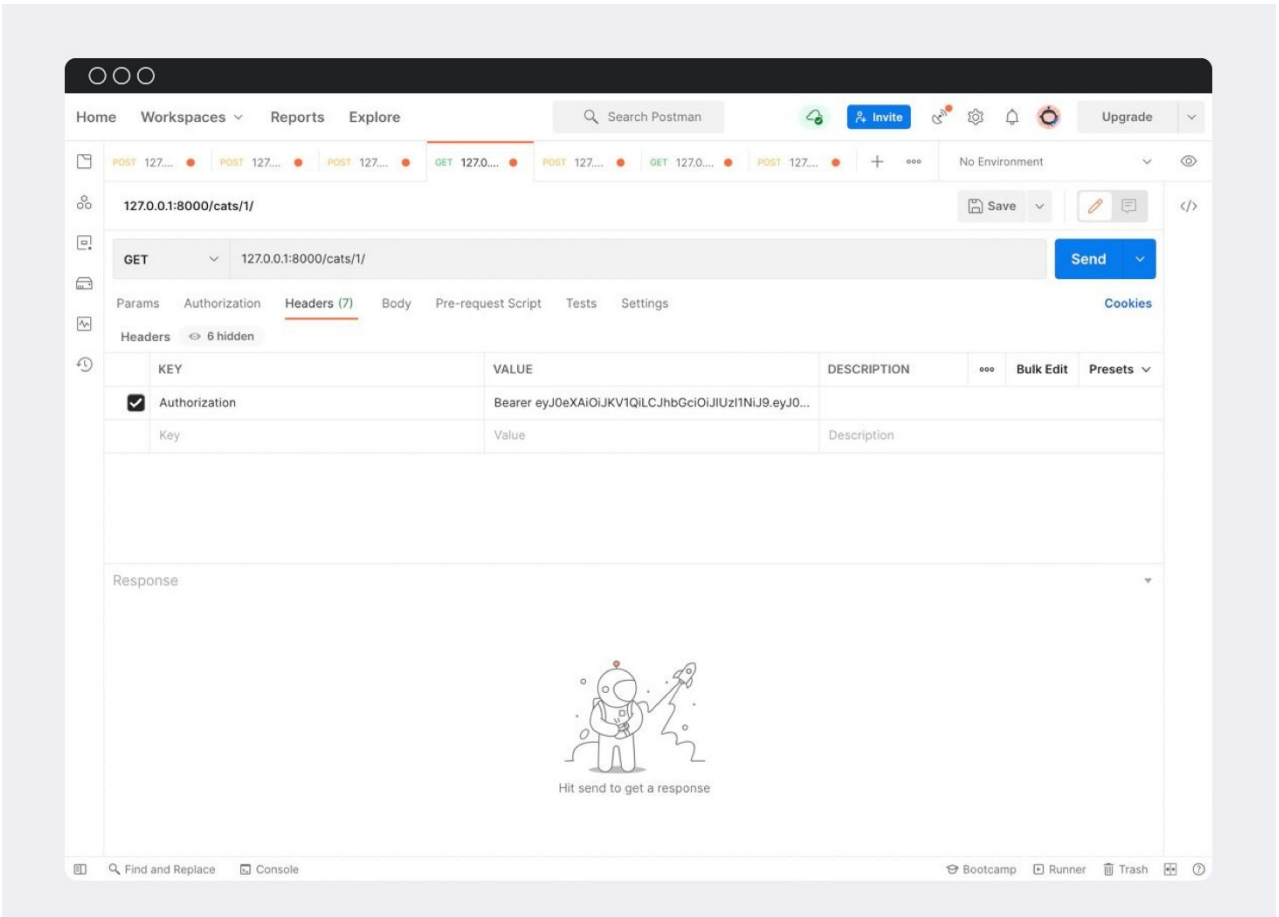
API вернёт JWT-токен:

Скопировать код
JSON

```
# Пример ответа
{
    "refresh":
"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0b2tlbl90eXBlIjoicmVmcmVzaCIsImV4cCI6MTYyMDk
0MTQ3NywianRpIjoiODUzYzE5MTg5NzMwNDQwNTk1ZjI3ZTBmOTAzZDcxZDEiLCJ1c2VyX2lkIjoxfQ.0vJBP
IUZG4MjeU_Q-mhr5Gqjx7sFlO6AShlfeINK8nA",
    "access":
"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0b2tlbl90eXBlIjoiYWNjZXNzIiwiZXhwIjoxNjIwODU
1Mzc3LCJqdGkiOiJkY2EwNmRiYTEzNWQ0ZjNiODdiZmQ3YzU2Y2ZjNGE0YiIsInVzZXJfaWQiOjF9.eZfkpeN
VfKLzBY7U0h5gMdTwUnGP3LjRn5g8EIvWlVg"
}
```

Токен вернётся в поле `access`, а данные из поля `refresh` пригодятся для обновления токена.

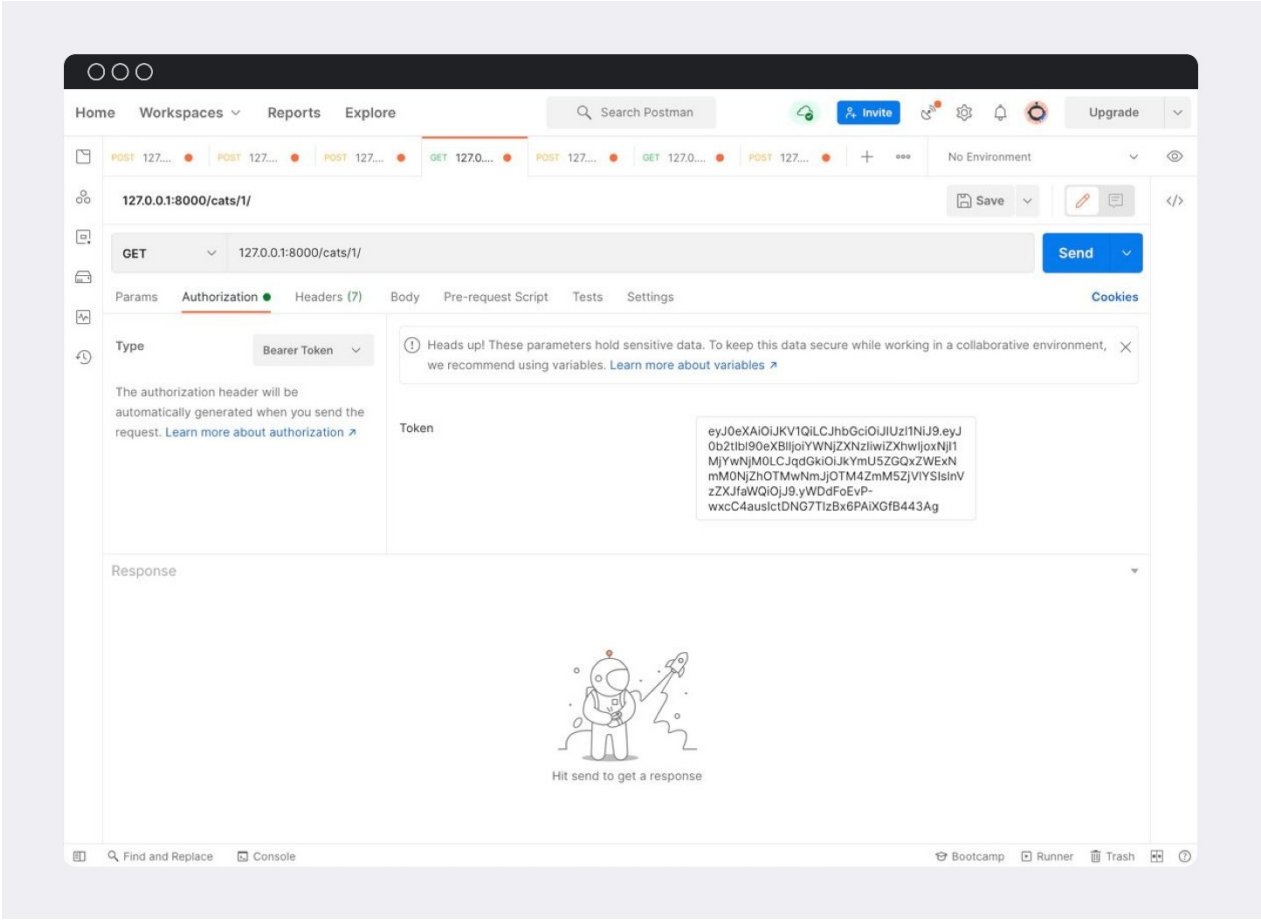
При каждом запросе к API нужно в заголовке запроса, в поле **Authorization,** передавать основной токен доступа, полученный в поле `access`. Перед самим токеном должно стоять ключевое слово `Bearer` и пробел: `Bearer токен`



Слово `Bearer` (англ. «носитель») здесь заменяет слово *Token* и означает, что за ним следует сам токен.

Если основной токен доступа утрачен, украден или каким-то иным образом скомпрометирован, его можно отключить и получить новый. Для этого нужно отправить POST-запрос на тот же адрес `/auth/jwt/create/`, а в теле запроса, в поле `refresh`, передать refresh-токен — тот самый, который был получен в поле `refresh` при запросе токена.

В Postman существует возможность передавать токен и по-другому: можно выбрать соответствующий тип авторизации во вкладке **Authorization** и указать JWT-токен уже там.



Сохраните ссылки на документацию рассмотренных библиотек. Вы с ними обязательно встретитесь в ближайшем будущем.

- Simple JWT
- Djoser

Как вам урок?

Перейти к заданию

Задание

Добавьте в проект Yatube API механизм аутентификации при помощи обычного токена, реализованный на view-функциях.

Сделайте так, чтобы при создании нового поста через POST-запрос в поле *author* автоматически записывался объект автора по токену.

Вам нужно:

- Сделать доступ к API только после авторизации; опишите эндпоинт *api/v1/api-token-auth/*, по которому можно будет получать токен.
- В сериализаторе настроить поле `author` в режим «только для чтения».
- При создании публикации в поле «автор» указать пользователя, полученного из объекта `request.user`.
- Описать обработчик GET-запроса для получения поста в функции `api_posts_detail`.
- Разрешить изменение или удаление постов только авторам этих постов, в противном случае возвращать статус 403.

```
from rest_framework import serializers
from .models import Post


class PostSerializer(serializers.ModelSerializer):
    class Meta:
        fields = ('id', 'text', 'author', 'image', 'pub_date')
        # укажите поля, доступные только для чтения
        ...
        model = Post
```

Kittygram 2: новые возможности

Kittygram серьёзно подрос, обзавёлся новыми возможностями и стал удобнее. А тут поступила новая задача: новые пользователи хотят самостоятельно регистрироваться в нашем сервисе. Мало того: нужно настроить проект так, чтобы добавлять, обновлять и удалять информацию о котиках могли только их хозяева.

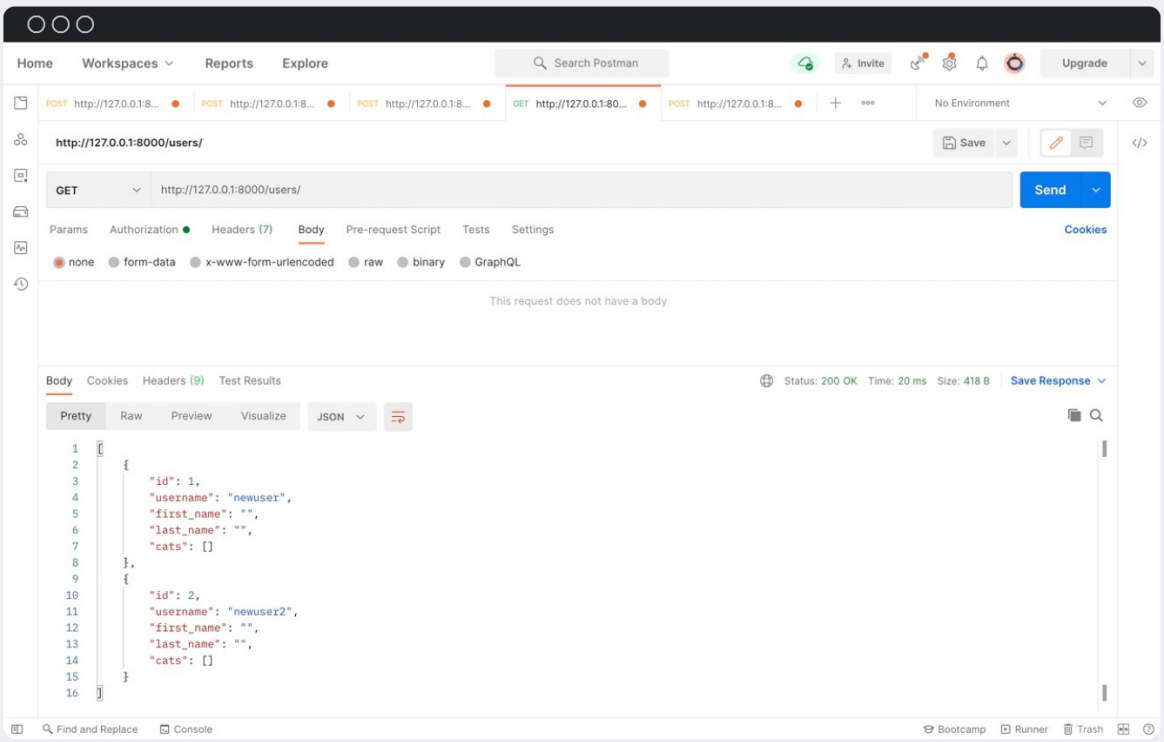
Судя по всему, нового релиза не избежать. Ну что ж, пусть будет Kittygram v2.0.



Релиз Kittygram v2.0

Склонируйте новую версию проекта, выполните миграции, запустите веб-сервер, создайте через API несколько пользователей и получите для них JWT-токены.

Проверьте, что пользователи появились в БД и котиков у них пока нет: сделайте несколько запросов через Postman.



Каждый релиз принято сопровождать обзором нововведений. Релиз второй версии Kittygram тоже заслуживает такого обзора.

Что новенького?

Начнём с моделей: **Owner** больше не понадобится, вместо неё будем использовать встроенную модель **User**.

Скопировать код
PYTHON

```python
#cats/models.py
from django.contrib.auth import get_user_model
from django.db import models

CHOICES = (
    ('Gray', 'Серый'),
    ('Black', 'Чёрный'),
    ('White', 'Белый'),
    ('Ginger', 'Рыжий'),
    ('Mixed', 'Смешанный'),
)
```

```python
User = get_user_model()


class Achievement(models.Model):
    name = models.CharField(max_length=64)

    def __str__(self):
        return self.name


class Cat(models.Model):
    name = models.CharField(max_length=16)
    color = models.CharField(max_length=16, choices=CHOICES)
    birth_year = models.IntegerField()
    owner = models.ForeignKey(
        User, related_name='cats', on_delete=models.CASCADE)
    achievements = models.ManyToManyField(Achievement, through='AchievementCat')

    def __str__(self):
        return self.name


class AchievementCat(models.Model):
    achievement = models.ForeignKey(Achievement, on_delete=models.CASCADE)
    cat = models.ForeignKey(Cat, on_delete=models.CASCADE)

    def __str__(self):
        return f'{self.achievement} {self.cat}'
```

В новой версии проекта работать будем исключительно с вьюсетами, так удобнее.

Аутентифицированные пользователи не должны иметь возможность удалять или изменять данные друг друга, поэтому вьюсет, отвечающий за работу с пользователями, унаследован от `ReadOnlyModelViewSet`.

Скопировать код
PYTHON

```python
# cats/views.py
from rest_framework import viewsets

from .models import Achievement, Cat, User
```

```python
from .serializers import AchievementSerializer, CatSerializer, UserSerializer


class CatViewSet(viewsets.ModelViewSet):
    queryset = Cat.objects.all()
    serializer_class = CatSerializer


class UserViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer


class AchievementViewSet(viewsets.ModelViewSet):
    queryset = Achievement.objects.all()
    serializer_class = AchievementSerializer
```

А чтобы дать возможность пользователям самостоятельно регистрироваться через API и обеспечить доступ по токену, воспользуемся связкой JWT+Djoser.

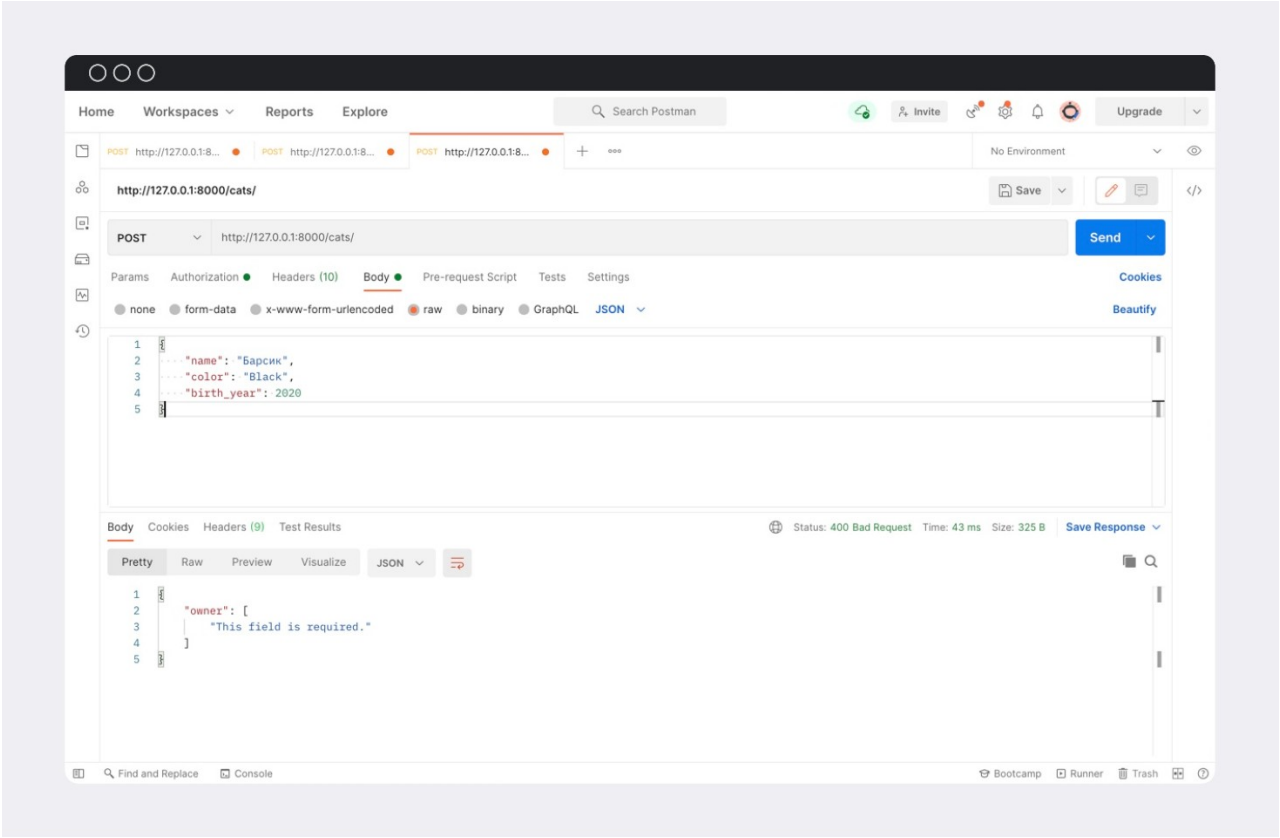
Добавление данных через API

Самое время наполнить базу данных котиками. Теперь это могут сделать только аутентифицированные пользователи.

Выполните POST-запрос на эндпоинт `cats/`; анонимные запросы к проекту теперь запрещены, так что передайте с запросом токен одного из пользователей; в теле запроса передайте всю необходимую информацию о домашнем питомце:

Скопировать код
PYTHON

```python
{
    "name": "Барсик",
    "color": "White",
    "birth_year": 2011
}
```



Запрос не обработался, ведь мы не передали в запросе id владельца котика, а это обязательное поле в сериализаторе `CatSerializer`. Исправить это не трудно, можно просто добавить в тело запроса id текущего пользователя.

Скопировать код
PYTHON

```
{
    "name": "Барсик",
    "color": "Black",
    "birth_year": 2020,
    "owner": 1
}
```

Если указать **id** другого пользователя (из списка существующих), то такой запрос тоже пройдёт. А вот это уже не очень хорошо, ведь добавлять, удалять и изменять информацию о котике может только хозяин. А пока что пользователь с id=3 может добавить котика от имени пользователя с id=1. Выглядит как подлог.

Самый простой и очевидный способ исправить ситуацию — получать владельца котика не из поля `owner` в теле запроса, а из объекта `request`: там доступен экземпляр пользователя, которому принадлежит токен.

Нечто подобное вы делали при работе с формами. А сериализатору надо сказать, чтобы он игнорировал это поле, если оно будет указано в теле POST-запроса.

Начнём с последнего пункта: переопределим поле `owner` и укажем параметр `read_only`.

Скопировать код
PYTHON

```
class CatSerializer(serializers.ModelSerializer):
    achievements = AchievementSerializer(read_only=True, many=True)
    color = serializers.ChoiceField(choices=CHOICES)
    owner = serializers.PrimaryKeyRelatedField(read_only=True)
    age = serializers.SerializerMethodField()

    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year','achievements', 'owner',
                  'age')


    ...
```

Параметр `read_only_fields` можно указать не в описании поля, а во внутреннем классе `Meta`: так удобнее — это позволяет перечислить все поля для чтения в одном месте, не переопределяя их.

Скопировать код
PYTHON

```
class CatSerializer(serializers.ModelSerializer):
    achievements = AchievementSerializer(read_only=True, many=True)
    color = serializers.ChoiceField(choices=CHOICES)
    age = serializers.SerializerMethodField()

    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year', 'achievements', 'owner',
                  'age')
        read_only_fields = ('owner',)

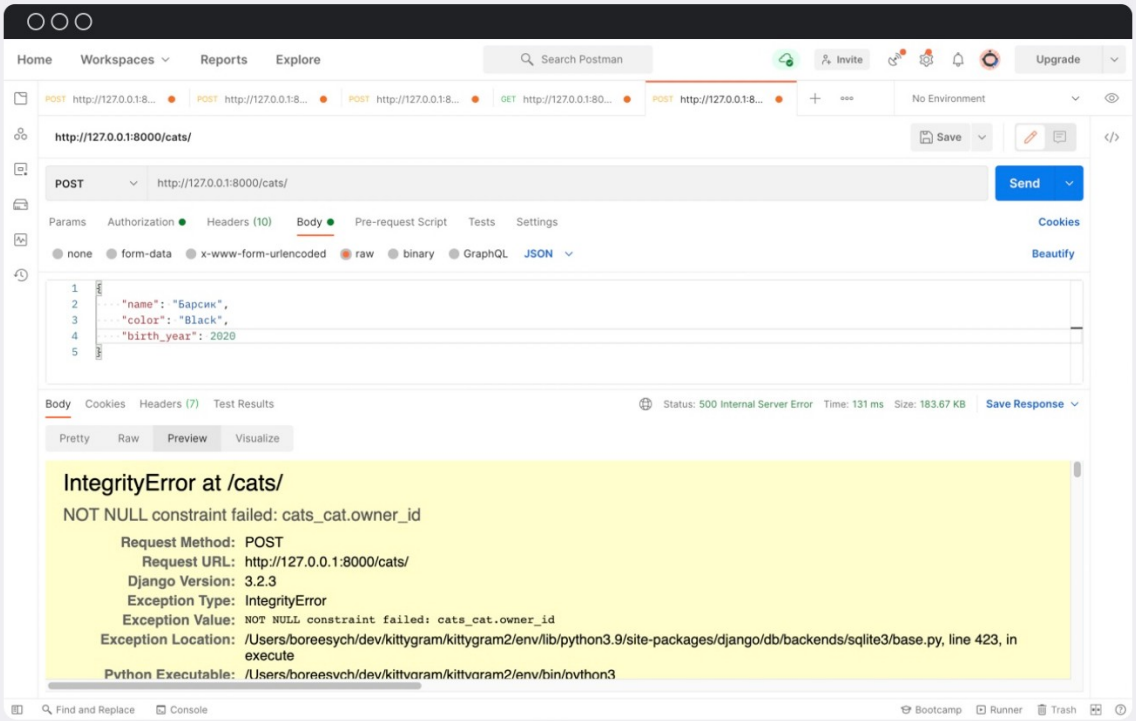

    ...
```

Теперь сериализатор не ждёт в теле POST-запроса поле `owner` (а если оно придёт, то будет проигнорировано).

Передача собственного значения в метод save()

Проблему с подделкой id пользователя решили, но легче не стало.

Теперь вьюсет не может создать новую запись в БД, ведь поле `owner` в модели **Cat** обязательное, а данные из сериализатора в него не приходят: при POST-запросах сериализатор игнорирует поле `owner`.



Задача состоит в том, чтобы при создании новой записи в БД поле `owner` не осталось пустым, а в него была записана информация о пользователе, отправившем запрос. Эта информация у нас есть — она доступна в `request.user`, осталось в нужный момент подсунуть её в базу.

Новая запись в БД создаётся при вызове метода `save()` сериализатора, а этот метод вызывается из метода вьюсета `perform_create()`.

Чтобы передать новое значение для какого-то поля в метод `save()`, нужно переопределить метод `perform_create()`.

В метод `save()` в поле `owner` передадим объект пользователя, отправившего запрос.

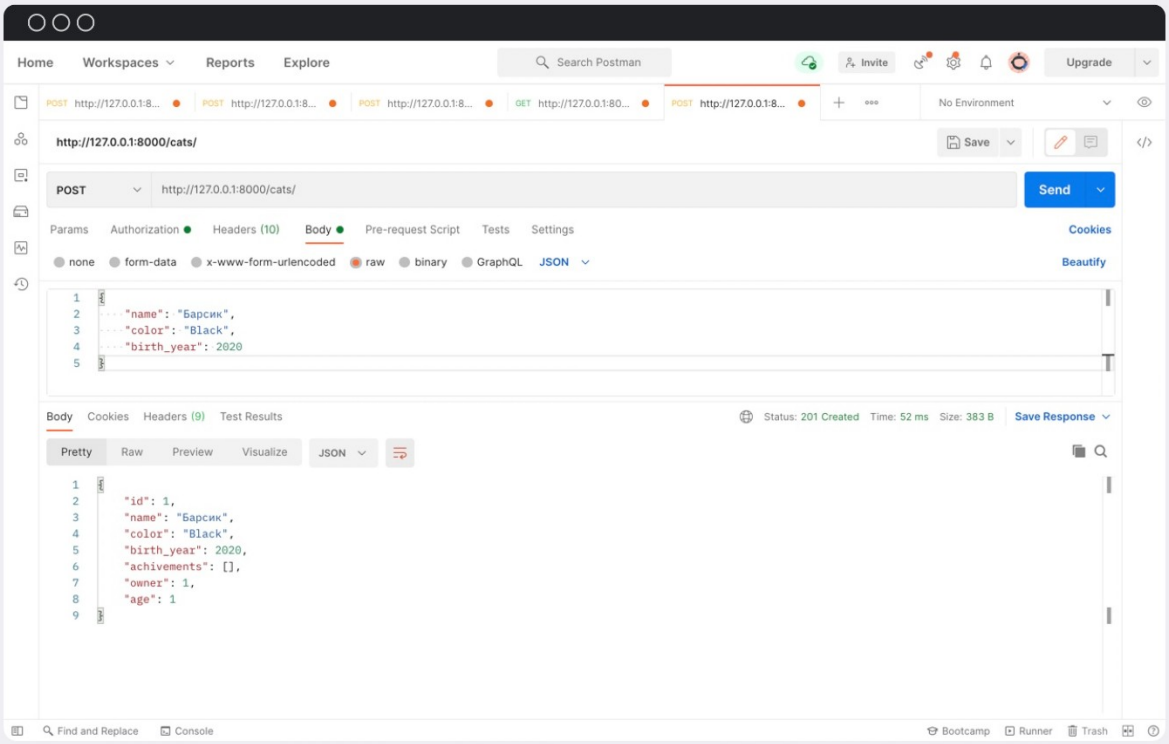
Скопировать код
PYTHON

```
class CatViewSet(viewsets.ModelViewSet):
```

```
    queryset = Cat.objects.all()
    serializer_class = CatSerializer

    def perform_create(self, serializer):
        serializer.save(owner=self.request.user)
```

Вот теперь всё работает как надо: сериализатор не ожидает id пользователя в POST-запросе (или игнорирует его при получении), а при создании записи о новом котике в БД информация о пользователе берётся из объекта `request.user`.



При подобных операциях с PUT- и PATCH-запросами следует переопределить метод `perform_update()`, а в остальном всё работает так же.

Валидаторы

По умолчанию валидация в сериализаторе происходит на базовом уровне. Если в поле ожидается целочисленное значение, а пришло число с плавающей точкой, то сериализатор вернёт ошибку.

А вот если в запросе придёт значение, которое по типу соответствует ожиданиям, но не удовлетворяет бизнес-логике или здравому смыслу (например, год рождения — в будущем), то ошибка не возникнет.

Валидация на уровне поля

Допустим, к проекту Kittygram отправлен POST-запрос, в котором переданы некорректные данные: например, год рождения котика указан в будущем или, наоборот, в далёком прошлом (а котики, к сожалению, не живут более сорока лет). Такие данные принимать нельзя, значит, надо провести валидацию полученных значений.

Проверку можно провести на уровне отдельно взятого поля. Сделаем это на примере поля `birth_year`.

Метод для валидации определённого поля должен называться `validate_<имя поля>`, он будет автоматически вызываться при получении данных.

Опишите в `CatSerializer` новый метод `validate_birth_year()`.

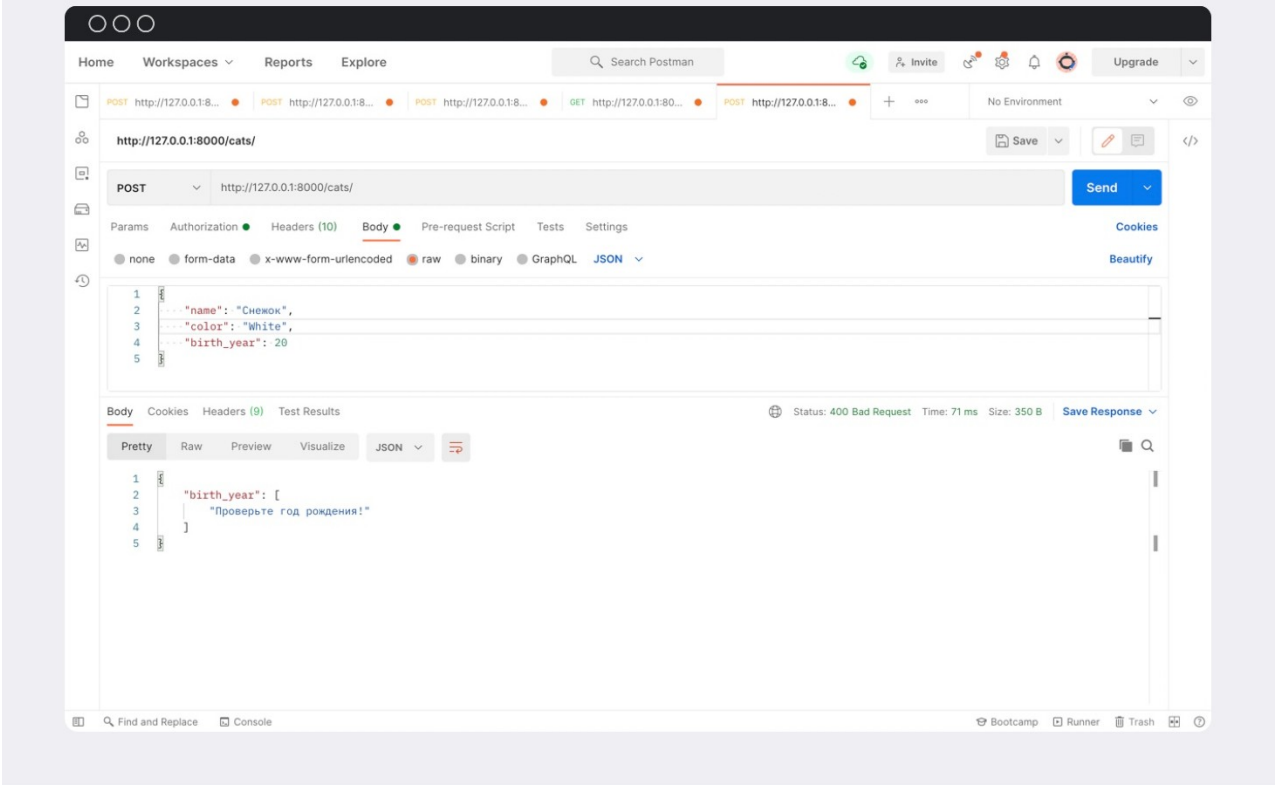
Скопировать код
PYTHON

```
class CatSerializer(serializers.ModelSerializer):
    achievements = AchievementSerializer(many=True, required=False)
    color = serializers.ChoiceField(choices=CHOICES)
    age = serializers.SerializerMethodField()
    owner = serializers.PrimaryKeyRelatedField(read_only=True)

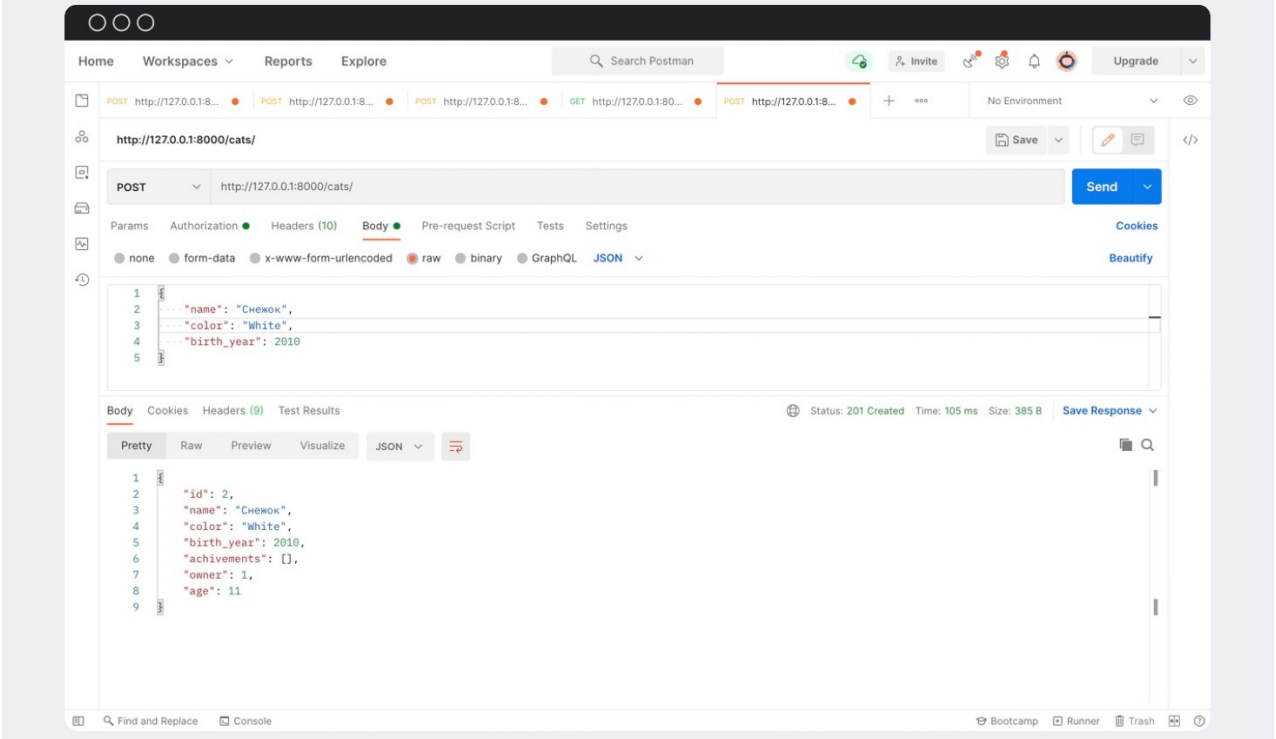
    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year',
                  'achievements', 'owner', 'age')

    def validate_birth_year(self, value):
        year = dt.date.today().year
        if not (year - 40 < value <= year):
            raise serializers.ValidationError('Проверьте год рождения!')
        return value
```

Проверьте результат: если всё сделано верно, то добавить котика, родившегося в двадцатом году нашей эры, будет невозможно:



Если год указан в нужном диапазоне, проблем не возникнет:



Ещё один важный момент: этот метод не только проверяет данные, но и возвращает значение.

Дело здесь в том, что DRF позволяет не только проверять данные, но и нормализовать их, приводить к нужному виду — например, к верхнему или нижнему регистру. Здесь эта возможность не используется, но DRF требует вернуть значение, и поэтому функция возвращает то, что сама получила на вход, — аргумент `value`.

Валидация на уровне объекта

Метод `validate_birth_year` проверяет одно-единственное поле `birth_year` и ничего не знает об остальных данных.

Однако значения отдельных полей могут по отдельности успешно проходить все валидации, а вот их совокупность не всегда бывает валидной.

Например, если в POST-запросе значения полей `name` и `color` совпали — это почти наверняка ошибка: редко бывает так, что имя котика совпадает с его цветом. Чтобы выявить эту ошибку, понадобится сравнить значения этих полей.

Чтобы выполнить проверку, требующую доступа к нескольким полям, добавьте в сериализатор метод `validate()`. Этот метод принимает один аргумент — словарь со значениями полей.

Скопировать код
PYTHON

```python
class CatSerializer(serializers.ModelSerializer):
    achievements = AchievementSerializer(many=True, required=False)
    color = serializers.ChoiceField(choices=CHOICES)
    age = serializers.SerializerMethodField()
    owner = serializers.PrimaryKeyRelatedField(read_only=True)

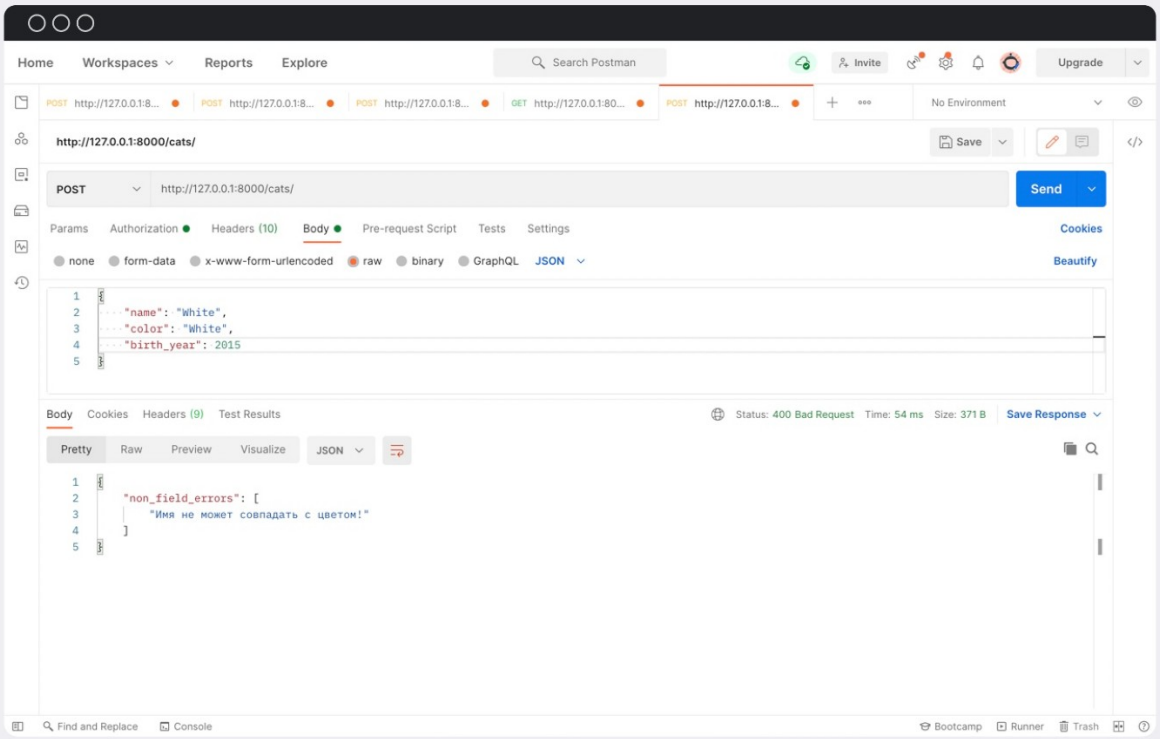
    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year',
            'achievements', 'owner', 'age')

    ...

    def validate(self, data):
        if data['color'] == data['name']:
```

```
            raise serializers.ValidationError(
                'Имя не может совпадать с цветом!')
        return data
```

Теперь ошибочные записи создаваться не будут, а владелец белого кота по кличке White получит сообщение об ошибке.



Встроенные валидаторы для сериализаторов

Для сериализаторов в DRF есть несколько встроенных классов-валидаторов, среди них есть `UniqueValidator` и `UniqueTogetherValidator`.

- **UniqueValidator** обеспечивает проверку уникальности значения поля.

  В моделях такое ограничение описывается параметром `unique=True` для поля.

- **UniqueTogetherValidator** обеспечивает проверку уникальности комбинации (например, если проверка уникальности идёт по полям `name` и `color`, в базе никогда не окажется несколько записей с `name='Мурзик'` и `color='Black'`, пусть даже остальные поля будут у них различаться).

Разработчики Kittygram 2.0 не хотят, чтобы забывчивые хозяева котиков дублировали записи о своих любимцах в БД. Поэтому реализовали проверку уникальности записей о котиках на примере двух полей: `owner` и `name`. В базе данных не должно быть двух или более записей, у которых имя котика и хозяин совпадают.

Эти ограничения реализуются на уровне модели: они описываются в классе Meta. Например, через атрибут `unique_together`:

Скопировать код
PYTHON

```python
class Cat(models.Model):
    name = models.CharField(max_length=16)
    color = models.CharField(max_length=16, choices=CHOICES)
    birth_year = models.IntegerField()
    owner = models.ForeignKey(
        User, related_name='cats', on_delete=models.CASCADE)
    achievements = models.ManyToManyField(Achievement, through='AchievementCat')

    class Meta:
        unique_together = ('name', 'owner')

    def __str__(self):
        return self.name
```

Но документация Django рекомендует вместо `unique_together` использовать `UniqueConstraint`: этот способ обеспечивает большую функциональность, а `unique_together` может быть признан устаревшим в будущем.

Скопировать код
PYTHON

```python
class Cat(models.Model):
    name = models.CharField(max_length=16)
    color = models.CharField(max_length=16, choices=CHOICES)
    birth_year = models.IntegerField()
    owner = models.ForeignKey(
        User, related_name='cats', on_delete=models.CASCADE)
    achievements = models.ManyToManyField(Achievement, through='AchievementCat')

    class Meta:
        constraints = [
```

```
            models.UniqueConstraint(
                fields=['name', 'owner'],
                name='unique_name_owner'
            )
        ]

    def __str__(self):
        return self.name
```

Такую же проверку нужно реализовать и на уровне сериализатора.

В классе `Meta` сериализатора нужно указать опциональное поле `validators`, значением которого будет список валидаторов. Сейчас нужен только один валидатор: **UniqueTogetherValidator**.

У него есть два обязательных аргумента и один необязательный:

- `queryset`, обязательный: это queryset, для которого должна быть проверена уникальность.
- `fields`, обязательный: список или кортеж имён полей сериализатора, которые должны составлять уникальный набор.
- `message`: сообщение об ошибке на случай, если данные не прошли валидацию.

Скопировать код
PYTHON

```
class CatSerializer(serializers.ModelSerializer):
    achievements = AchievementSerializer(many=True, required=False)
    color = serializers.ChoiceField(choices=CHOICES)
    age = serializers.SerializerMethodField()
    owner = serializers.PrimaryKeyRelatedField(read_only=True)

    class Meta:
        model = Cat
        fields = ('id', 'name', 'color', 'birth_year', 'achievements', 'owner',
                  'age')

        validators = [
            UniqueTogetherValidator(
                queryset=Cat.objects.all(),
                fields=('name', 'owner')
```

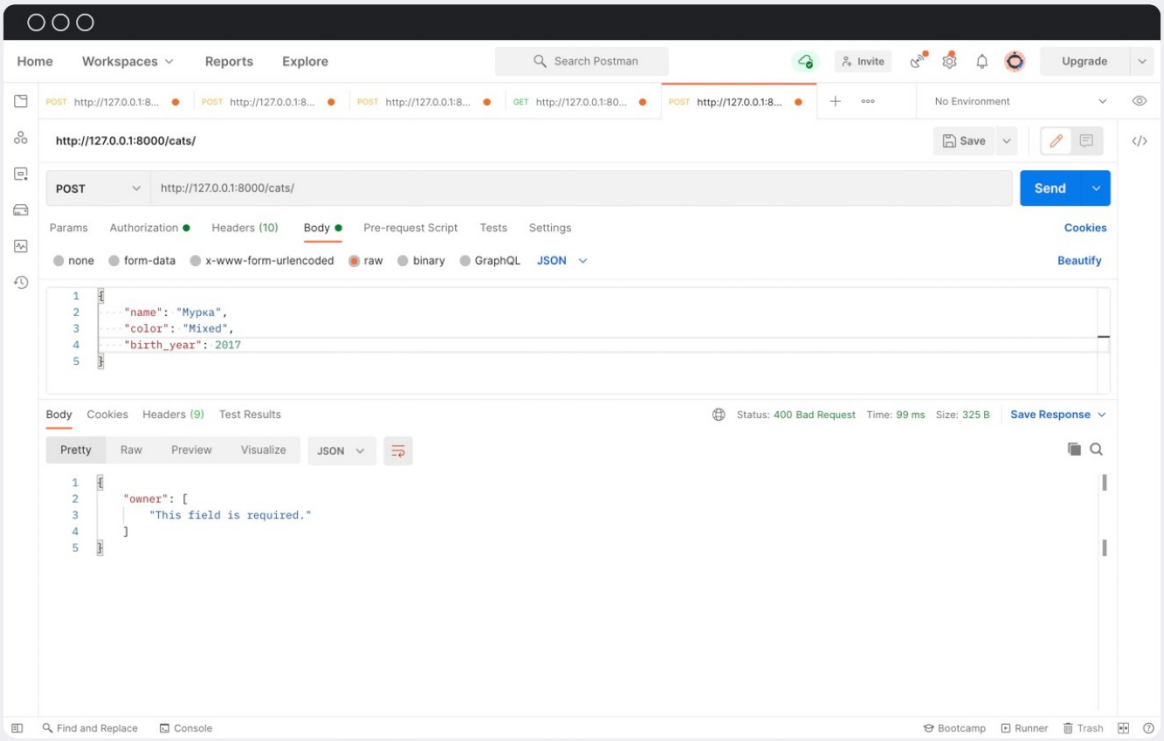
```
            )
        ]


    ...
```

Если при работе с `ModelSerializer` в целевой модели указан параметр `unique_together`, то валидатор `UniqueTogetherValidator` в сериализаторе можно не прописывать — он будет применён автоматически.

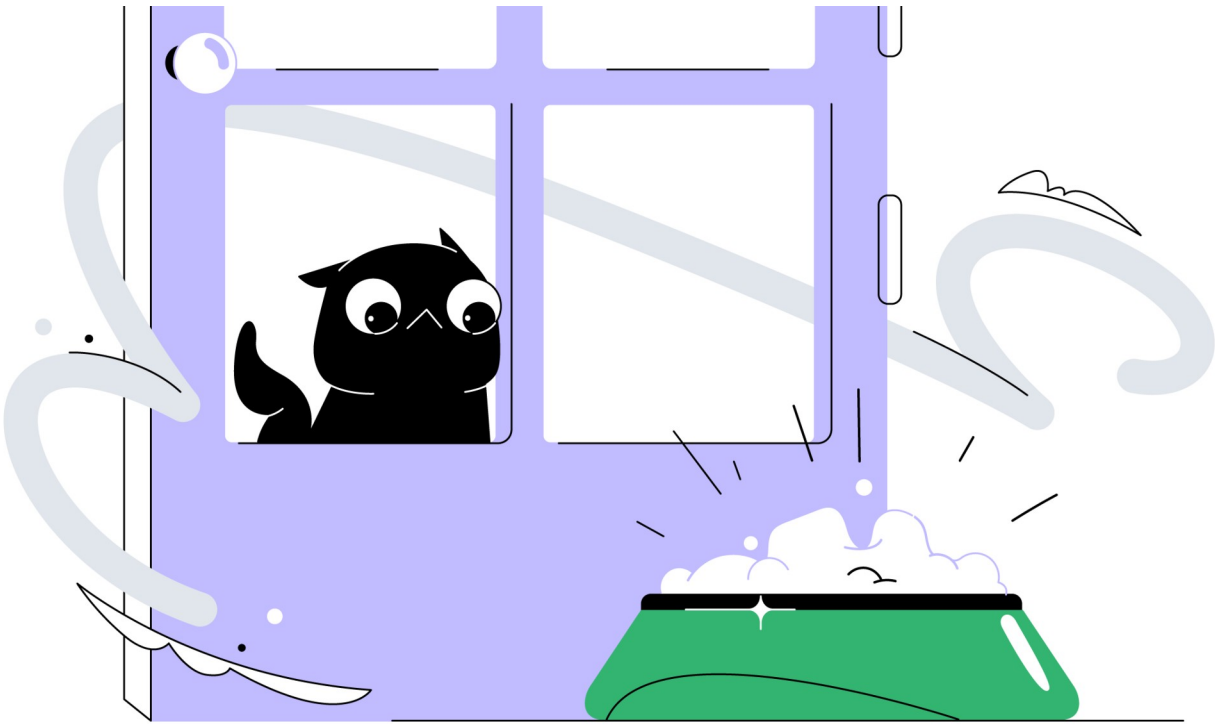
Если же требуется осуществлять подобную проверку только на уровне сериализатора, не устанавливая ограничения в модели, — в сериализаторе нужно явно указывать необходимые валидаторы.

После изменений в модели создадим и применим миграции — и сделаем запрос на добавление нового котика. Начнём с такого, которого точно нет в БД.



Ошибка! Странное дело: DRF сообщает, что поле `owner` — *is required*, обязательное. Как так, мы же только что назначили его необязательным!

Фокус ещё и в том, что если в перечне полей валидатора вместо поля `owner` указать какое-либо другое поле из модели, то всё будет работать: валидатор всё проверит, записи создадутся. А вот с полем `owner` — нет.



Давайте разбираться что тут происходит.

Поле `owner` — необязательное, оно может отсутствовать в запросе (либо будет игнорироваться, если присутствует). Мы переопределили метод `save()`, теперь туда явным образом передаются данные для поля `owner`. Это происходит на этапе создания нового экземпляра класса.

Но этому этапу предшествует запуск валидаторов, и в этот момент сериализатор ещё не знает о новом содержимом поля `owner`: оно там просто отсутствует. Но валидатору это поле нужно: без него он не может выполнить проверку.

Выйти из сложившейся ситуации помогут дополнительные опции сериализатора: **скрытые поля** и **значения по умолчанию**.

Скрытые поля и значения по умолчанию

Для полей сериализатора можно указать значения по умолчанию. В этом случае, если в запросе будет отсутствовать требуемое поле, будет использовано заранее указанное значение.

Для поля `owner` можно указать значение по умолчанию, передав в него объект пользователя, отправившего запрос. Для определения пользователя есть встроенный класс `CurrentUserDefault` — именно его и нужно указать в качестве значения по умолчанию для поля `owner`.

Существует два варианта использования значений по умолчанию в сериализаторе для решения возникшей задачи.

1. Можно переопределить поле `owner`, указав ему тип `HiddenField` (скрытое поле), и передать ему значение по умолчанию `CurrentUserDefault`.

   `HiddenField` не принимает передаваемые в запросе данные и всегда возвращает значение по умолчанию в словарь **validated_data** в сериализаторе.

Скопировать код
PYTHON
```
class CatSerializer(serializers.ModelSerializer):
    ...
    owner = serializers.HiddenField(default=serializers.CurrentUserDefault())
    ...
```

В этом случае поле `owner` всегда будет присутствовать в словаре `validated_data` (и больше нет необходимости передавать его значение в метод `save()` во вьюсете), **но его нельзя будет использовать в ответах сериализатора**, даже если это поле будет явно указано в списке необходимых полей: оно скрытое, такие поля не попадают в ответ.

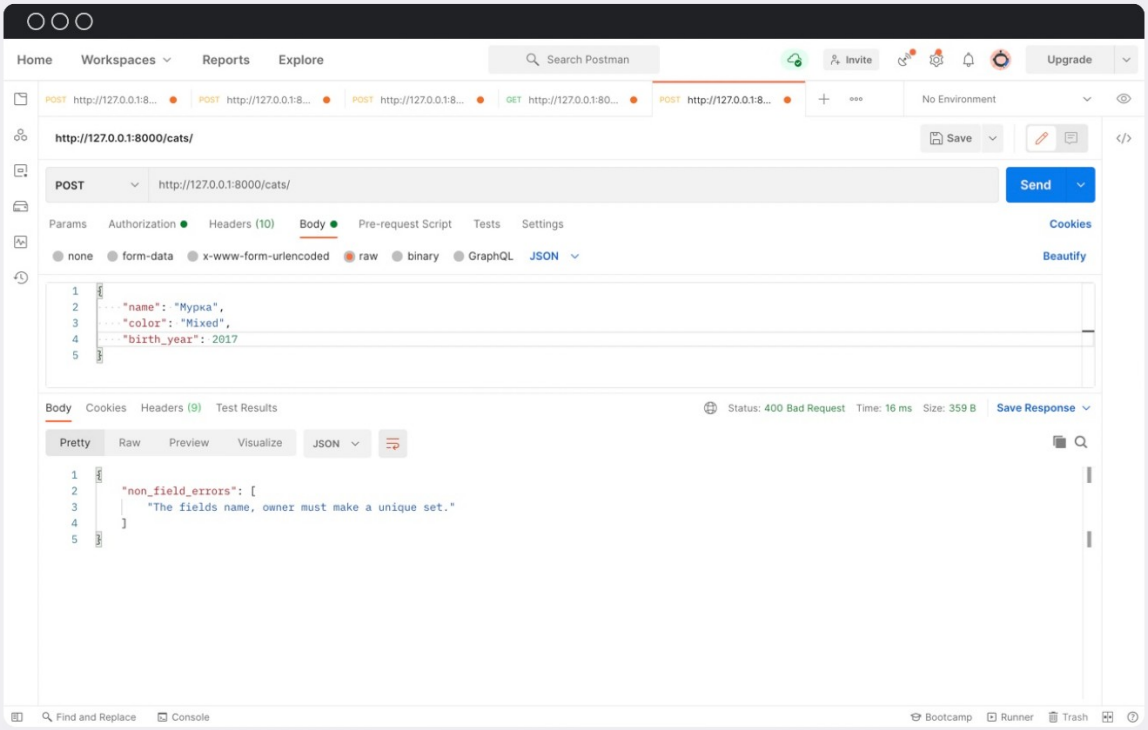
2. Можно переопределить поле `owner`, указав ему любой стандартный тип поля, но обязательно передать параметры `read_only=True` и `default=<значение_по_умолчанию>` (для поля `owner` это будет `default=serializers.CurrentUserDefault()`).

Скопировать код
PYTHON
```
class CatSerializer(serializers.ModelSerializer):
    ...
    owner = serializers.PrimaryKeyRelatedField(
        read_only=True, default=serializers.CurrentUserDefault())
    ...
```

В этом варианте поле `owner` может быть использовано на выходе сериализатора, но без передачи его значения в метод `save()` во вьюсете тут уже не обойтись.

В обоих случаях проблема будет решена, новые записи начнут добавляться в БД после обработки запросов и валидация `UniqueTogetherValidator` будет работать корректно.

Попробуем дважды добавить одного и того же котика; при попытке добавить его второй раз получим сообщение об ошибке:



Важно: класс `UniqueTogetherValidator` всегда накладывает неявное ограничение: все поля сериализатора, к которым применён этот валидатор, обрабатываются как обязательные. Поля со значением `default` — исключение: они всегда предоставляют значение, даже если пользователь не передал их в запросе.

Как вам урок?

К следующей теме